

JENKINS-TASK-04

1. Setup Jenkins CICD pipeline using freestyle job using Docker containers using below code.

Source Code: <https://github.com/betawins/hiring-app.git>

1) Stages:

- Git Clone
- SonarQube Integration
- Maven Compilation
- Nexus Artifactory
- Slack Notification
- Deploy On tomcat

The final flow will be:

GitHub → Jenkins Freestyle → Maven Compile/Test → SonarQube → Nexus → Tomcat → Slack

1. What you need before starting

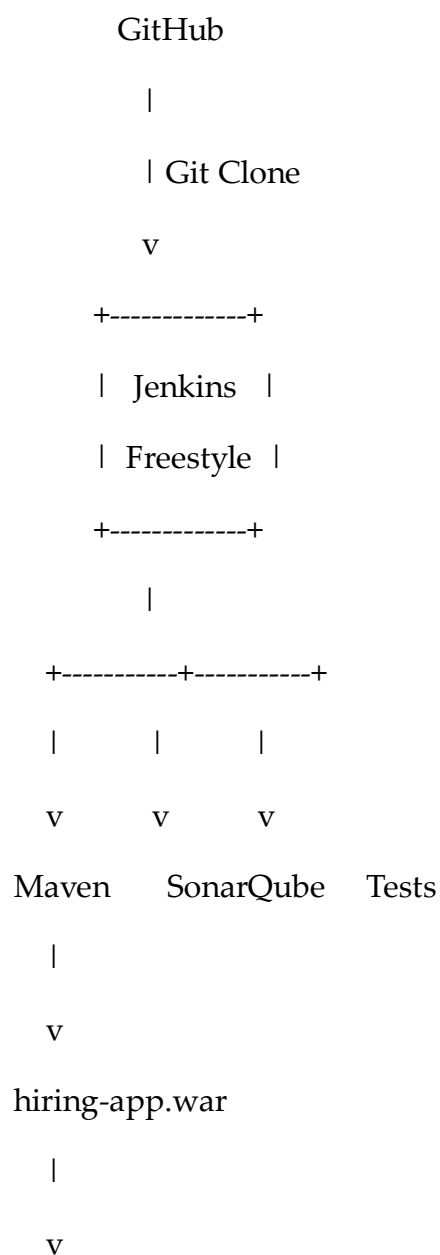
You already have much of the Jenkins/SonarQube setup from your previous work, so don't reinstall everything unnecessarily.

Required components

Component	Where to run	Purpose
Jenkins	EC2	CI/CD automation
Git	Jenkins EC2	Clone source code
Java 17	Jenkins EC2	Maven/application
Maven	Jenkins EC2	Compile/package

Component	Where to run	Purpose
SonarQube	Your existing EC2	Code quality
Nexus Repository	Docker container	Store WAR artifact
Tomcat	Docker container	Deploy application
Slack	Cloud	Notifications
Docker	Jenkins EC2	Run Nexus/Tomcat containers

The architecture should look approximately like:



Nexus Repository

(Docker)

|

v

Tomcat

(Docker)

|

v

Running Application

Jenkins

|

v

Slack

Notification

Important point about "Docker containers"

For this assignment, don't try to put **every single Jenkins build step inside a separate Docker container** unless your instructor specifically requires that.

A much simpler and cleaner implementation is:

- Jenkins → EC2 installation
- Nexus → Docker container
- Tomcat → Docker container
- Jenkins controls Docker using docker commands
- Maven runs on Jenkins
- SonarQube runs on its existing server
- Slack receives build notifications

Sonatype officially supports running Nexus Repository in a containerized environment and provides the sonatype/nexus3 image.

Step 1 — Verify Jenkins server

SSH into your Jenkins EC2:

```
ssh -i your-key.pem ec2-user@<JENKINS_PUBLIC_IP>
```

Check Jenkins:

```
sudo systemctl status jenkins
```

Check Java:

```
java -version
```

Check Maven:

```
mvn -version
```

Check Git:

```
git --version
```

Check Docker:

```
docker --version
```

If Docker is not installed on Amazon Linux 2023:

```
sudo dnf install docker -y
```

Start Docker:

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Check:

```
sudo systemctl status docker
```

```
us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressF...
aws | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari

● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-08-16 04:06:59 UTC; 19s ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 5488 (dockerd)
      Tasks: 9
     Memory: 30.5M
        CPU: 323ms
    CGroup: /system.slice/docker.service
            └─5488 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nfile=32768:65536

Aug 16 04:06:58 ip-172-31-20-75.ec2.internal systemd[1]: Starting docker.service - Docker Application Container Engine...
Aug 16 04:06:58 ip-172-31-20-75.ec2.internal dockerd[5488]: time="2026-08-16T04:06:58.525670718Z" level=info msg="Starting up"
Aug 16 04:06:58 ip-172-31-20-75.ec2.internal dockerd[5488]: time="2026-08-16T04:06:58.601469480Z" level=info msg="Loading containers: start."
Aug 16 04:06:59 ip-172-31-20-75.ec2.internal dockerd[5488]: time="2026-08-16T04:06:59.015821031Z" level=info msg="Loading containers: done."
Aug 16 04:06:59 ip-172-31-20-75.ec2.internal dockerd[5488]: time="2026-08-16T04:06:59.042778702Z" level=info msg="Docker daemon" commit=6fdf0a6 containerd-snapshotter=f
Aug 16 04:06:59 ip-172-31-20-75.ec2.internal dockerd[5488]: time="2026-08-16T04:06:59.042954814Z" level=info msg="Daemon has completed initialization"
Aug 16 04:06:59 ip-172-31-20-75.ec2.internal dockerd[5488]: time="2026-08-16T04:06:59.070424107Z" level=info msg="API listen on /run/docker.sock"
Aug 16 04:06:59 ip-172-31-20-75.ec2.internal systemd[1]: Started docker.service - Docker Application Container Engine.
[root@ip-172-31-20-75 ~]# s
```

Step 2 — Give Jenkins permission to use Docker

This is important.

Find Jenkins user:

```
id jenkins
```

Add Jenkins to Docker group:

```
sudo usermod -aG docker jenkins
```

Restart Jenkins:

```
sudo systemctl restart jenkins
```

You may also restart Docker:

```
sudo systemctl restart docker
```

Then verify:

```
sudo -u jenkins docker ps
```

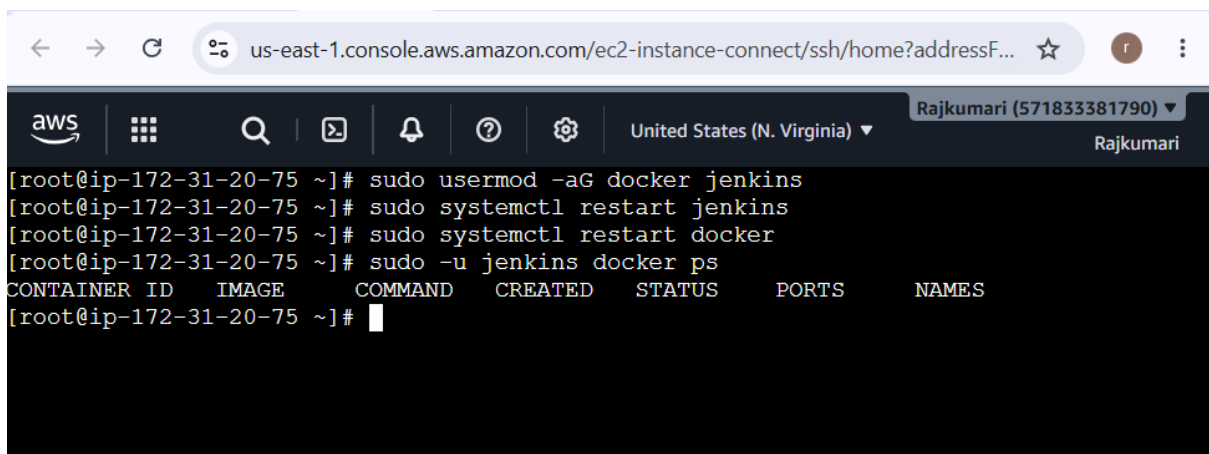
You want:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

and **not**:

permission denied while trying to connect to the Docker daemon

If permission is still denied, log out of SSH and reconnect, then restart Jenkins.



```
us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressF...
aws | [grid icon] | [search icon] | [terminal icon] | [notifications icon] | [help icon] | [settings icon] | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari
[root@ip-172-31-20-75 ~]# sudo usermod -aG docker jenkins
[root@ip-172-31-20-75 ~]# sudo systemctl restart jenkins
[root@ip-172-31-20-75 ~]# sudo systemctl restart docker
[root@ip-172-31-20-75 ~]# sudo -u jenkins docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
[root@ip-172-31-20-75 ~]#
```

Step 3 — Create Nexus using Docker

Pull Nexus:

```
docker pull sonatype/nexus3
```

Create a persistent volume:

```
docker volume create nexus-data
```

Run Nexus:

```
docker run -d \
  --name nexus \
  -p 8081:8081 \
  -v nexus-data:/nexus-data \
  sonatype/nexus3
```

Check:

```
docker ps
```

You should see something similar to:

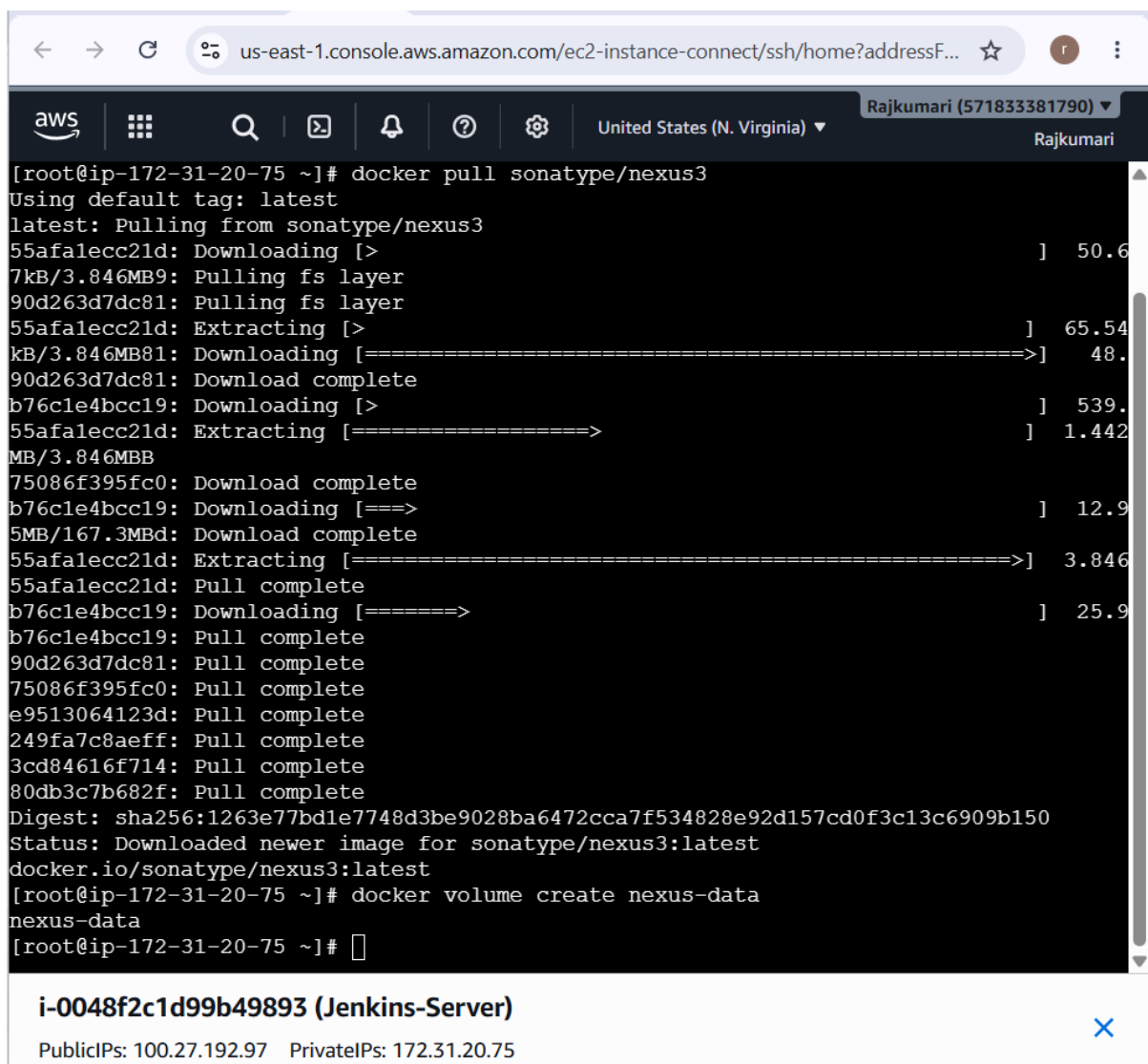
nexus sonatype/nexus3 Up ...

Check logs:

docker logs -f nexus

Wait until Nexus finishes starting.

Sonatype documents Docker/container deployment for Nexus Repository, and the current Nexus 3 Docker image is sonatype/nexus3.



```
[root@ip-172-31-20-75 ~]# docker pull sonatype/nexus3
Using default tag: latest
latest: Pulling from sonatype/nexus3
55afalecc21d: Downloading [>] 50.6
7kB/3.846MB9: Pulling fs layer
90d263d7dc81: Pulling fs layer
55afalecc21d: Extracting [>] 65.54
kB/3.846MB81: Downloading [=====>] 48.
90d263d7dc81: Download complete
b76c1e4bcc19: Downloading [>] 539.
55afalecc21d: Extracting [=====>] 1.442
MB/3.846MBB
75086f395fc0: Download complete
b76c1e4bcc19: Downloading [====>] 12.9
5MB/167.3MBd: Download complete
55afalecc21d: Extracting [=====>] 3.846
55afalecc21d: Pull complete
b76c1e4bcc19: Downloading [=====>] 25.9
b76c1e4bcc19: Pull complete
90d263d7dc81: Pull complete
75086f395fc0: Pull complete
e9513064123d: Pull complete
249fa7c8aeff: Pull complete
3cd84616f714: Pull complete
80db3c7b682f: Pull complete
Digest: sha256:1263e77bd1e7748d3be9028ba6472cca7f534828e92d157cd0f3c13c6909b150
Status: Downloaded newer image for sonatype/nexus3:latest
docker.io/sonatype/nexus3:latest
[root@ip-172-31-20-75 ~]# docker volume create nexus-data
nexus-data
[root@ip-172-31-20-75 ~]#
```

i-0048f2c1d99b49893 (Jenkins-Server)
PublicIPs: 100.27.192.97 PrivateIPs: 172.31.20.75

Download Nexus

cd /opt

```
wget https://download.sonatype.com/nexus/3/nexus-3.93.1-04-linux-x86_64.tar.gz
```

Verify:

```
ls -lh nexus-3.93.1-04-linux-x86_64.tar.gz
```

2. Extract Nexus

```
sudo tar -xvzf nexus-3.93.1-04-linux-x86_64.tar.gz
```

Check:

```
ls -ld /opt/nexus-3.93.1-04
```

3. Create a dedicated Nexus user

Do **not** run Nexus as root.

```
sudo useradd -r -m -d /opt/nexus -s /bin/bash nexus
```

4. Change ownership

```
sudo chown -R nexus:nexus /opt/nexus-3.93.1-04
```

Create the Nexus data directory:

```
sudo mkdir -p /opt/sonatype-work
```

```
sudo chown -R nexus:nexus /opt/sonatype-work
```

5. Configure Nexus to run as the nexus user

Open:

```
sudo vi /opt/nexus-3.93.1-04/bin/nexus.rc
```

Find:

```
#run_as_user=""
```

Change it to:

```
run_as_user="nexus"
```

Save and exit.

6. Create a symbolic link

This makes future Nexus upgrades easier:

```
sudo ln -s /opt/nexus-3.93.1-04 /opt/nexus
```


Check:

```
ls -l /opt/nexus
```

7. Check Java

Run:

```
java -version
```

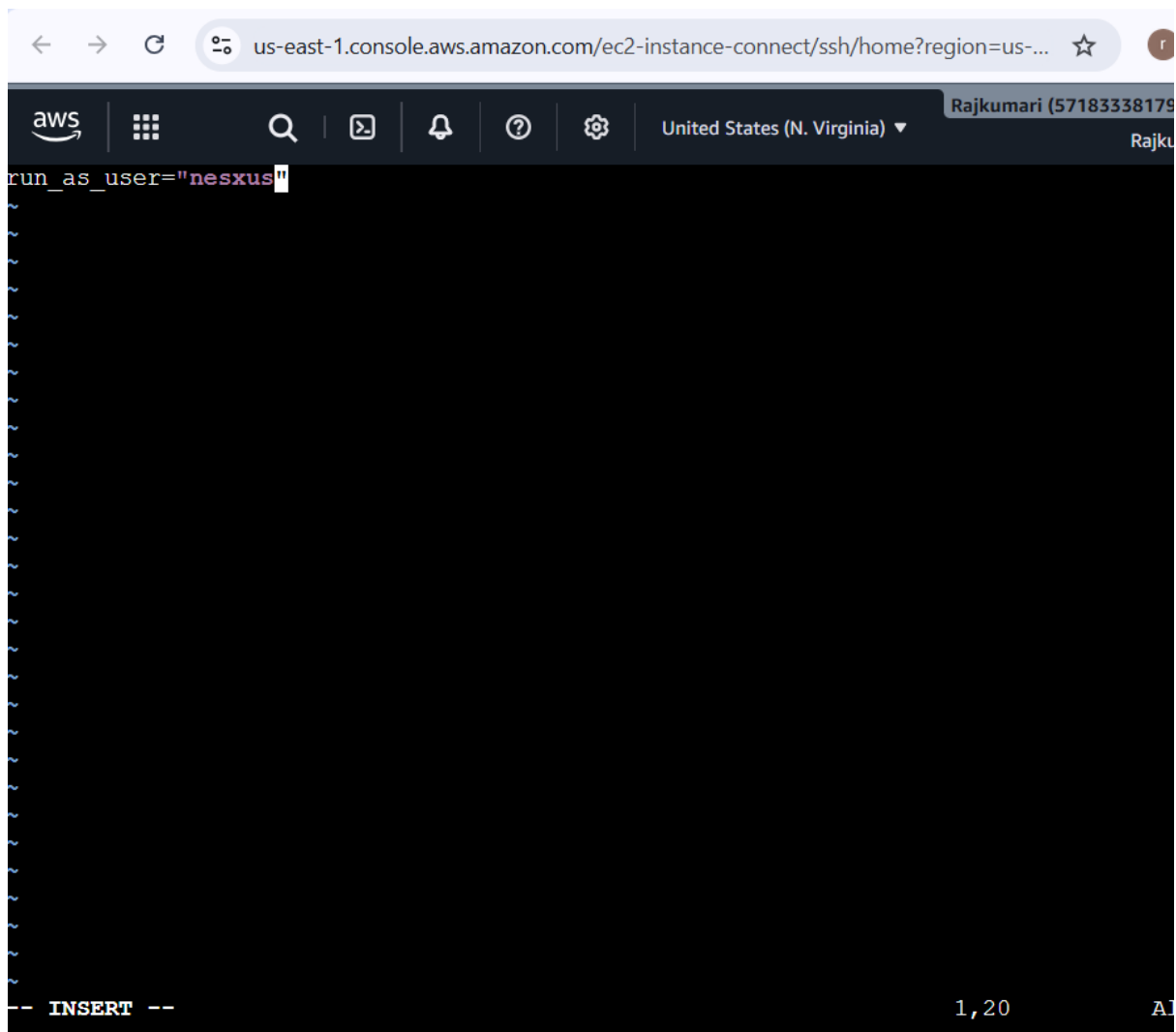
For your Nexus installation, **Java compatibility is important**. If Java is missing or you have the wrong version, don't start Nexus yet.

Paste the output of:

```
java -version
```

and:

```
ls -lh /opt/nexus-3.93.1-04
```



i-0f2a1c34f8f54497a (Nexus-Server)

PublicIPs: 107.21.188.143 PrivateIPs: 172.31.30.117

Download Nexus

```
cd /opt
```

```
wget https://download.sonatype.com/nexus/3/nexus-3.93.1-04-linux-x86_64.tar.gz
```

Verify:

```
ls -lh nexus-3.93.1-04-linux-x86_64.tar.gz
```

2. Extract Nexus

```
sudo tar -xvzf nexus-3.93.1-04-linux-x86_64.tar.gz
```

Check:

```
ls -ld /opt/nexus-3.93.1-04
```

3. Create a dedicated Nexus user

Do **not** run Nexus as root.

```
sudo useradd -r -m -d /opt/nexus -s /bin/bash nexus
```

4. Change ownership

```
sudo chown -R nexus:nexus /opt/nexus-3.93.1-04
```

Create the Nexus data directory:

```
sudo mkdir -p /opt/sonatype-work
```

```
sudo chown -R nexus:nexus /opt/sonatype-work
```

5. Configure Nexus to run as the nexus user

Open:

```
sudo vi /opt/nexus-3.93.1-04/bin/nexus.rc
```

Find:

```
#run_as_user=""
```

Change it to:

```
run_as_user="nexus"
```

Save and exit.

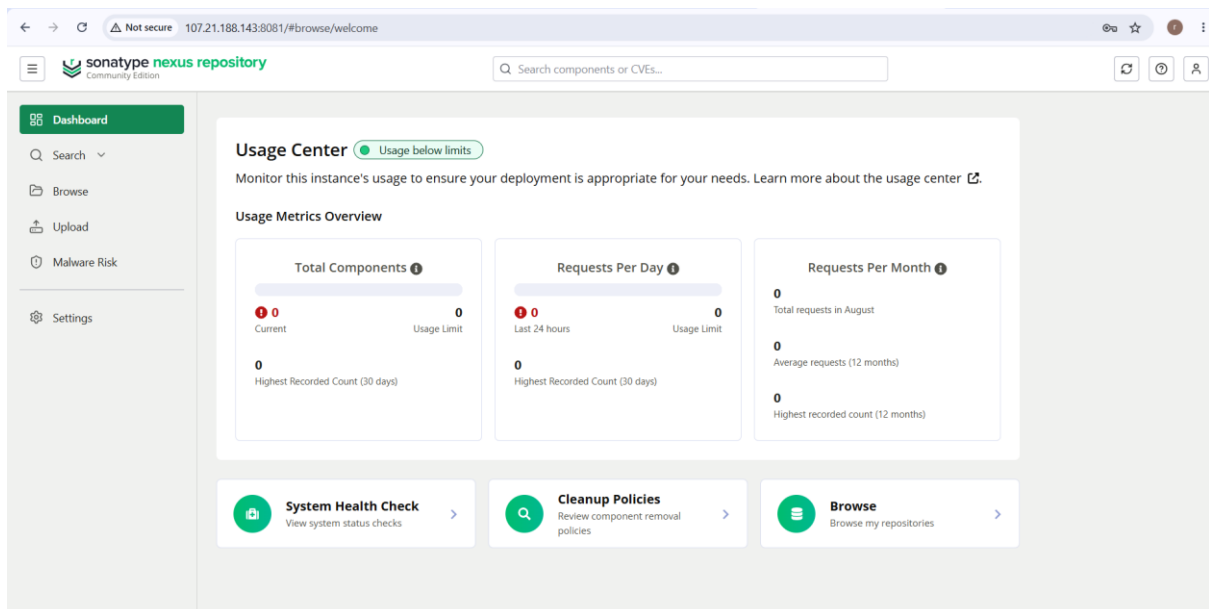
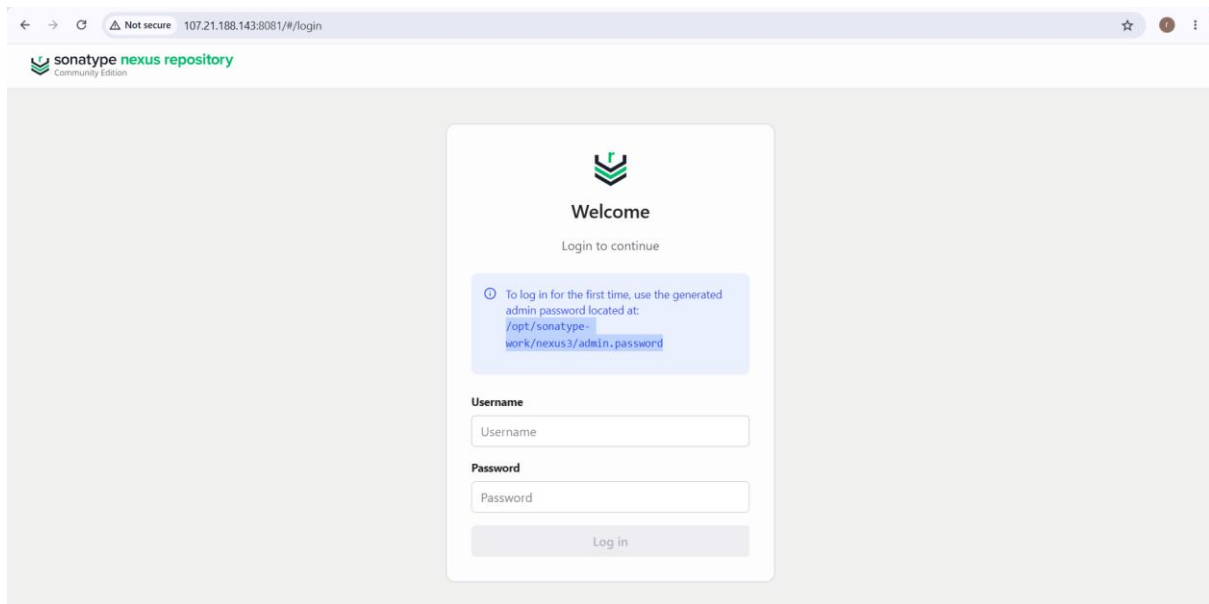
6. Create a symbolic link

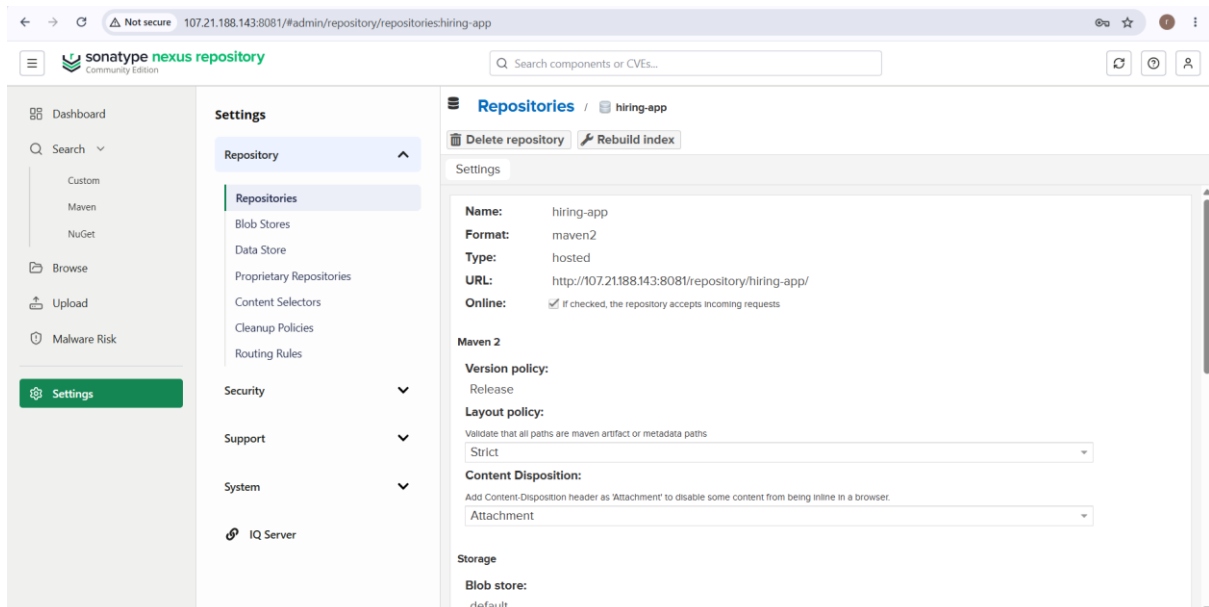
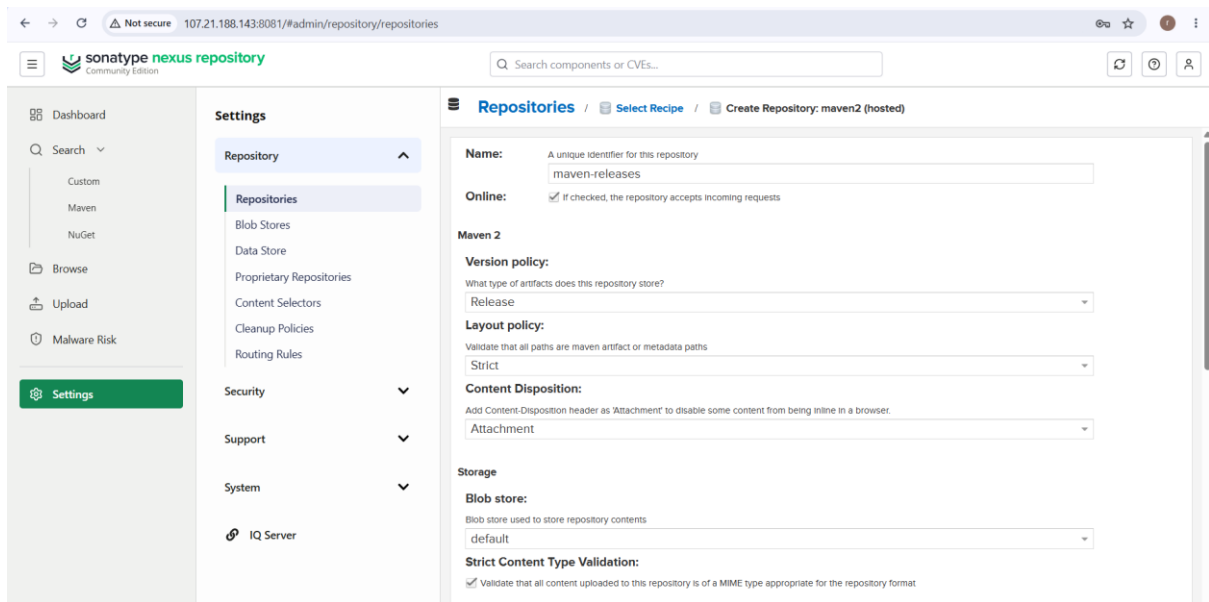
This makes future Nexus upgrades easier:

```
sudo ln -s /opt/nexus-3.93.1-04 /opt/nexus
```

Check:

```
ls -l /opt/nexus
```





1. Click Create local user

You should get a form with fields similar to:

- User ID
- First name
- Last name
- Email
- Password
- Status

- Roles

Enter:

Field	Value
-------	-------

User ID	jenkins
---------	---------

First name	Jenkins
------------	---------

Last name	CI/CD
-----------	-------

Email	Your email
-------	------------

Password	Create a strong password
----------	--------------------------

Status	Active
--------	--------

2. Assign the appropriate role

For your assignment, Jenkins needs permission to upload artifacts to Nexus.

For a simple lab setup, you can select:

nx-admin

However, **for a real production environment, don't give Jenkins nx-admin**. A dedicated role with only repository read/write permissions is better.

Since this is your training/assignment environment, nx-admin will make the Jenkins → Nexus integration easier.

3. Save the user

Click:

Create local user

sonatype nexus repository
Community Edition

Search components or CVEs...

Users / Create User

ID:
This will be used as the username
jenkins

First name:
Jenkins

Last name:
CI/CD

Email:
Used for notifications
brajkumari7675@gmail.com

Password:
.....

Confirm password:
.....

Status:
Active

Roles:

Available	Granted
nx-admin	nx-admin

Then in Jenkins:

Manage Jenkins → Credentials → Global → Add Credentials

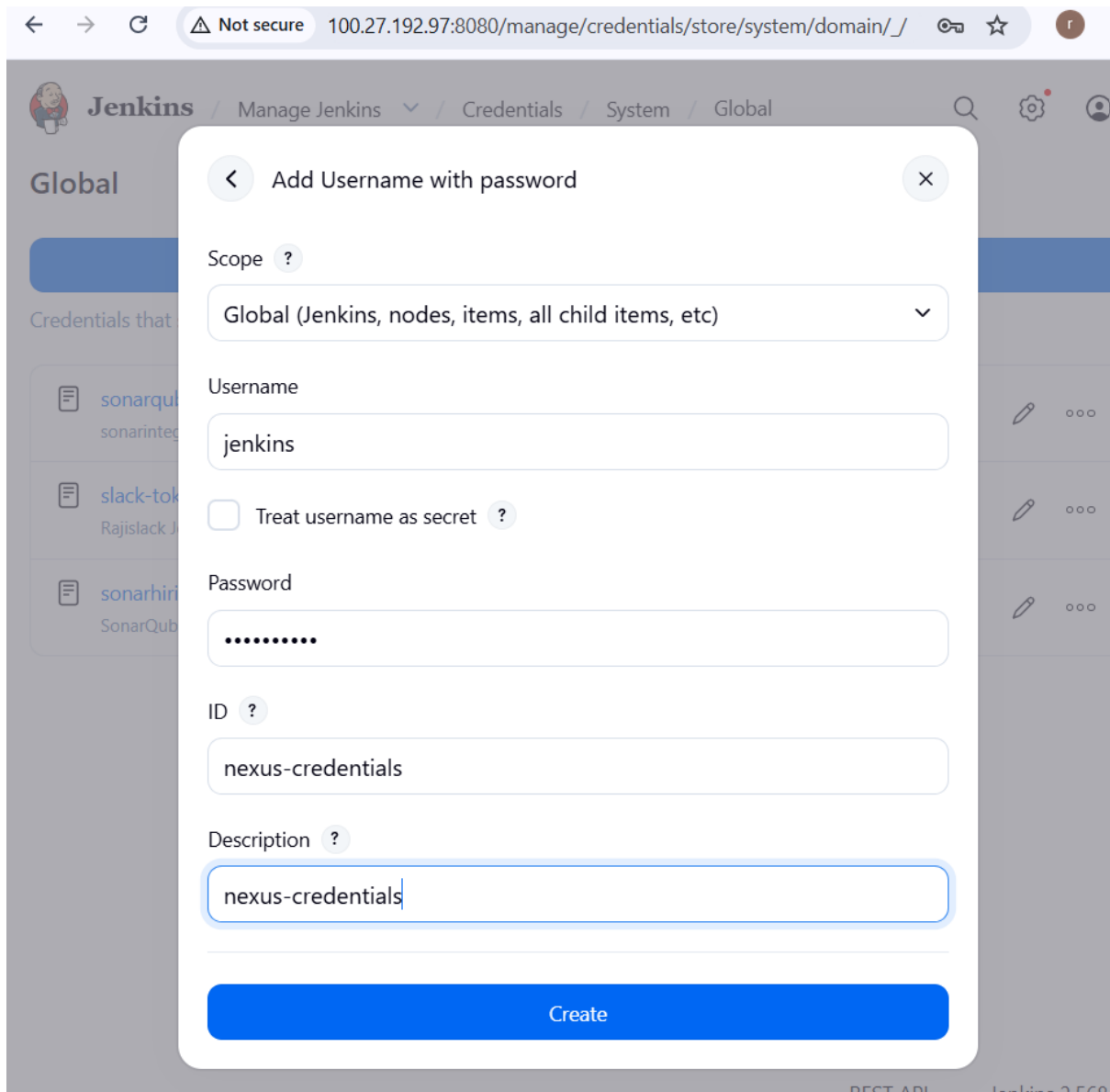
Select:

Kind: Username with password

ID: nexus-credentials

Username: jenkins

Password: <Nexus password>



10. Verify Tomcat server

On your Tomcat server, verify Tomcat is running:

```
sudo systemctl status tomcat
```

or:

```
sudo ss -lntp | grep 8080
```

Test:

```
curl http://localhost:8080
```

From your browser:

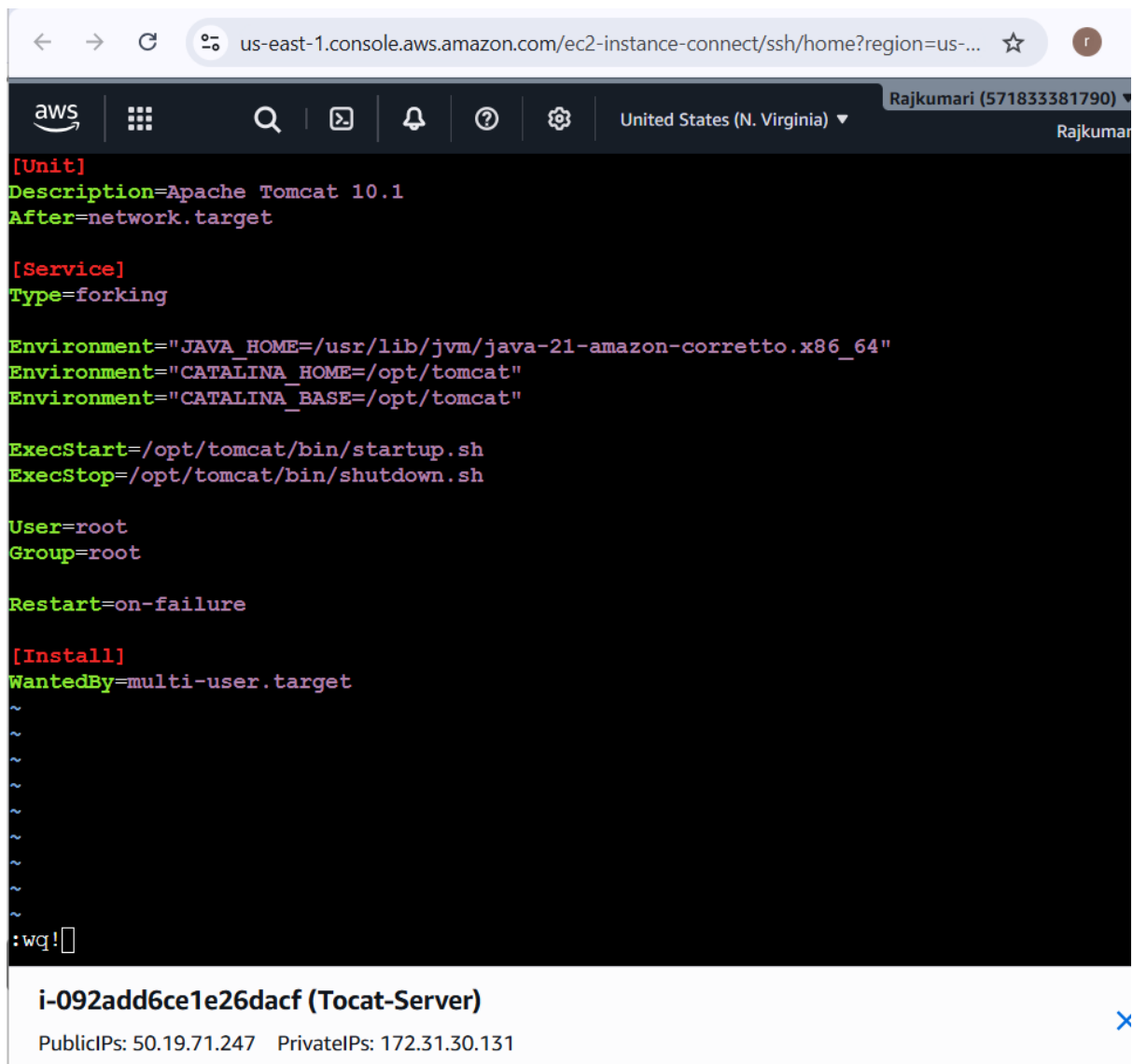
```
http://<TOMCAT_PUBLIC_IP>:8080
```


If your assignment specifically requires **Docker containers**, this is where your Tomcat container can be used.

For example:

docker ps

should show your Tomcat container.



The screenshot shows an AWS Management Console terminal window for an EC2 instance. The terminal displays the configuration of a Tomcat service. The configuration includes the following details:

```
[Unit]
Description=Apache Tomcat 10.1
After=network.target

[Service]
Type=forking

Environment="JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto.x86_64"
Environment="CATALINA_HOME=/opt/tomcat"
Environment="CATALINA_BASE=/opt/tomcat"

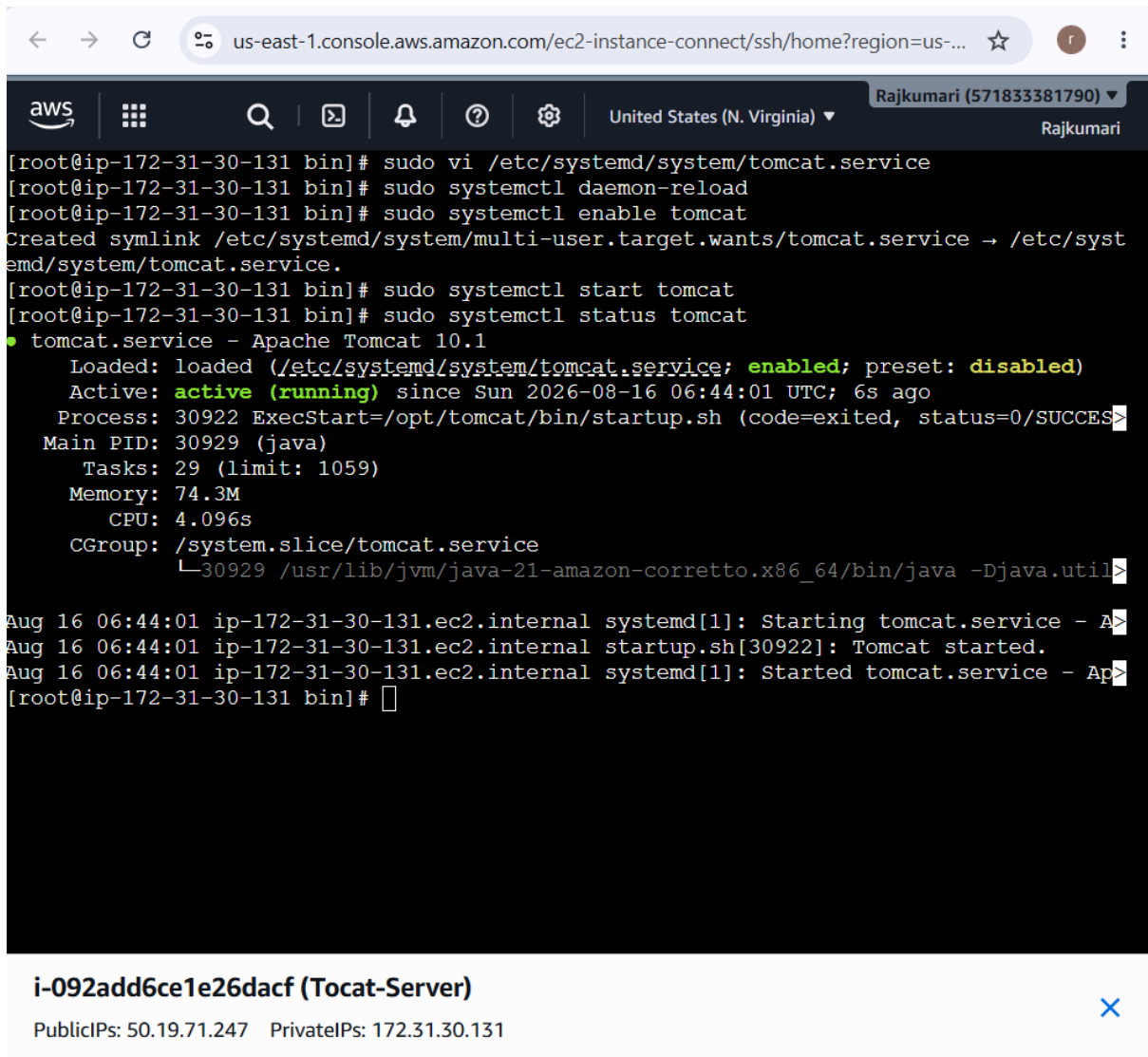
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh

User=root
Group=root

Restart=on-failure

[Install]
WantedBy=multi-user.target
~
~
~
~
~
~
~
~
~
~
:wq!
```

Below the terminal output, the instance ID **i-092add6ce1e26dacf (Tocat-Server)** is displayed, along with its Public IP (50.19.71.247) and Private IP (172.31.30.131).



The screenshot shows the AWS Management Console interface. At the top, the browser address bar displays 'us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-...'. The console header includes the AWS logo, navigation icons, and the region 'United States (N. Virginia)'. The user's name 'Rajkumari (571833381790)' is visible in the top right corner.

The main content area displays a terminal session for an EC2 instance with IP 'ip-172-31-30-131'. The terminal shows the following commands and output:

```
[root@ip-172-31-30-131 bin]# sudo vi /etc/systemd/system/tomcat.service
[root@ip-172-31-30-131 bin]# sudo systemctl daemon-reload
[root@ip-172-31-30-131 bin]# sudo systemctl enable tomcat
Created symlink /etc/systemd/system/multi-user.target.wants/tomcat.service → /etc/systemd/system/tomcat.service.
[root@ip-172-31-30-131 bin]# sudo systemctl start tomcat
[root@ip-172-31-30-131 bin]# sudo systemctl status tomcat
```

The status output for 'tomcat.service' is as follows:

```
● tomcat.service - Apache Tomcat 10.1
   Loaded: loaded (/etc/systemd/system/tomcat.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-08-16 06:44:01 UTC; 6s ago
     Process: 30922 ExecStart=/opt/tomcat/bin/startup.sh (code=exited, status=0/SUCCESS)
    Main PID: 30929 (java)
      Tasks: 29 (limit: 1059)
     Memory: 74.3M
        CPU: 4.096s
    CGroup: /system.slice/tomcat.service
            └─30929 /usr/lib/jvm/java-21-amazon-corretto.x86_64/bin/java -Djava.util
```

Below the terminal output, system logs show the service starting:

```
Aug 16 06:44:01 ip-172-31-30-131.ec2.internal systemd[1]: Starting tomcat.service - Apache Tomcat 10.1
Aug 16 06:44:01 ip-172-31-30-131.ec2.internal startup.sh[30922]: Tomcat started.
Aug 16 06:44:01 ip-172-31-30-131.ec2.internal systemd[1]: Started tomcat.service - Apache Tomcat 10.1
[root@ip-172-31-30-131 bin]#
```

At the bottom of the console window, a summary box for the instance 'i-092add6ce1e26daf (Tocat-Server)' is shown, including Public IPs (50.19.71.247) and Private IPs (172.31.30.131).

Configure Tomcat for Jenkins Deployment

Your assignment requires:

Jenkins → Build WAR → Deploy to Tomcat

So next configure Tomcat Manager.

8. Edit tomcat-users.xml

```
sudo vi /opt/tomcat/conf/tomcat-users.xml
```

Before the closing `</tomcat-users>` tag, add:

```
<role rolename="manager-gui"/>
```

```
<role rolename="manager-script"/>
```

```
<user username="jenkins" password="Jenkins@123" roles="manager-gui,manager-script"/>
```

It should look like:

```
<tomcat-users>
```

```
  <role rolename="manager-gui"/>
```

```
  <role rolename="manager-script"/>
```

```
  <user username="jenkins"
```

```
    password="Jenkins@123"
```

```
    roles="manager-gui,manager-script"/>
```

```
</tomcat-users>
```

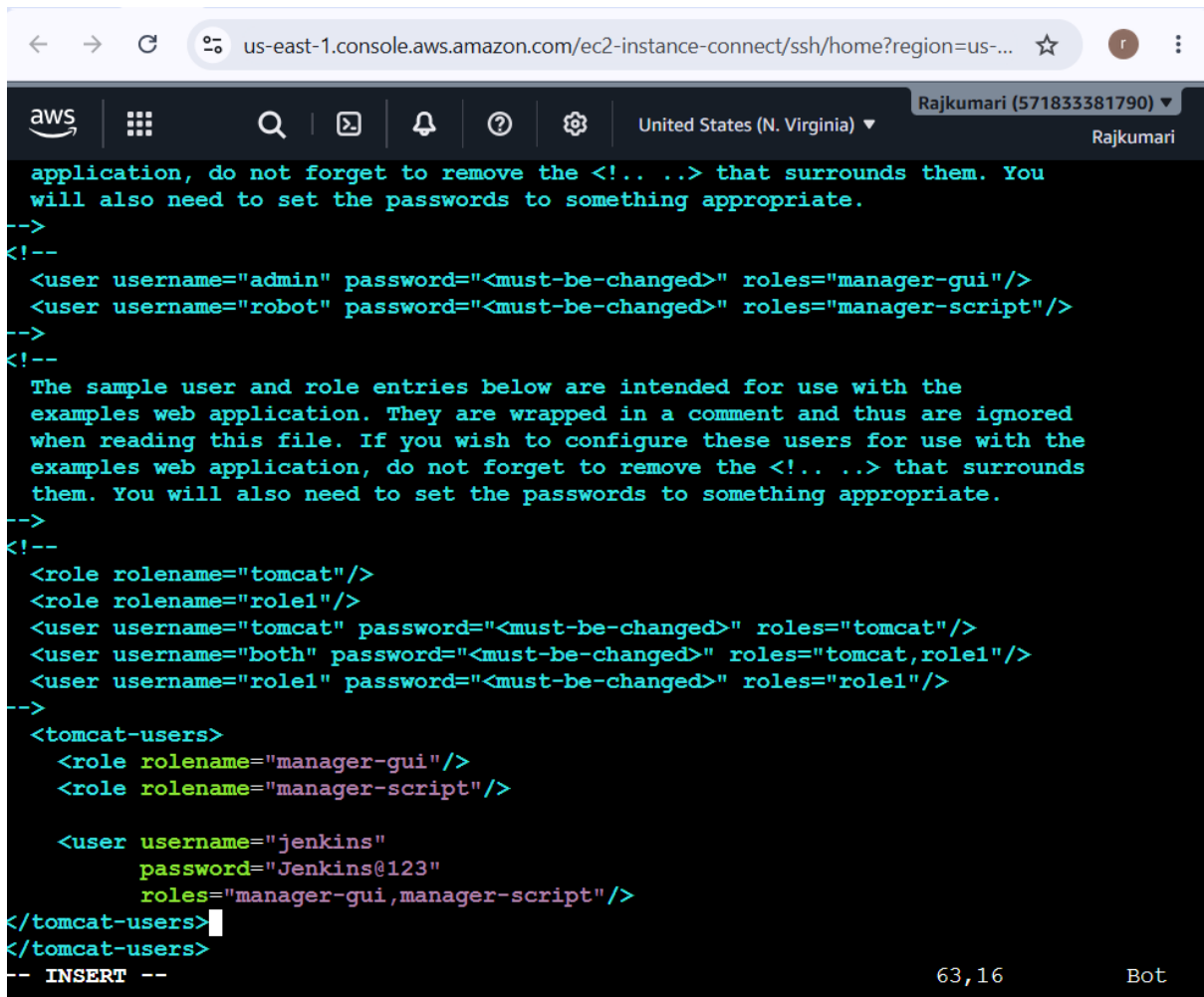
Save:

ESC

:wq

ENTER

For your assignment, you can use this password, but for a real environment use a strong secret and don't store it directly in your Jenkinsfile.



The screenshot shows an AWS console terminal window. The browser address bar displays 'us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-...'. The AWS console header includes the AWS logo, navigation icons, the region 'United States (N. Virginia)', and the user 'Rajkumari (571833381790)'. The terminal content shows XML configuration for Tomcat users and roles, with comments in red. The configuration includes roles for 'manager-gui' and 'manager-script', and users for 'admin', 'robot', 'tomcat', 'both', 'role1', and 'jenkins'. The 'jenkins' user is configured with the password 'Jenkins@123' and roles 'manager-gui,manager-script'. The terminal ends with a cursor on a new line and the text '-- INSERT --'.

```
application, do not forget to remove the <!-- ... --> that surrounds them. You
will also need to set the passwords to something appropriate.
-->
<!--
<user username="admin" password="<must-be-changed>" roles="manager-gui"/>
<user username="robot" password="<must-be-changed>" roles="manager-script"/>
-->
<!--
The sample user and role entries below are intended for use with the
examples web application. They are wrapped in a comment and thus are ignored
when reading this file. If you wish to configure these users for use with the
examples web application, do not forget to remove the <!-- ... --> that surrounds
them. You will also need to set the passwords to something appropriate.
-->
<!--
<role rolename="tomcat"/>
<role rolename="role1"/>
<user username="tomcat" password="<must-be-changed>" roles="tomcat"/>
<user username="both" password="<must-be-changed>" roles="tomcat,role1"/>
<user username="role1" password="<must-be-changed>" roles="role1"/>
-->
<tomcat-users>
  <role rolename="manager-gui"/>
  <role rolename="manager-script"/>

  <user username="jenkins"
    password="Jenkins@123"
    roles="manager-gui,manager-script"/>
</tomcat-users>
</tomcat-users>
-- INSERT --
```

For your Tomcat server, use this Jenkins private IP to restrict Tomcat Manager access to Jenkins.

1. On the Tomcat server

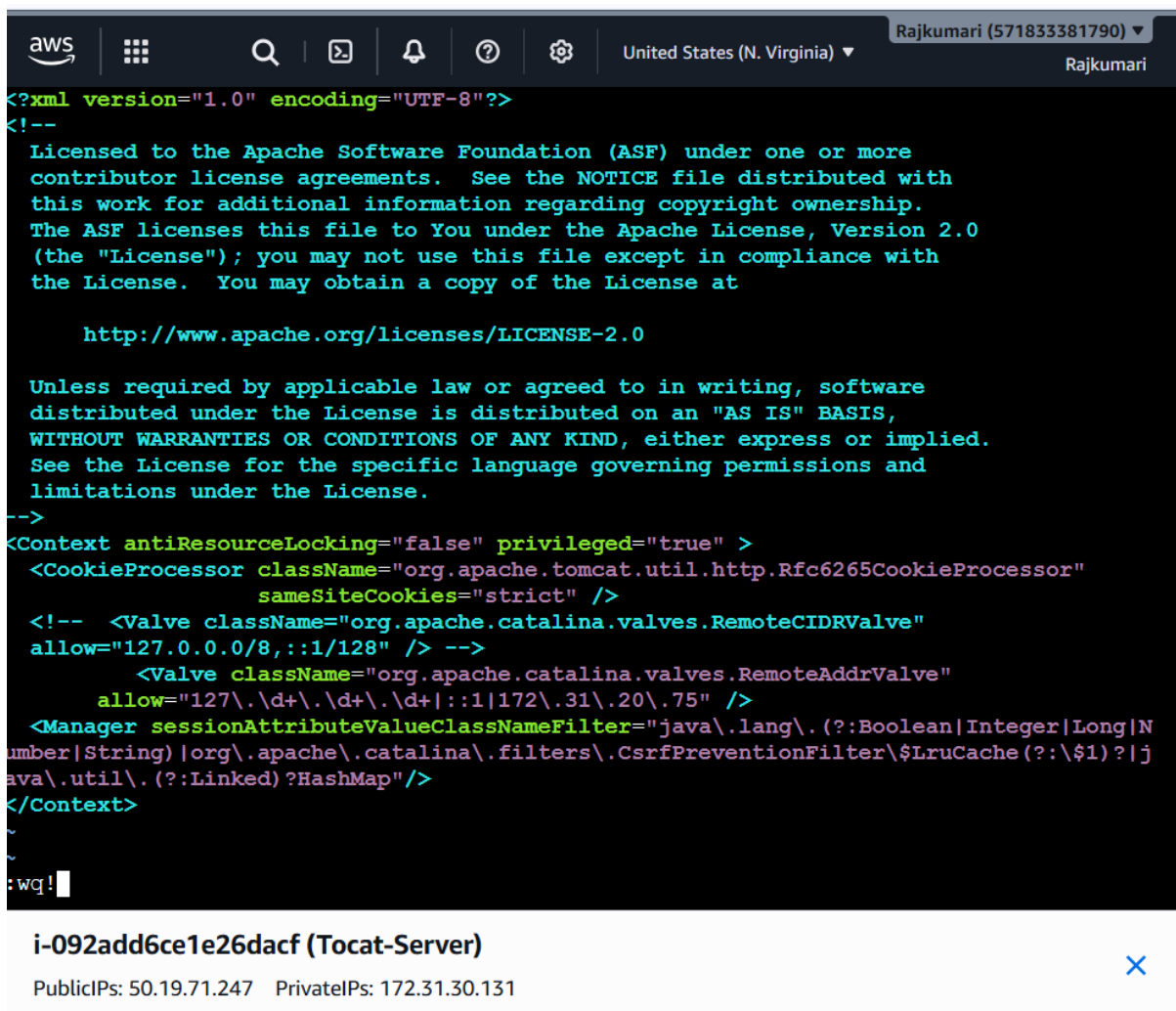
Edit:

```
sudo vi /opt/tomcat/webapps/manager/META-INF/context.xml
```

Find the RemoteAddrValve and change it to:

```
<Valve className="org.apache.catalina.valves.RemoteAddrValve"
```

```
  allow="127\.\d+\.\d+\.\d+|::1|172\.\31\.\20\.\75" />
```



```
<?xml version="1.0" encoding="UTF-8"?>
<!--
Licensed to the Apache Software Foundation (ASF) under one or more
contributor license agreements.  See the NOTICE file distributed with
this work for additional information regarding copyright ownership.
The ASF licenses this file to You under the Apache License, Version 2.0
(the "License"); you may not use this file except in compliance with
the License.  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
-->
<Context antiResourceLocking="false" privileged="true" >
  <CookieProcessor className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
    sameSiteCookies="strict" />
  <!-- <Valve className="org.apache.catalina.valves.RemoteCIDRValve"
    allow="127.0.0.0/8,::1/128" /> -->
    <Valve className="org.apache.catalina.valves.RemoteAddrValve"
      allow="127\.\d+\.\d+\.\d+|::1|172\.\31\.\20\.\75" />
  <Manager sessionAttributeValueClassNameFilter="java\.lang\.(?:Boolean|Integer|Long|N
umber|String)|org\.apache\.catalina\.filters\.CsrfPreventionFilter\$LruCache(?:\$1)?|j
ava\.util\.(?:Linked)?HashMap"/>
</Context>
~
:wq!
```

i-092add6ce1e26dacf (Tocat-Server) ×

PublicIPs: 50.19.71.247 PrivateIPs: 172.31.30.131

6. Set JAVA_HOME

Check the installed directory:

```
ls -l /usr/lib/jvm/
```

Then create the environment variable:

```
sudo vi /etc/profile.d/java.sh
```

Add:

```
export JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto
```

```
export PATH=$JAVA_HOME/bin:$PATH
```

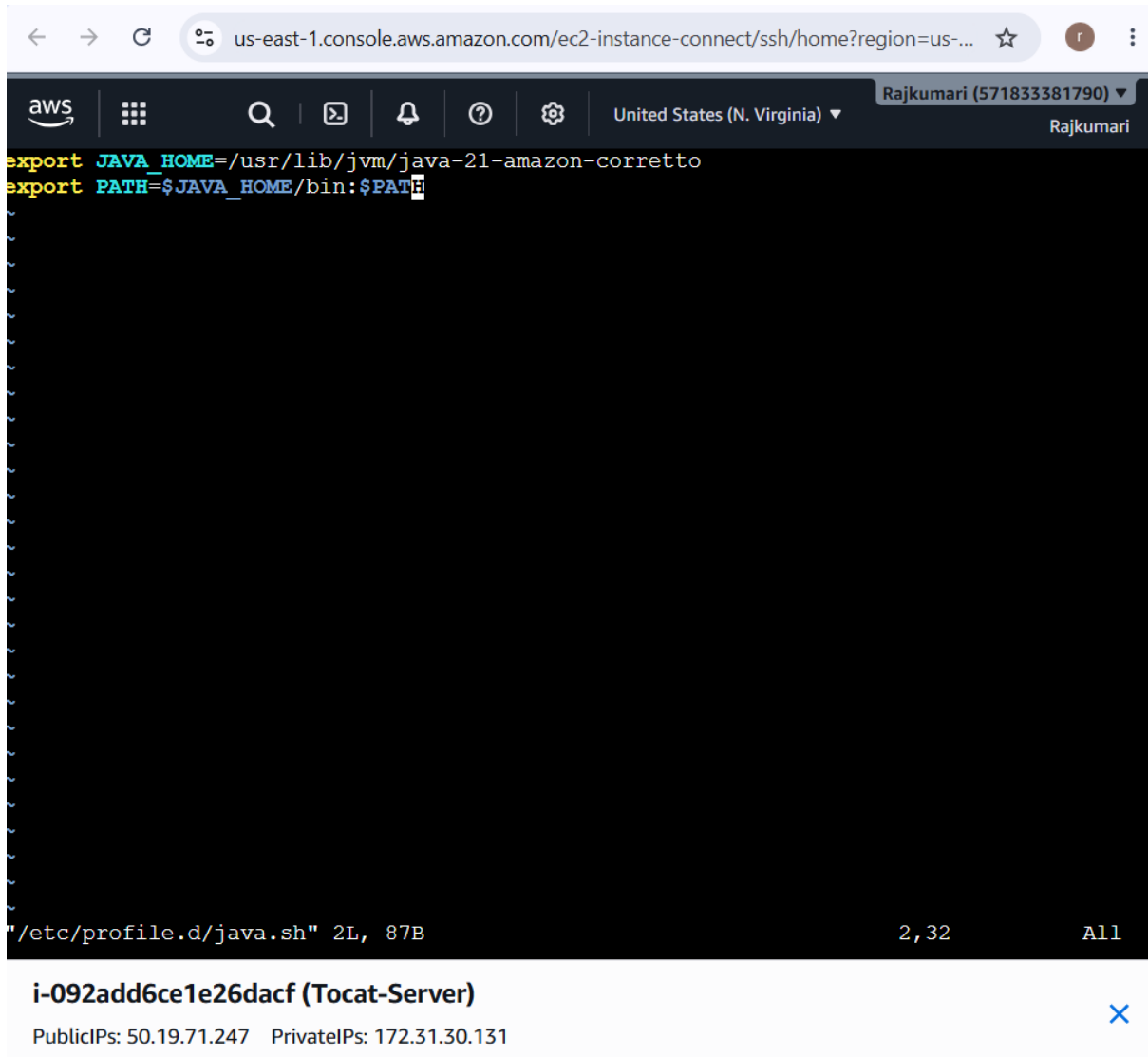
Save and exit, then run:

```
source /etc/profile.d/java.sh
```

Verify:

```
echo $JAVA_HOME
```

```
java -version
```



1. Install Nexus-related plugin

Go to:

Jenkins → Manage Jenkins → Plugins → Available plugins

Search for:

Nexus Artifact Uploader

Install it.

This allows Jenkins to upload the generated .war/.jar artifact to your Nexus repository.

After installation, restart Jenkins if Jenkins asks you to.

2. Install Tomcat-related plugin

Search for:

Deploy to container

Install:

Deploy to container Plugin

This plugin supports deployment of WAR files to containers such as Tomcat.

After installation, restart Jenkins if required.



Plugins



Updates

1



Available plugins



Installed plugins



Advanced settings



Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

Nexus Artifact Uploader  Success

Loading plugin extensions  Success

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

→ ☐ Restart Jenkins when installation is complete and no jobs are running

The screenshot shows the Jenkins web interface. At the top, there's a navigation bar with the Jenkins logo, the text "Jenkins", and a breadcrumb trail: "Manage Jenkins" with a dropdown arrow, followed by "Plugins". On the right side of the navigation bar are icons for search, settings, and a user profile. Below the navigation bar, the main heading is "Plugins". Under this heading, there are four menu items: "Updates" (with a download icon), "Available plugins" (with a package icon), "Installed plugins" (with a puzzle piece icon), and "Advanced settings" (with a gear icon). Below these is a light blue bar with a hamburger menu icon and the text "Download progress". The "Download progress" section has a sub-heading "Download progress". Under this, there's a "Preparation" section with a list of steps: "Checking internet connectivity", "Checking update center connectivity", and "Success". Below that, there are four items, each with a green checkmark and the word "Success": "Nexus Artifact Uploader", "Loading plugin extensions", "Deploy to container", and "Loading plugin extensions". At the bottom of the section, there's a blue link "Go back to the top page" with the text "(you can start using the installed plugins right away)" below it.

Jenkins / Manage Jenkins / Plugins

Plugins

- Updates
- Available plugins
- Installed plugins
- Advanced settings

Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

Nexus Artifact Uploader ✓ Success

Loading plugin extensions ✓ Success

Deploy to container ✓ Success

Loading plugin extensions ✓ Success

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

Now create Maven settings.xml

This is the **next step**.

On your **Jenkins server**, run:

```
sudo mkdir -p /var/lib/jenkins/.m2
```

Then:

```
sudo vi /var/lib/jenkins/.m2/settings.xml
```

Put this inside:

```
<settings>
```

```
  <servers>
```

```
    <server>
```

```
<id>hiring-app</id>

<username>${env.NEXUS_USERNAME}</username>

<password>${env.NEXUS_PASSWORD}</password>

</server>

</servers>

</settings>
```

Notice this important part:

```
<id>hiring-app</id>
```

It **must exactly match** the <id> in your pom.xml.

Your setup is therefore:

pom.xml

|

| id = hiring-app

↓

settings.xml

|

| id = hiring-app

↓

Nexus

|

↓

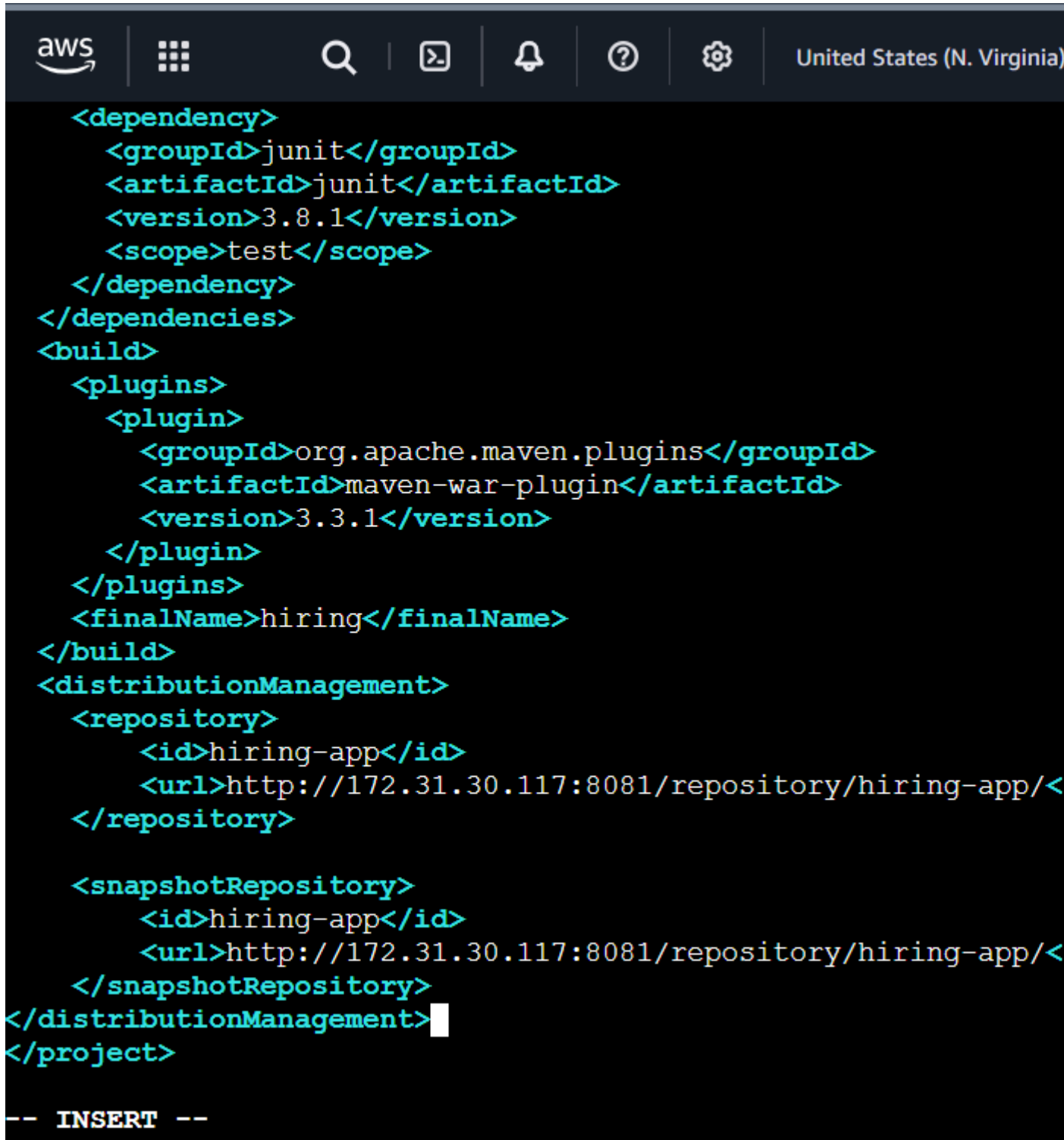
hiring-app repository

Save it:

Esc

:wq

Enter



3. Set ownership

Run:

```
sudo chown -R jenkins:jenkins /var/lib/jenkins/.m2
```

Then verify:

```
sudo cat /var/lib/jenkins/.m2/settings.xml
```

←

→

↻

🌐

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFa...

☆

👤

⋮

aws

🗑

🔍

📄

🔔

❓

⚙

United States (N. Virginia) ▾

Rajkumari (571833381790) ▾

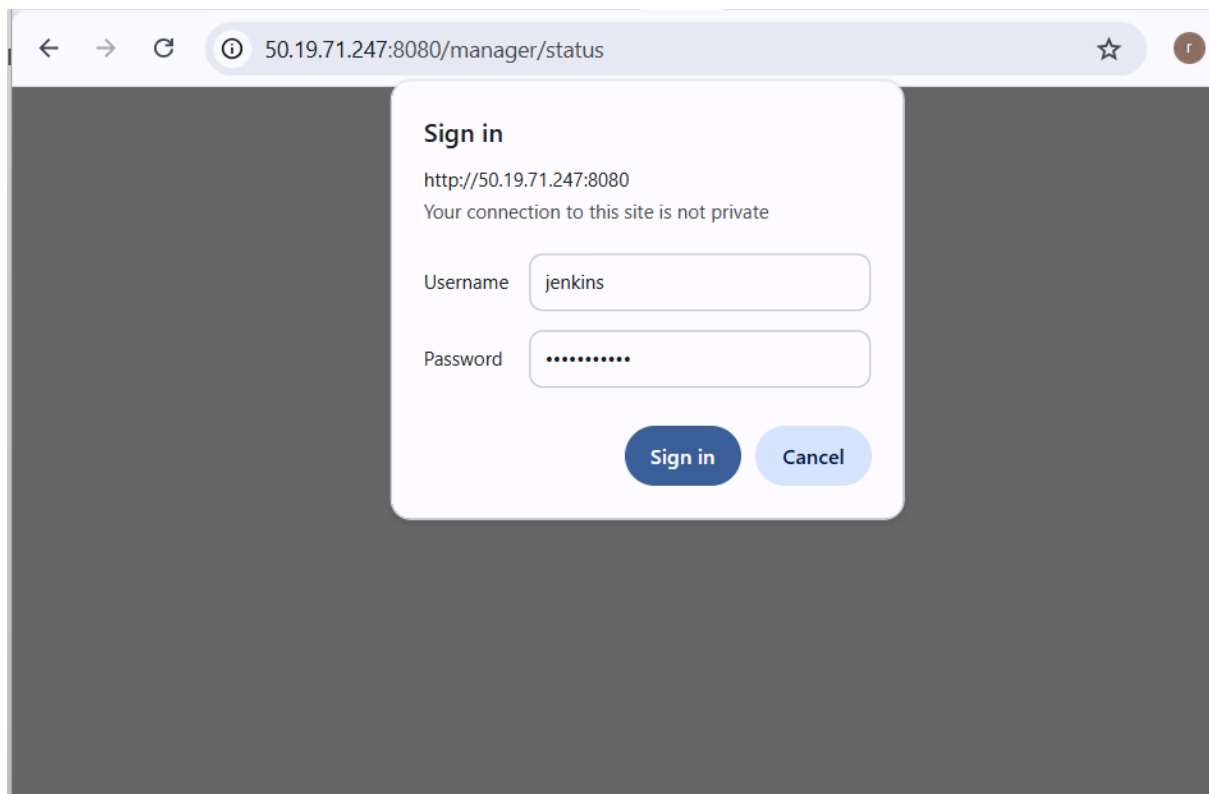
Rajkumari

```
root@ip-172-31-20-75 opt]# cd ~/hiring-app
s
Dockerfile      README.md          jenkinsfile-cicd  src
Jenkinsfile     'Untitled Diagram.drawio'  pom.xml           target
root@ip-172-31-20-75 hiring-app]# vi pom.xml
root@ip-172-31-20-75 hiring-app]# sudo mkdir -p /var/lib/jenkins/.m2
root@ip-172-31-20-75 hiring-app]# sudo vi /var/lib/jenkins/.m2/settings.xml
root@ip-172-31-20-75 hiring-app]# sudo chown -R jenkins:jenkins /var/lib/jenkins/.m2
root@ip-172-31-20-75 hiring-app]# sudo cat /var/lib/jenkins/.m2/settings.xml
<settings>
  <servers>
    <server>
      <id>hiring-app</id>
      <username>${env.NEXUS_USERNAME}</username>
      <password>${env.NEXUS_PASSWORD}</password>
    </server>
  </servers>
</settings>
root@ip-172-31-20-75 hiring-app]#
```

i-0048f2c1d99b49893 (Jenkins-Server)

PublicIPs: 100.27.192.97 PrivateIPs: 172.31.20.75

✕





Server Status

Manager

List Applications	HTML Manager Help	Manager Help	Complete Server Status
-----------------------------------	-----------------------------------	------------------------------	--

Server Information

Tomcat Version	JVM Version	JVM Vendor	OS Name	OS Version	OS Architecture	Hostname	IP Address
Apache Tomcat/10.1.57	21.0.12+8-LTS	Amazon.com Inc.	Linux	6.18.39-79.141.amzn2023.x86_64	amd64	ip-172-31-30-131.ec2.internal	172.31.30.131

JVM

Free Memory: 3.90 MiB Total Memory: 15.50 MiB Max Memory: 222.37 MiB

Memory Pool	Type	Initial	Total	Maximum	Used
Eden Space	Heap memory	4.31 MiB	4.31 MiB	61.37 MiB	0.87 MiB (1%)
Survivor Space	Heap memory	0.50 MiB	0.50 MiB	7.62 MiB	0.50 MiB (6%)
Tenured Gen	Heap memory	10.68 MiB	10.68 MiB	153.37 MiB	10.23 MiB (6%)
CodeHeap 'non-nmethods'	Non-heap memory	2.43 MiB	2.43 MiB	5.55 MiB	1.23 MiB (22%)
CodeHeap 'non-profiled nmethods'	Non-heap memory	2.43 MiB	2.43 MiB	117.22 MiB	1.06 MiB (0%)
CodeHeap 'profiled nmethods'	Non-heap memory	2.43 MiB	5.62 MiB	117.21 MiB	5.58 MiB (4%)
Compressed Class Space	Non-heap memory	0.00 MiB	1.93 MiB	1024.00 MiB	1.76 MiB (0%)
Metaspace	Non-heap memory	0.00 MiB	17.56 MiB	-0.00 MiB	17.22 MiB

"http-nio-8080"

Max threads: 200 Current thread count: 10 Current threads busy: 1 Keep alive sockets count: 2
 Max processing time: 180 ms Processing time: 0.235 s Request count: 2 Error count: 1 Bytes received: 0.00 MiB Bytes sent: 0.00 MiB

Stage	Time	Bytes Sent	Bytes Recv	Client (Forwarded)	Client (Actual)	VHost	Request
S	23 ms	0 KIB	0 KIB	103.248.210.235	103.248.210.235	50.19.71.247	GET /manager/status HTTP/1.1

Step 1 — Install Jenkins Tomcat deployment plugin

In Jenkins:

Manage Jenkins → Plugins → Available plugins

Search:

Deploy to container

Install:

Deploy to container Plugin

If Jenkins asks for a restart, restart Jenkins.

Step 2 — Add Tomcat credentials to Jenkins

Go to:

Manage Jenkins → Credentials

Then:

System → Global credentials → Add Credentials

Select:

Kind: Username with password

Enter:

Username: jenkins

Password: Jenkins@123

ID: tomcat-credentials

Description: Tomcat deployment

Click **Create**.

Step 3 — Configure your Freestyle job

Go to:

Jenkins → hiring-app → Configure

You already have your Git configuration:

Repository:

<https://github.com/rajkumari-hub/hiring-app.git>

Branch:

*/main

And your existing Maven build:

```
mvn clean deploy -s /var/lib/jenkins/.m2/settings.xml
```

This already uploads your WAR to Nexus.

Now add Tomcat deployment

Scroll to:

Post-build Actions

Click:

Add post-build action

Select:

Deploy war/ear to a container

You should get a configuration section.

WAR/EAR files

Enter:

target/hiring.war

Containers

Select:

Tomcat 10.x

Credentials

Select:

tomcat-credentials

Tomcat URL

Enter:

http://172.31.30.131:8080

Then **Save**.

Step 1 — Don't use "Deploy war/ear to a container"

In your Freestyle job:

Jenkins → hiring-app → Configure

You can leave the plugin section unused.

Go to:

Build → Add build step → Execute shell

Add this:

```
#!/bin/bash
```

```
TOMCAT_URL="http://172.31.30.131:8080"
```

```
TOMCAT_USER="jenkins"
```

```
TOMCAT_PASSWORD="Jenkins@123"
```

```
WAR_FILE="target/hiring.war"
```

```
echo "==== Deploying hiring.war to Tomcat ===="
```

```
curl --fail --silent --show-error \
```

```
-u "${TOMCAT_USER}:${TOMCAT_PASSWORD}" \
```

```
-T "${WAR_FILE}" \
```

```
"${TOMCAT_URL}/manager/text/deploy?path=/hiring&update=true"
```

```
echo
```

```
echo "==== Tomcat deployment completed ===="
```


← → ↻ Not secure 100.27.192.97:8080/job/hiring-app/ ☆ ⓘ ⋮

 **Jenkins** / hiring-app 🔍 ⚙️ 👤

Status

📄 Changes

📁 Workspace

▶ Build Now

🗑 Delete Project

⚙ Configure

✎ Rename

🟢 hiring-app Add description

Permalinks

- Last build (#5), 2 min 38 sec ago
- Last stable build (#5), 2 min 38 sec ago
- Last successful build (#5), 2 min 38 sec ago
- Last failed build (#3), 1 hr 23 min ago
- Last unsuccessful build (#3), 1 hr 23 min ago
- Last completed build (#5), 2 min 38 sec ago

Builds > ⋮

🔍 Filter /

Today

- 🟢 #5 10:23 AM ⌵
- 🟢 #4 9:41 AM ⌵
- 🔴 #3 9:02 AM ⌵
- 🟢 #2 9:01 AM ⌵
- 🔴 #1 8:57 AM ⌵

REST API Jenkins 2.568.2

← → ↻ Not secure 100.27.192.97:8080/job/hiring-app/ ☆ ⓘ ⋮

 **Jenkins** / hiring-app 🔍 ⚙️ 👤

Status

📄 Changes

📁 Workspace

▶ Build Now

🗑 Delete Project

⚙ Configure

✎ Rename

🟢 hiring-app Add description

Permalinks

- Last build (#6), 5 min 36 sec ago
- Last stable build (#6), 5 min 36 sec ago
- Last successful build (#6), 5 min 36 sec ago
- Last failed build (#3), 1 hr 48 min ago
- Last unsuccessful build (#3), 1 hr 48 min ago
- Last completed build (#6), 5 min 36 sec ago

Builds > ⋮

🔍 Filter /

Today

- 🟢 #9 10:51 AM ⌵
- 🟢 #8 10:51 AM ⌵
- 🟢 #7 10:51 AM ⌵
- 🟢 #6 10:45 AM ⌵
- 🟢 #5 10:23 AM ⌵
- 🟢 #4 9:41 AM ⌵
- 🔴 #3 9:02 AM ⌵

← → ↻ Not secure 100.27.192.97:8080/job/hiring-app/5/console ☆ ⓘ ⋮

 **Jenkins** / hiring-app ▾ / #5 ▾ 🔍 ⚙️ 👤

Progress (1): 1.2 kB

Uploaded to hiring-app: <http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/0.1/hiring-0.1.pom> (1.2 kB at 5.4 kB/s)

Downloading from hiring-app: <http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/maven-metadata.xml>

Progress (1): 293 B

Downloaded from hiring-app: <http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/maven-metadata.xml> (293 B at 339 B/s)

Uploading to hiring-app: <http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/maven-metadata.xml>

Progress (1): 293 B

Uploaded to hiring-app: <http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/maven-metadata.xml> (293 B at 1.7 kB/s)

[INFO] -----

[INFO] BUILD SUCCESS

[INFO] -----

[INFO] Total time: 4.619 s

[INFO] Finished at: 2026-08-16T10:23:53Z

[INFO] -----

[hiring-app] \$ /bin/bash /var/lib/jenkins/tmp/jenkins211369987654152429.sh

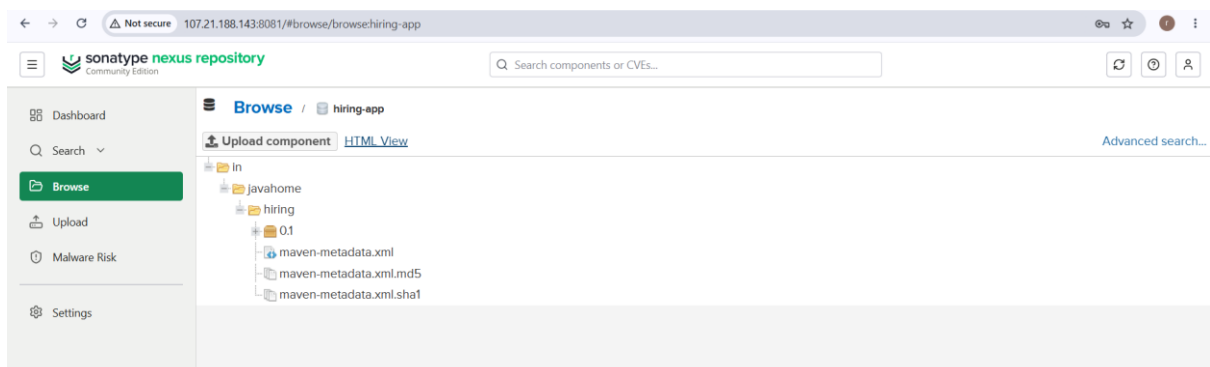
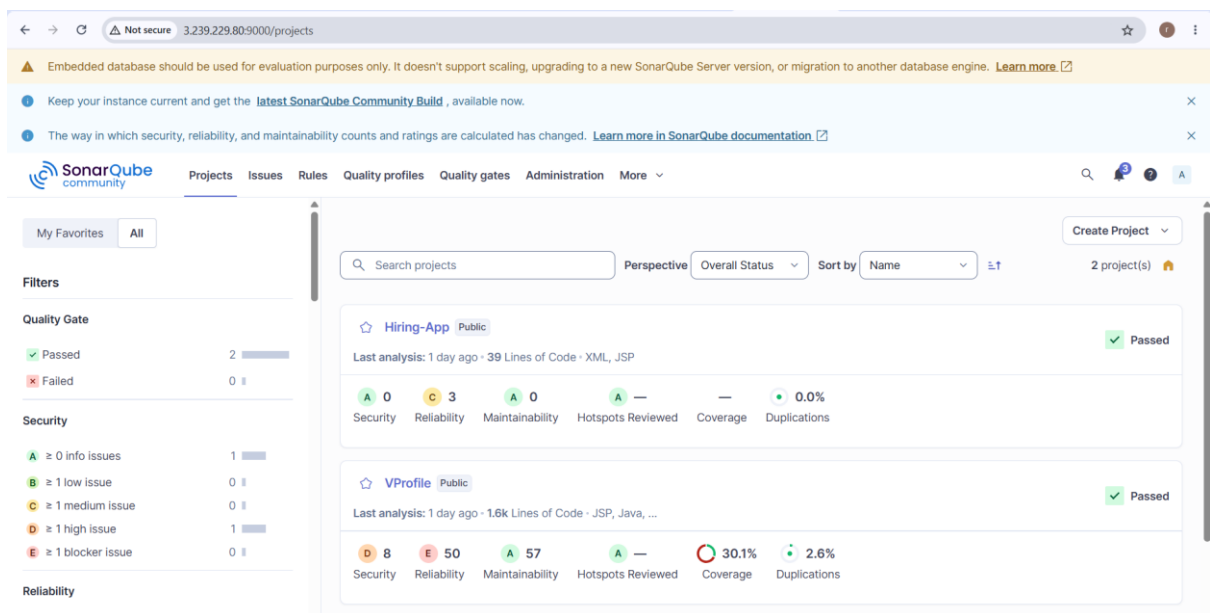
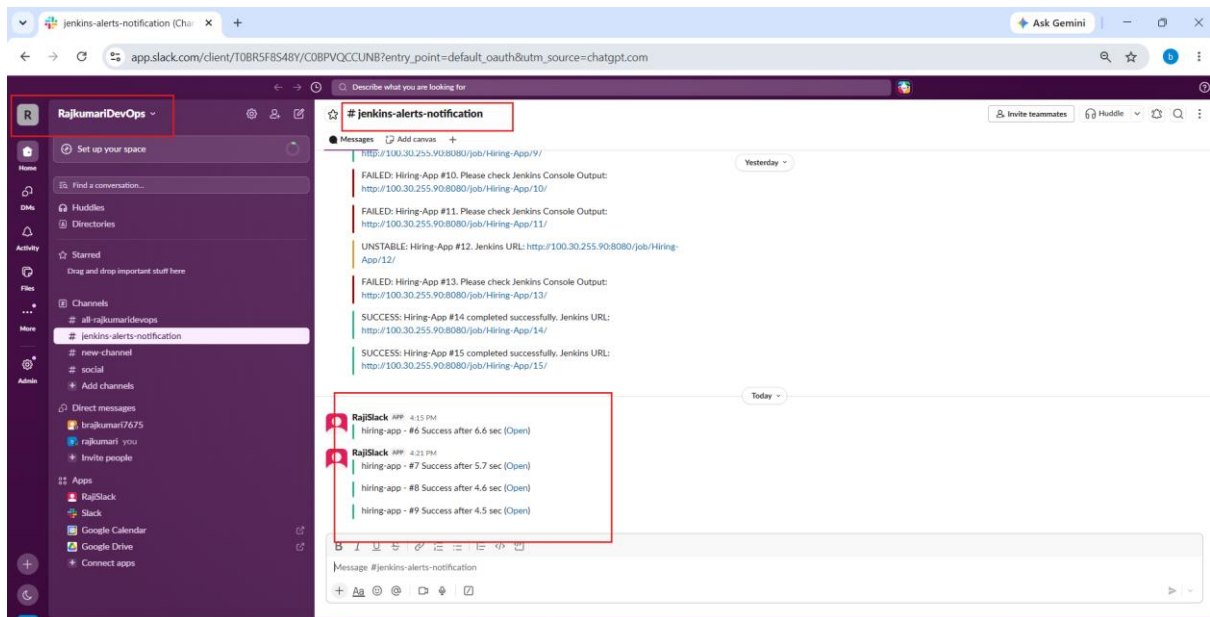
===== Deploying hiring.war to Tomcat =====

OK - Deployed application at context path [/hiring]

===== Tomcat deployment completed =====

Finished: SUCCESS

Jenkins 2.568.2



sonatype nexus repository

Search components or CVEs...

Dashboard

Search

Browse

Upload

Malware Risk

Settings

Browse / hiring-app

Upload component HTML View

Advanced search...

/in/javahome/hiring/0.1/hiring-0.1.war

Delete asset

Summary

Repository hiring-app

Format maven2

Component Group in.javahome

Component Name hiring

Component Version 0.1

Path /in/javahome/hiring/0.1/hiring-0.1.war

Content type application/java-archive

File size 1.7 KB

Blob created Sun Aug 16 2026 15:11:27 GMT+0530 (India Standard Time)

Blob updated Sun Aug 16 2026 15:53:51 GMT+0530 (India Standard Time)

Last downloaded has not been downloaded

Locally cached true

Tomcat Web Application Manager

Message: OK

Manager

List Applications HTML Manager Help Manager Help Server Status

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/hiring	None specified	Archetype Created Web Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy

Deploy directory or WAR file located on server

Context Path:

Welcome to Techie Horizon

2) Setup a Jenkins CICD pipeline using Declarative pipeline using feature-1.1 branch.

Source Code:

https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

stages: Git Clone

SonarQube Integration

Maven Compilation

Nexus Artifactory

Slack Notification

Deploy On tomcat

The required flow is:

GitHub (feature-1.1) → Git Clone → SonarQube → Maven Build → Nexus → Slack → Tomcat

Jenkins Declarative Pipeline must use the top-level pipeline {} structure, and Maven can be configured through Jenkins' tools directive.

Before starting: values you need

Keep these ready:

Item	Example
Git repository	https://github.com/betawins/sabear_simplecutomerapp.git
Branch	feature-1.1

Item	Example
Jenkins	Your Jenkins EC2
SonarQube	http://SONAR_PRIVATE_IP:9000
Nexus	http://NEXUS_PRIVATE_IP:8081
Tomcat	http://TOMCAT_PRIVATE_IP:8080

Slack workspace rajislack

Slack channel #jenkins-alerts-notification

Part 1 — Prepare Jenkins

Step 1 — Verify Jenkins server

SSH into your Jenkins EC2:

```
ssh -i your-key.pem ec2-user@<JENKINS-PUBLIC-IP>
```

Check:

```
java -version
```

```
mvn -version
```

```
git --version
```

You should get versions for all three.

For your existing setup, you already have Java/Maven/Git installed, so don't reinstall them if these commands work.

Step 2 — Check the Git branch

Run:

```
git ls-remote --heads https://github.com/betawins/sabear_simplecutomerapp.git
```

You should see:

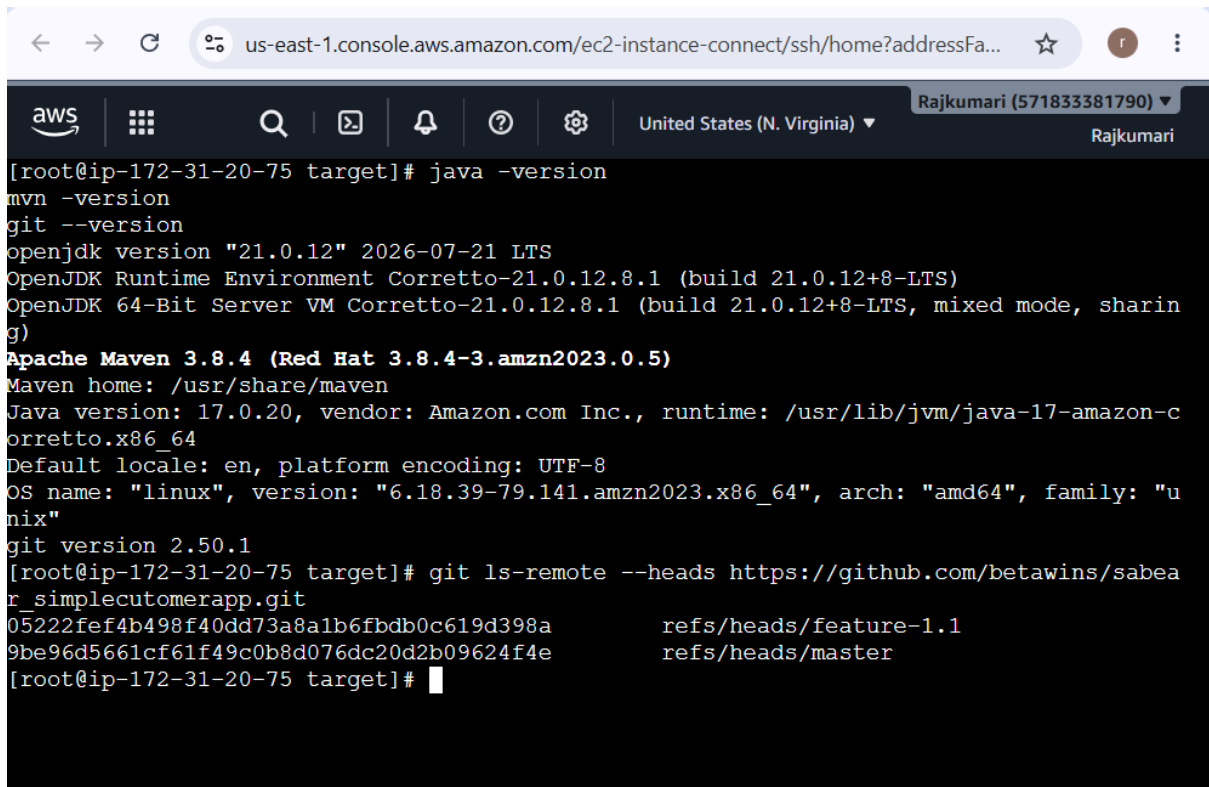
```
refs/heads/feature-1.1
```

This confirms the branch exists.

Your Jenkins pipeline will specifically clone:

feature-1.1

—not main and not master.



The screenshot shows a terminal window within the AWS Management Console. The browser address bar indicates the URL is `us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFa...`. The terminal header shows the AWS logo, navigation icons, and the user's location as "United States (N. Virginia)" and their ID as "Rajkumari (571833381790)". The terminal output shows the following commands and results:

```
[root@ip-172-31-20-75 target]# java -version
java version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)

[root@ip-172-31-20-75 target]# mvn -version
Apache Maven 3.8.4 (Red Hat 3.8.4-3.amzn2023.0.5)
Maven home: /usr/share/maven
Java version: 17.0.20, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-17-amazon-corretto.x86_64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.18.39-79.141.amzn2023.x86_64", arch: "amd64", family: "unix"

[root@ip-172-31-20-75 target]# git --version
git version 2.50.1

[root@ip-172-31-20-75 target]# git ls-remote --heads https://github.com/betawins/sabea
05222fef4b498f40dd73a8a1b6fbdb0c619d398a refs/heads/feature-1.1
9be96d5661cf61f49c0b8d076dc20d2b09624f4e refs/heads/master
```

Step 3 — Install Jenkins plugins

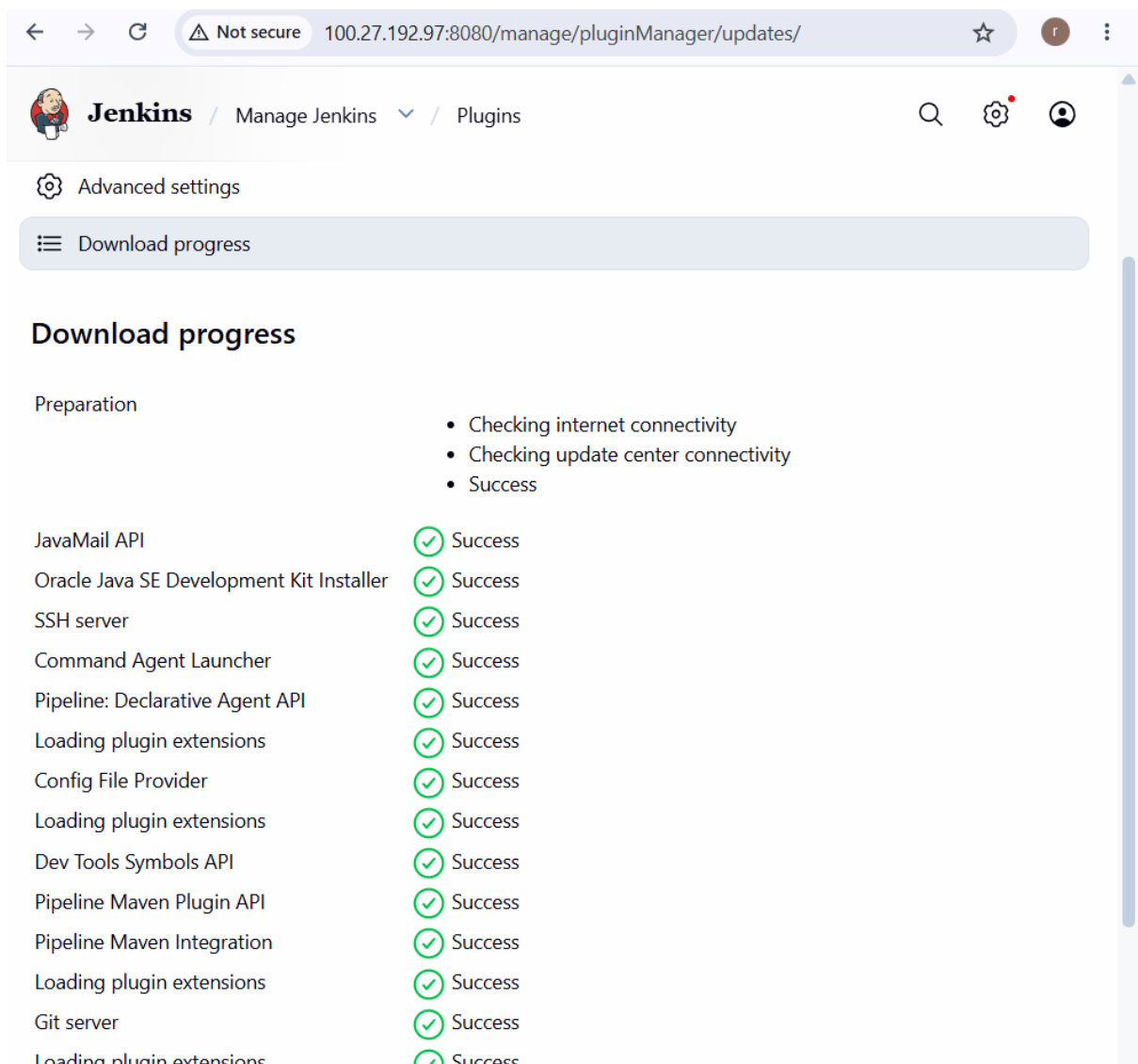
Go to:

Jenkins Dashboard → Manage Jenkins → Plugins → Available plugins

Make sure these are installed:

- Pipeline
- Pipeline: Declarative
- Git
- Git client
- SonarQube Scanner for Jenkins
- Credentials Binding
- Slack Notification
- Config File Provider
- Pipeline Maven Integration

Restart Jenkins if Jenkins asks you to.



The screenshot shows the Jenkins web interface at the URL `100.27.192.97:8080/manage/pluginManager/updates/`. The page title is "Jenkins / Manage Jenkins / Plugins". Below the title bar, there are links for "Advanced settings" and "Download progress". The "Download progress" section is active and shows a list of plugins and their download status. The status for all listed items is "Success".

Item	Status
Preparation	• Checking internet connectivity • Checking update center connectivity • Success
JavaMail API	✓ Success
Oracle Java SE Development Kit Installer	✓ Success
SSH server	✓ Success
Command Agent Launcher	✓ Success
Pipeline: Declarative Agent API	✓ Success
Loading plugin extensions	✓ Success
Config File Provider	✓ Success
Loading plugin extensions	✓ Success
Dev Tools Symbols API	✓ Success
Pipeline Maven Plugin API	✓ Success
Pipeline Maven Integration	✓ Success
Loading plugin extensions	✓ Success
Git server	✓ Success
Loading plugin extensions	✓ Success

Step 4 — Configure Maven

Go to:

Manage Jenkins → Tools

Find:

Maven installations

Click:

Add Maven

Use:

Name: Maven-3.8.4

If you are using the existing system Maven, you can also configure its installation path.

Your pipeline can then use:

```
tools {  
    maven 'Maven-3.8.4'  
}
```

Jenkins supports Maven under the Declarative Pipeline tools directive, provided the tool is configured under **Manage Jenkins** → **Tools**.

The screenshot shows the Jenkins web interface at the URL `100.27.192.97:8080/manage/configureTools/`. The page title is "Manage Jenkins" and the sub-section is "Tools". Under the "Maven installations" section, there is a list of installed tools. One tool named "Maven" is shown with the following configuration:

- Name: Maven
- MAVEN_HOME: /usr/share/maven
- Install automatically: ☐

At the bottom of the page, there are two buttons: "Save" and "Apply".

Part 2 — Configure SonarQube

Step 5 — Verify SonarQube

From Jenkins:


```
curl http://<SONARQUBE-PRIVATE-IP>:9000
```

You should receive SonarQube HTML output.

Also make sure your AWS Security Group allows:

Jenkins-SG → SonarQube-SG

TCP 9000

Step 6 — Configure SonarQube in Jenkins

Go to:

Manage Jenkins → System

Find:

SonarQube servers

Click:

Add SonarQube

Use:

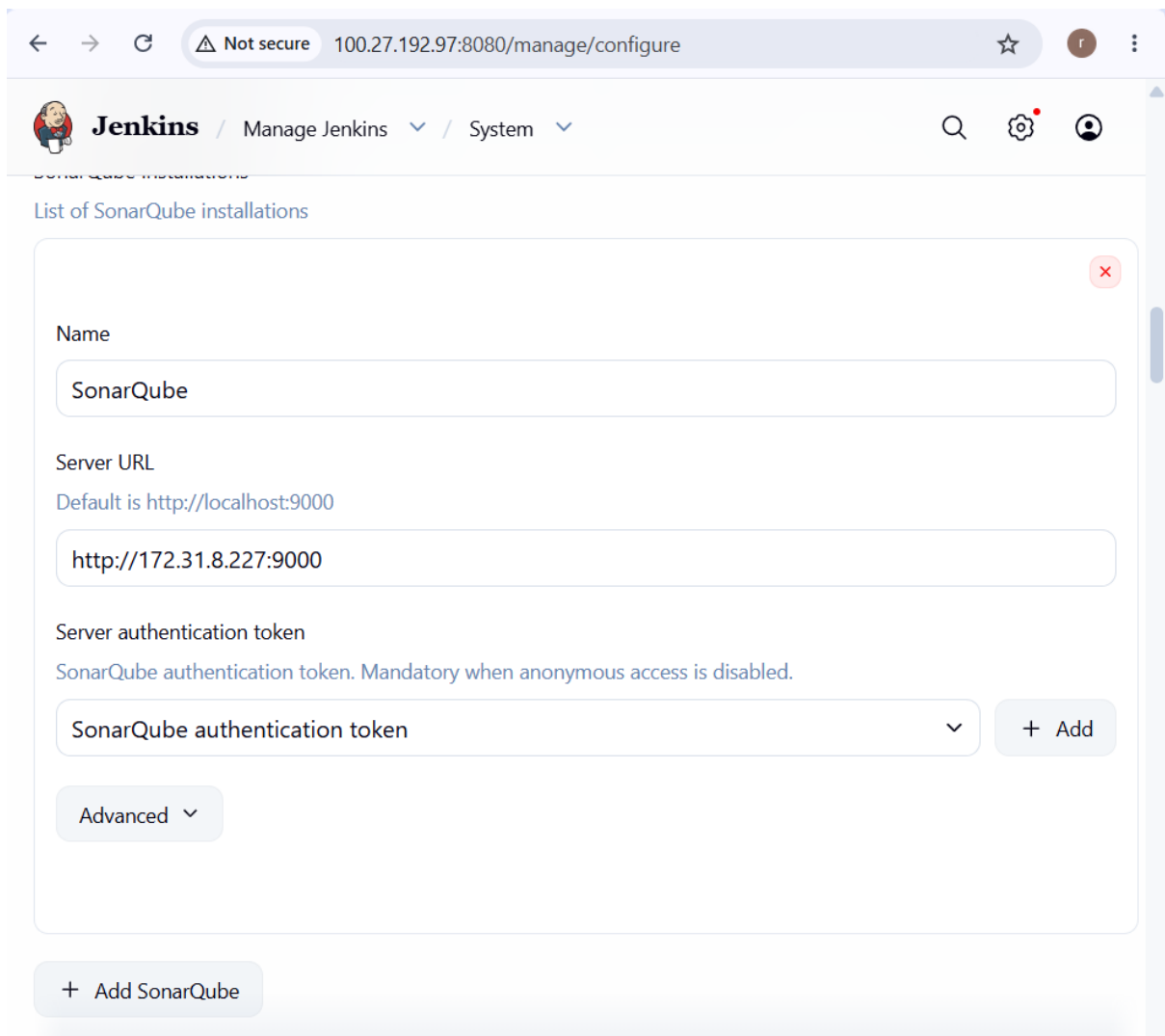
Name: SonarQube

Server URL: `http://<SONARQUBE-PRIVATE-IP>:9000`

Add the SonarQube authentication token through Jenkins Credentials if your SonarQube setup requires it.

The name **must match** what you later put in:

```
withSonarQubeEnv('SonarQube')
```



The screenshot shows the Jenkins web interface at the URL `100.27.192.97:8080/manage/configure`. The page title is "List of SonarQube installations". It features a configuration form for a SonarQube installation. The form includes a "Name" field with the value "SonarQube", a "Server URL" field with the value `http://172.31.8.227:9000`, and a "Server authentication token" field with the value "SonarQube authentication token". There is a "+ Add" button next to the token field and an "Advanced" dropdown menu. At the bottom of the form, there is a "+ Add SonarQube" button.

← → ↻ ⚠ Not secure 100.27.192.97:8080/manage/configure ☆

Jenkins / Manage Jenkins ▾ / System ▾ 🔍 ⚙️ 👤

SonarQube installations

List of SonarQube installations

Name

SonarQube

Server URL

Default is `http://localhost:9000`

`http://172.31.8.227:9000`

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

SonarQube authentication token ▾ + Add

Advanced ▾

+ Add SonarQube

Part 3 — Configure Nexus

Step 7 — Verify Nexus from Jenkins

From Jenkins:

```
curl http://<NEXUS-PRIVATE-IP>:8081
```

You should receive Nexus response/HTML.

Make sure AWS Security Group allows:

Jenkins-SG

↓

Nexus-SG

TCP 8081

Step 8 — Create Nexus repository

Open:

http://<NEXUS-PUBLIC-IP>:8081

Log in to Nexus.

Go to:

Settings → Repositories → Create repository

For a release application, create:

maven2 (hosted)

For example:

Name: hiring-releases

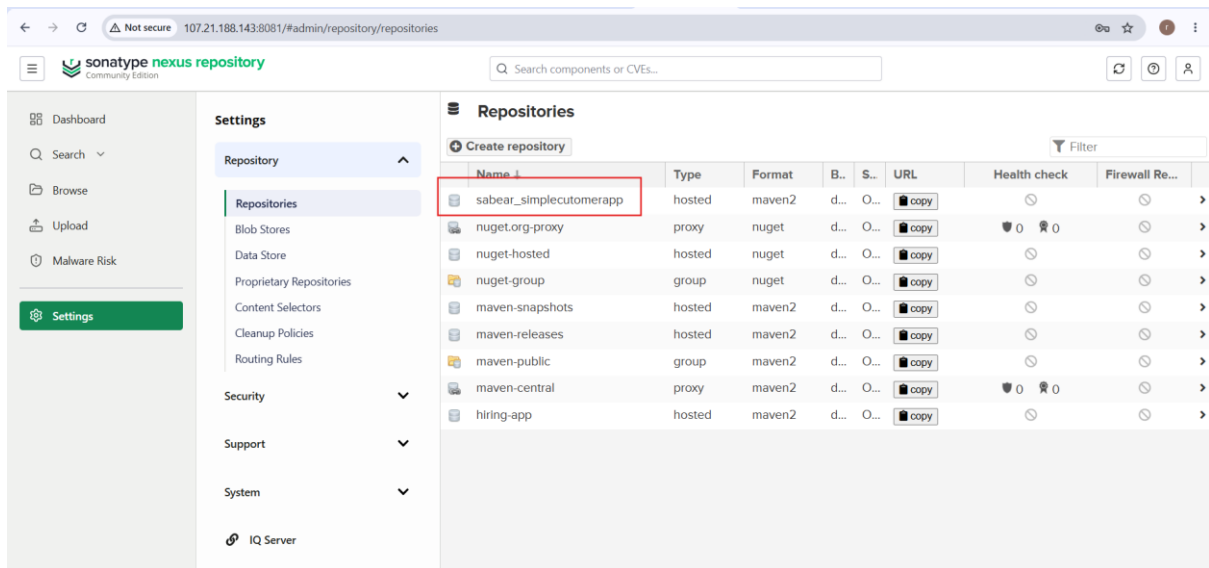
Version policy: Snapshot

Deployment policy: Allow redeploy

If your project's version ends in -SNAPSHOT, create/use a Maven snapshots repository instead.

The screenshot shows the Sonatype Nexus Repository Community Edition interface. The left sidebar contains navigation links: Dashboard, Search, Browse, Upload, Malware Risk, and Settings (highlighted). The main content area is titled 'Repositories' and shows the 'Create Repository: maven2 (hosted)' configuration page. The configuration includes the following fields:

- Name:** A unique identifier for this repository. Value: `sabear_simplecutomerapp`.
- Online:** ☒ If checked, the repository accepts incoming requests.
- Maven 2:**
 - Version policy:** What type of artifacts does this repository store? Value: `Snapshot`.
 - Layout policy:** Validate that all paths are maven artifact or metadata paths. Value: `Strict`.
 - Content Disposition:** Add Content-Disposition header as 'Attachment' to disable some content from being inline in a browser. Value: `Attachment`.
- Storage:**
 - Blob store:** Blob store used to store repository contents. Value: `default`.
 - Strict Content Type Validation:** ☒ Validate that all content uploaded to this repository is of a MIME type appropriate for the repository format.



Step 9 — Create Nexus Jenkins credential

In Jenkins:

Manage Jenkins → Credentials → System → Global credentials → Add Credentials

Select:

Kind: Username with password

Username: <NEXUS_USERNAME>

Password: <NEXUS_PASSWORD>

ID: nexus-credentials

Save it.

Don't put the Nexus password directly into your Jenkinsfile.

Jenkins' credentials binding mechanism is designed to expose credentials only within the scope where they're needed.

← → × Not secure 100.27.192.97:8080/manage/credentials/ ☆ r ⋮

Jenkins / Manage Jenkins ▾ / Credentials 🔍 ⚙️ 👤

Credentials

+ Add Credentials

?

sonarqube-token System - Global - sonarintegrate	⋮
slack-token System - Global - Rajislack Jenkins Bot Token	⋮
sonarhiringapp-token System - Global - SonarQube authentication token	⋮
nexus-credentials jenkins/***** System - Global - nexus-credentials	⋮

Stores scoped to Jenkins

Step 10 — Check the project's pom.xml

This is **very important** before running the Nexus stage.

Clone the project temporarily:

```
cd /tmp
```

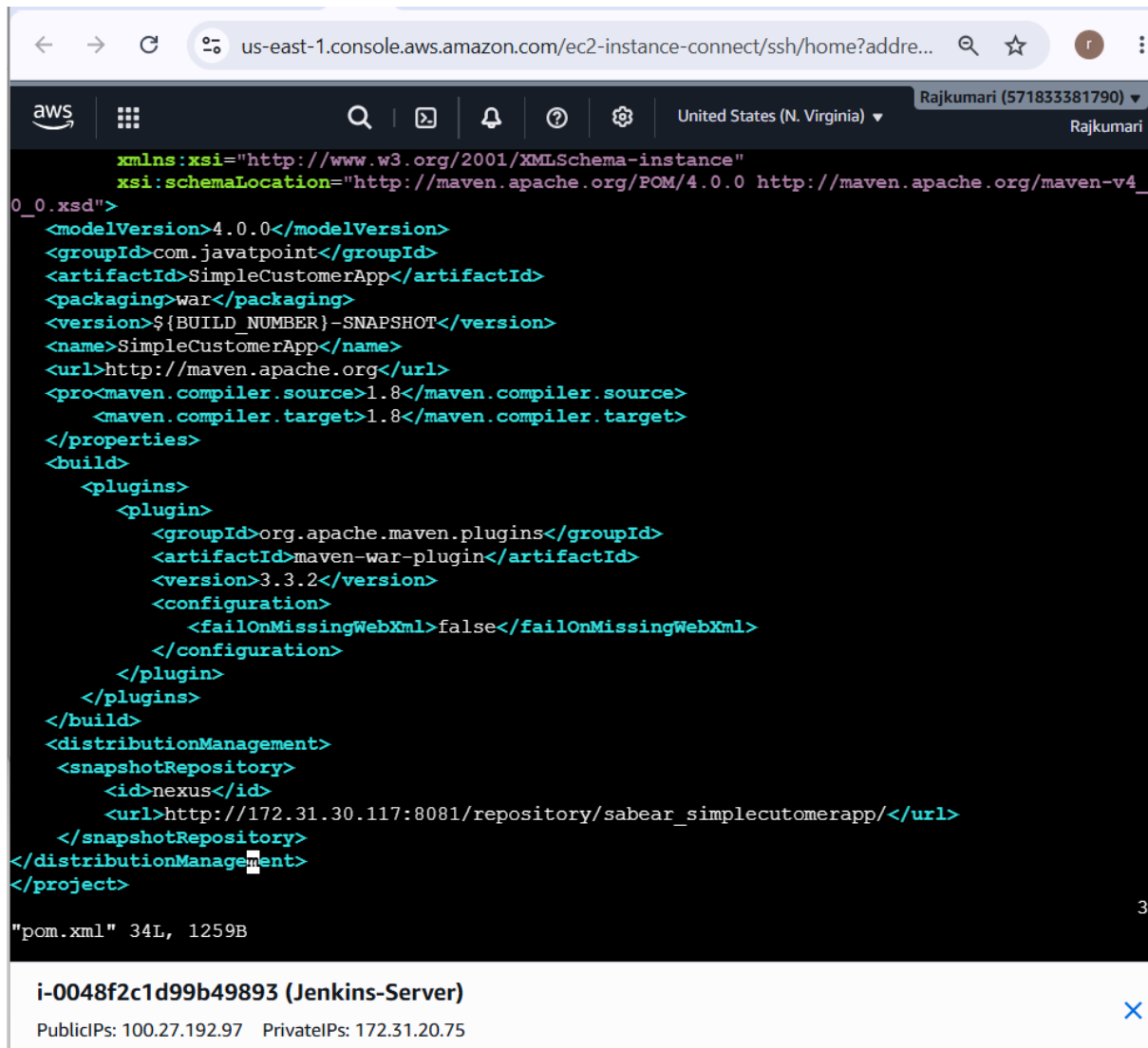
```
git clone -b feature-1.1 https://github.com/betawins/sabear_simplecutomerapp.git
```

```
cd sabear_simplecutomerapp
```

Check:

```
grep -n "distributionManagement" pom.xml
```

If the project doesn't already have Nexus distributionManagement, you'll need to configure it.



```
us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addre...
aws
United States (N. Virginia)
Rajkumari (571833381790)
Rajkumari

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.javatpoint</groupId>
  <artifactId>SimpleCustomerApp</artifactId>
  <packaging>war</packaging>
  <version>${BUILD_NUMBER}-SNAPSHOT</version>
  <name>SimpleCustomerApp</name>
  <url>http://maven.apache.org</url>
  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
  </properties>
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.3.2</version>
        <configuration>
          <failOnMissingWebXml>>false</failOnMissingWebXml>
        </configuration>
      </plugin>
    </plugins>
  </build>
  <distributionManagement>
    <snapshotRepository>
      <id>nexus</id>
      <url>http://172.31.30.117:8081/repository/sabear_simplecutomerapp/</url>
    </snapshotRepository>
  </distributionManagement>
</project>

"pom.xml" 34L, 1259B

i-0048f2c1d99b49893 (Jenkins-Server)
PublicIPs: 100.27.192.97 PrivateIPs: 172.31.20.75
```

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addre...

aws United States (N. Virginia) Rajkumari (571833381790) Rajkumari

```
Downloaded from central: https://repo.maven.apache.org/maven2/javax/inject/javax.inject/1/javax.inject-1.jar (2.5 kB at 6.0 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/aopalliance/aopalliance/1.0/aopalliance-1.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org.eclipse.sisu/org.eclipse.sisu.plexus/0.0.0.M2a/org.eclipse.sisu.plexus-0.0.0.M2a.jar (202 kB at 482 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org.eclipse.sisu/org.eclipse.sisu.inject/0.0.0.M2a/org.eclipse.sisu.inject-0.0.0.M2a.jar
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/code/findbugs/jsr305/1.3.9/jsr305-1.3.9.jar (33 kB at 77 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/asm/asm/3.3.1/asm-3.3.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/aopalliance/aopalliance/1.0/aopalliance-1.0.jar (4.5 kB at 9.8 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/asm/asm/3.3.1/asm-3.3.1.jar (44 kB at 89 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org.eclipse.sisu/org.eclipse.sisu.inject/0.0.0.M2a/org.eclipse.sisu.inject-0.0.0.M2a.jar (202 kB at 376 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonatype/sisu/sisu-guice/3.1.0/sisu-guice-3.1.0-no_aop.jar (357 kB at 626 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/10.0.1/guava-10.0.1.jar (1.5 MB at 2.4 MB/s)
[INFO] Packaging webapp
[INFO] Assembling webapp [SimpleCustomerApp] in [/tmp/sabear_simplecutomerapp/target/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT]
[INFO] Processing war project
[INFO] Building war: /tmp/sabear_simplecutomerapp/target/SimpleCustomerApp-${BUILD_NUMBER}-SNAPSHOT.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.699 s
[INFO] Finished at: 2026-08-16T11:48:49Z
[INFO] -----
root@ip-172-31-20-75 sabear_simplecutomerapp]#
```

i-0048f2c1d99b49893 (Jenkins-Server)

PublicIPs: 100.27.192.97 PrivateIPs: 172.31.20.75

Part 4 — Configure Maven → Nexus authentication

Step 11 — Create Maven settings.xml

Create a Maven settings file containing:

```
<settings>

  <servers>

    <server>

      <id>nexus</id>

      <username>YOUR_NEXUS_USERNAME</username>

      <password>YOUR_NEXUS_PASSWORD</password>

    </server>

  </servers>

</settings>
```

```
</servers>
```

```
</settings>
```

For your real assignment, don't commit this file containing a password to GitHub.

Instead, use Jenkins Credentials + Config File Provider.

Step 12 — Add settings.xml to Jenkins

Go to:

Manage Jenkins → Managed files

Create a Maven settings file.

For example:

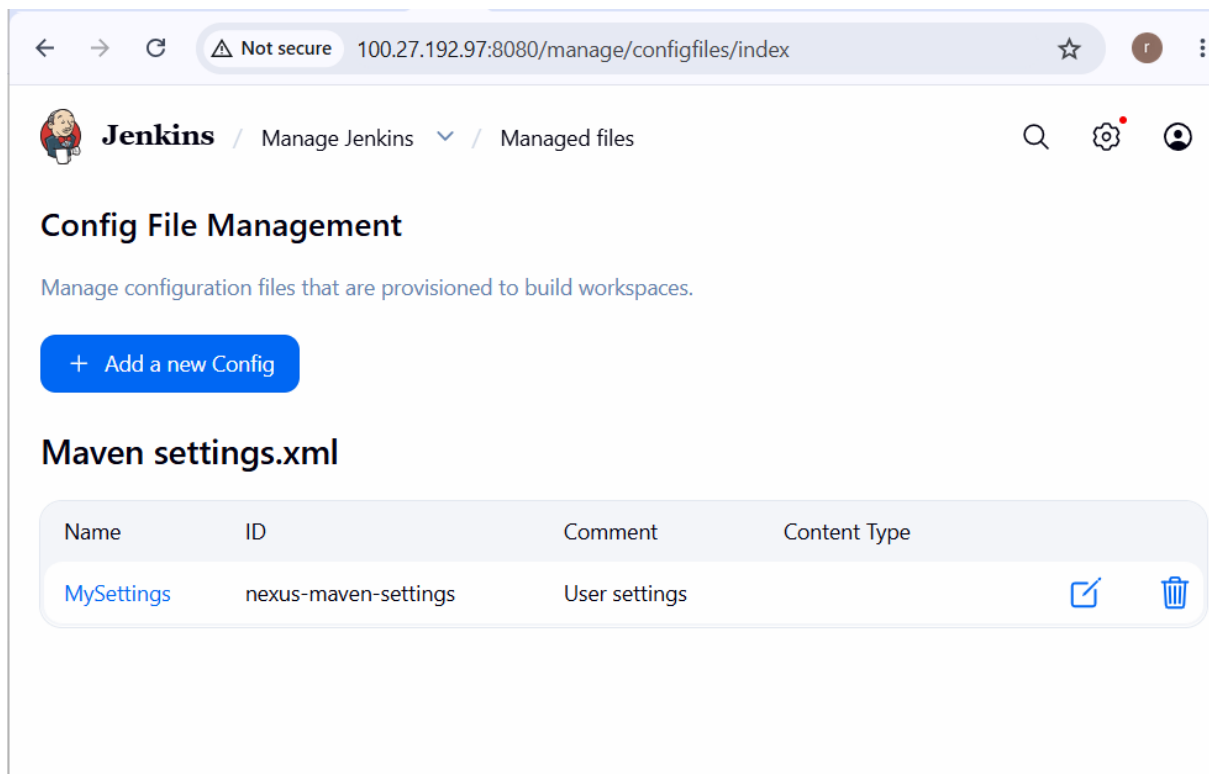
ID: nexus-settings

Configure the Nexus credentials appropriately.

Your pipeline can then use:

```
withMaven(  
    mavenSettingsConfig: 'nexus-settings'  
)
```

This keeps Nexus credentials outside your Jenkinsfile.



Part 5 — Prepare Tomcat

Step 13 — Verify Tomcat

SSH into your Tomcat server:

```
ssh -i your-key.pem ec2-user@<TOMCAT-PUBLIC-IP>
```

Check:

```
java -version
```

Then:

```
sudo systemctl status tomcat
```

Test:

```
curl http://localhost:8080
```

You should get the Tomcat page/response.

Step 14 — Configure Tomcat Manager

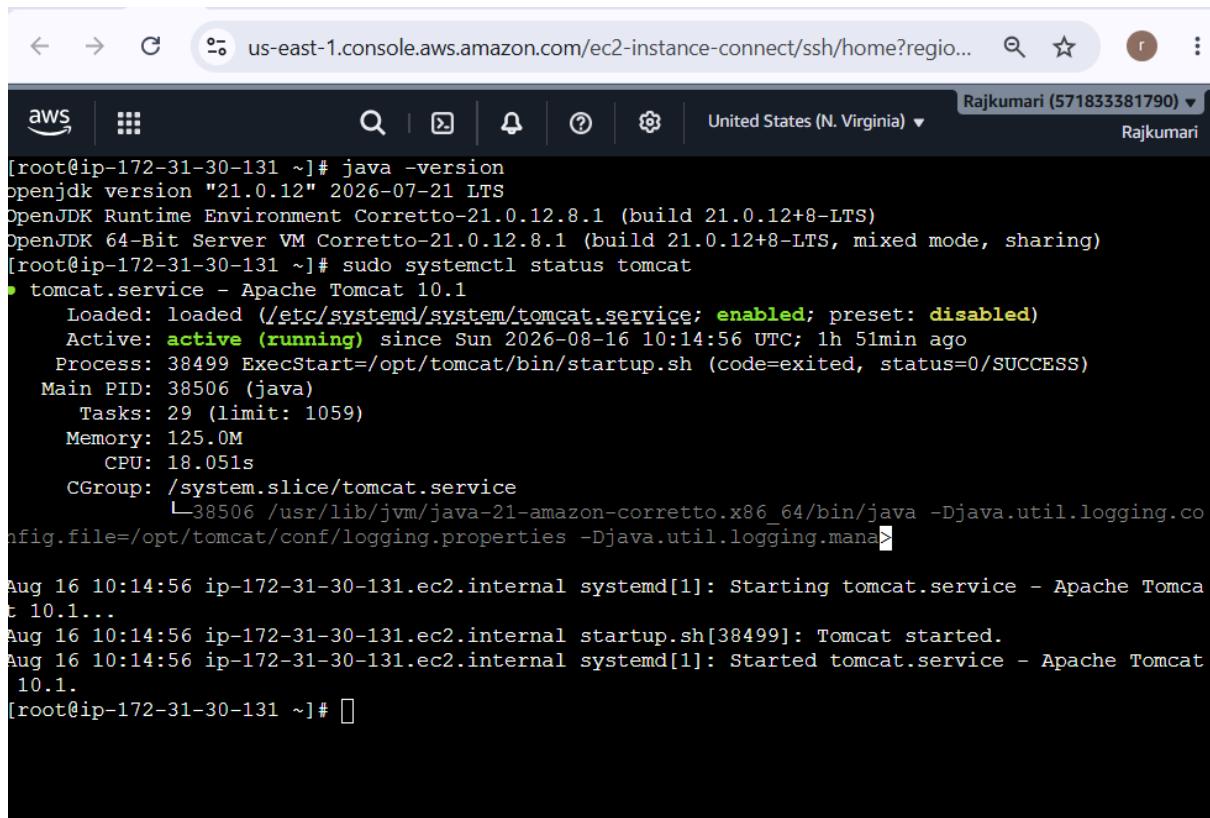
For automated WAR deployment, Jenkins can use Tomcat's Manager text interface.

Test from Jenkins:

```
curl http://<TOMCAT-PRIVATE-IP>:8080/manager/text/list
```

If authentication is required, use your Tomcat Manager credentials.

Tomcat's Manager application supports remote WAR deployment using an HTTP PUT request to `/manager/text/deploy?path=/....`



The screenshot shows an AWS console terminal window with the following content:

```
[root@ip-172-31-30-131 ~]# java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[root@ip-172-31-30-131 ~]# sudo systemctl status tomcat
● tomcat.service - Apache Tomcat 10.1
   Loaded: loaded (/etc/systemd/system/tomcat.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-08-16 10:14:56 UTC; 1h 51min ago
     Process: 38499 ExecStart=/opt/tomcat/bin/startup.sh (code=exited, status=0/SUCCESS)
    Main PID: 38506 (java)
      Tasks: 29 (limit: 1059)
     Memory: 125.0M
        CPU: 18.051s
      CGroup: /system.slice/tomcat.service
              └─38506 /usr/lib/jvm/java-21-amazon-corretto.x86_64/bin/java -Djava.util.logging.co
nfig.file=/opt/tomcat/conf/logging.properties -Djava.util.logging.mana
Aug 16 10:14:56 ip-172-31-30-131.ec2.internal systemd[1]: Starting tomcat.service - Apache Tomcat 10.1...
Aug 16 10:14:56 ip-172-31-30-131.ec2.internal startup.sh[38499]: Tomcat started.
Aug 16 10:14:56 ip-172-31-30-131.ec2.internal systemd[1]: Started tomcat.service - Apache Tomcat 10.1.
[root@ip-172-31-30-131 ~]#
```

Step 15 — Create Tomcat deployment user

On Tomcat:

```
sudo vi /opt/tomcat/conf/tomcat-users.xml
```

Add an appropriate Manager role/user for your Tomcat version, for example:

```
<role rolename="manager-script"/>
```

```
<user username="jenkins"
```

```
    password="YOUR_PASSWORD"
```

```
    roles="manager-script"/>
```

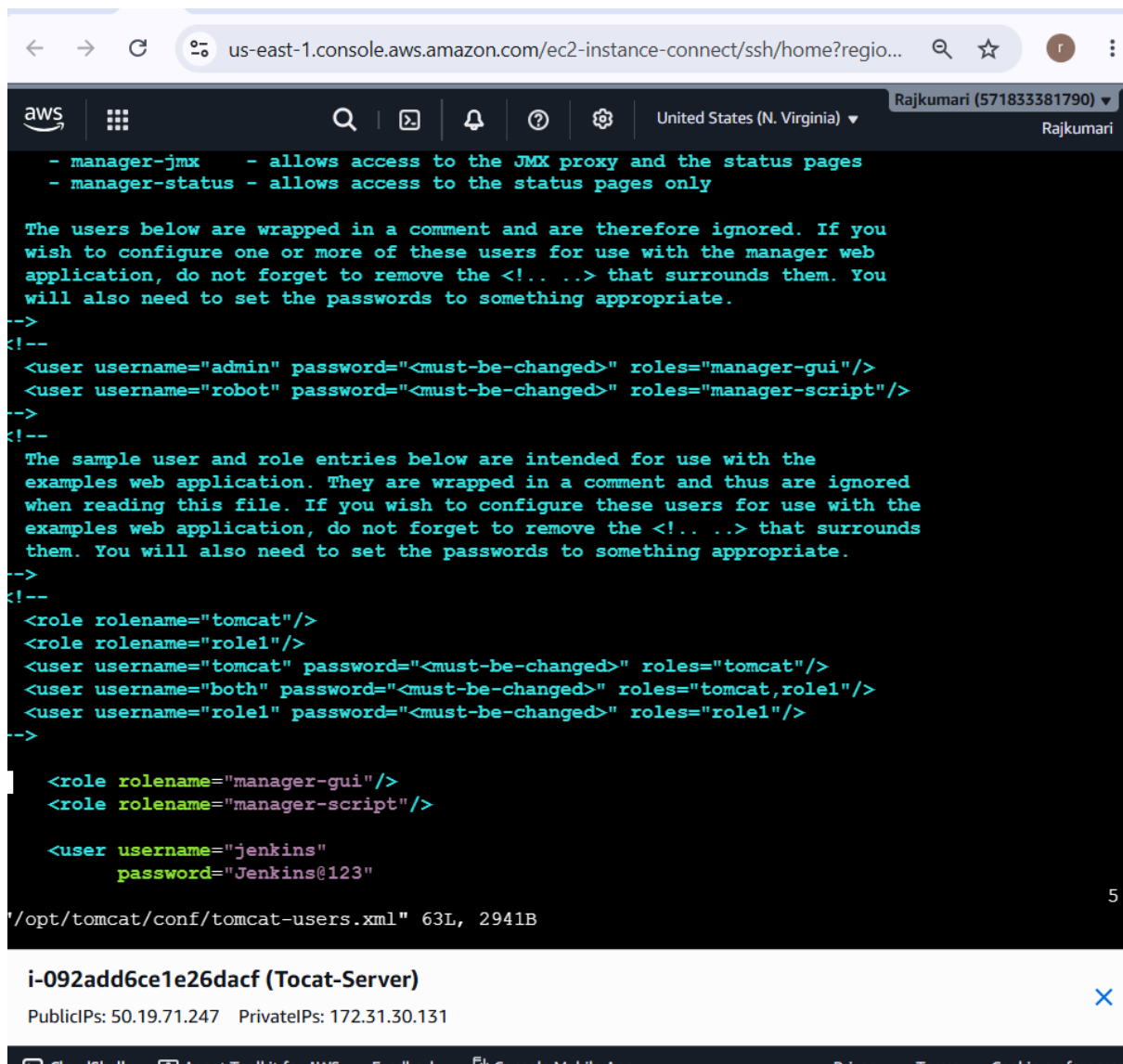
Then restart Tomcat:

```
sudo systemctl restart tomcat
```

Verify:

sudo systemctl status tomcat

Note: Tomcat Manager access restrictions may also need adjustment so that your Jenkins server can reach the Manager application.



The screenshot shows an AWS Management Console terminal window for an EC2 instance. The terminal displays the contents of the file `/opt/tomcat/conf/tomcat-users.xml`. The file contains XML configuration for Tomcat users and roles. Comments explain that users and roles are wrapped in `<!-- ... -->` to be ignored. The configuration includes roles for `manager-gui` and `manager-script`, and users for `admin`, `robot`, `tomcat`, `both`, `role1`, and `jenkins`. The `jenkins` user is configured with the password `Jenkins@123`. The terminal output ends with the file path and size: `/opt/tomcat/conf/tomcat-users.xml" 63L, 2941B`.

```
aws | [Search] | [Copy] | [Refresh] | [Help] | [Settings] | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari
```

```
- manager-jmx      - allows access to the JMX proxy and the status pages
- manager-status   - allows access to the status pages only

The users below are wrapped in a comment and are therefore ignored. If you
wish to configure one or more of these users for use with the manager web
application, do not forget to remove the <!-- ... --> that surrounds them. You
will also need to set the passwords to something appropriate.
-->
<!--
<user username="admin" password="<must-be-changed>" roles="manager-gui"/>
<user username="robot" password="<must-be-changed>" roles="manager-script"/>
-->
<!--
The sample user and role entries below are intended for use with the
examples web application. They are wrapped in a comment and thus are ignored
when reading this file. If you wish to configure these users for use with the
examples web application, do not forget to remove the <!-- ... --> that surrounds
them. You will also need to set the passwords to something appropriate.
-->
<!--
<role rolename="tomcat"/>
<role rolename="role1"/>
<user username="tomcat" password="<must-be-changed>" roles="tomcat"/>
<user username="both" password="<must-be-changed>" roles="tomcat,role1"/>
<user username="role1" password="<must-be-changed>" roles="role1"/>
-->

<role rolename="manager-gui"/>
<role rolename="manager-script"/>

<user username="jenkins"
      password="Jenkins@123"

/opt/tomcat/conf/tomcat-users.xml" 63L, 2941B
```

i-092add6ce1e26dacf (Tocat-Server) [X]

PublicIPs: 50.19.71.247 PrivateIPs: 172.31.30.131

CloudShell | Agent Toolkit for AWS | Feedback | Console Mobile App | Privacy | Terms | Cookie preferences

Step 16 — Create Tomcat Jenkins credential

Go to:

Jenkins → Manage Jenkins → Credentials → Global → Add Credentials

Create:

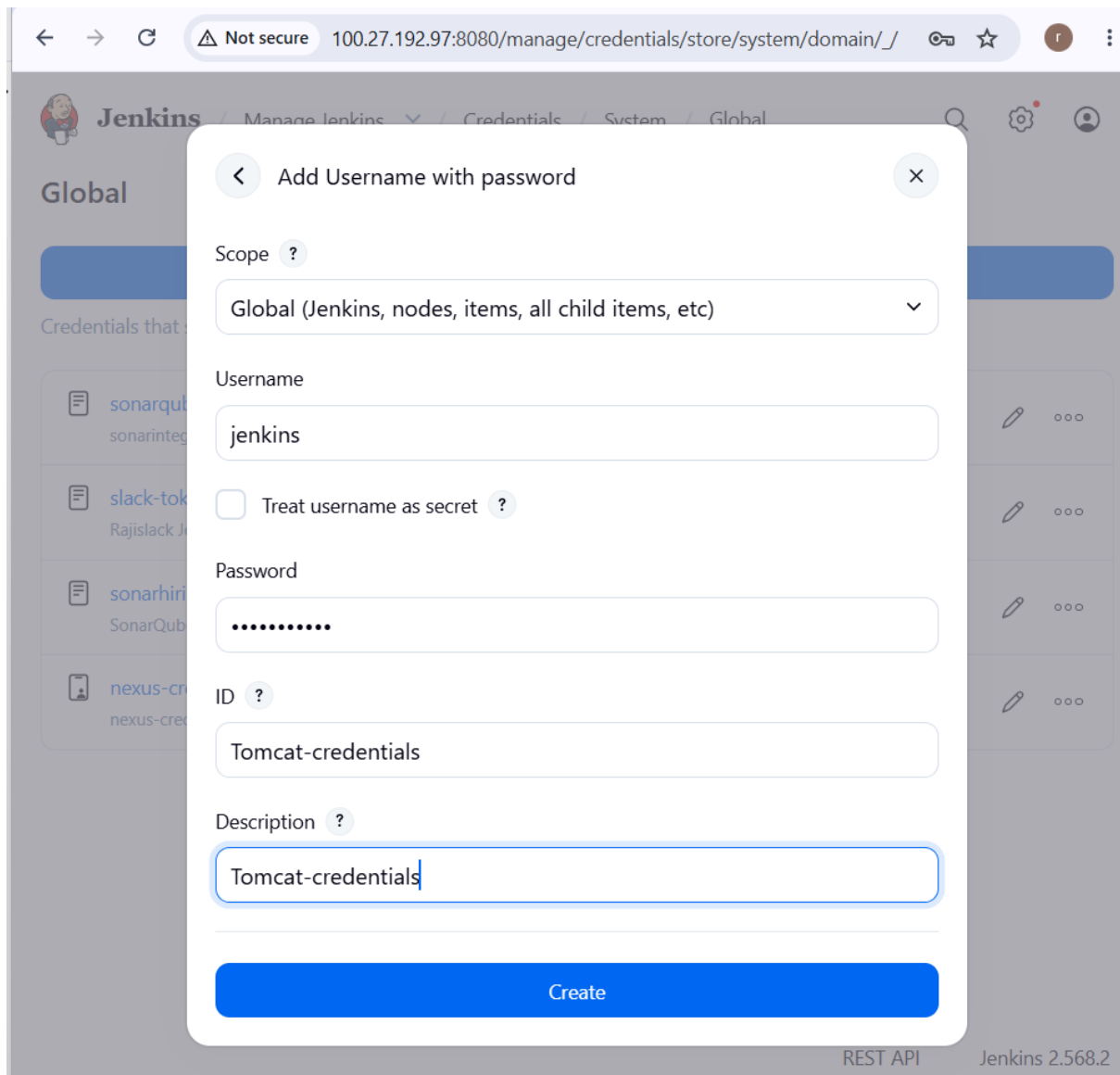
Kind: Username with password

Username: jenkins

Password: YOUR_TOMCAT_PASSWORD

ID: tomcat-credentials

Save it.



The screenshot shows the Jenkins web interface with a modal dialog box titled "Add Username with password". The dialog is open over the "Global" credentials page. The form fields are as follows:

- Scope**: A dropdown menu showing "Global (Jenkins, nodes, items, all child items, etc)".
- Username**: A text input field containing "jenkins".
- Treat username as secret**: An unchecked checkbox.
- Password**: A password input field with masked characters (dots).
- ID**: A text input field containing "Tomcat-credentials".
- Description**: A text input field containing "Tomcat-credentials".

At the bottom of the dialog is a blue "Create" button. The background shows the Jenkins interface with a list of credentials on the left and a search bar on the right. The footer of the page indicates "REST API" and "Jenkins 2.568.2".

Part 6 — Configure Slack

Step 17 — Configure your existing Slack workspace

Use the workspace you recovered:

Workspace: rajislack

Channel: #jenkins-alerts-notification

In Jenkins, configure the Slack notification plugin under:

Manage Jenkins → System

Configure the Slack workspace/token credential.

Your pipeline will use:

```
slackSend(  
    channel: '#jenkins-alerts-notification'  
)
```

You can test Slack before running the entire pipeline.

Part 7 — Create Jenkins Pipeline

Step 18 — Create the Pipeline job

Go to:

Jenkins Dashboard → New Item

Enter:

sabear-simplecustomerapp-feature-1.1

Select:

Pipeline

Click:

OK

Then scroll to:

Pipeline → Definition

Select:

Pipeline script

We are intentionally using a **Declarative Pipeline**, not a Scripted Pipeline. Jenkins requires a Declarative Pipeline to have the pipeline {} top-level structure.

Step 10 — Check Nexus after successful deployment

After Jenkins reports:

BUILD SUCCESS

open:

Nexus → Repositories → sabear_simplecutomerapp

You should see something similar to:

com/

└── javatpoint/

└── SimpleCustomerApp/

└── 8-SNAPSHOT/

└── SimpleCustomerApp-8-...war

└── SimpleCustomerApp-8-...pom

└── maven-metadata.xml

That confirms your **Nexus Artifactory stage is working**.

Pipeline in Declarative

```
pipeline {
```

```
    agent any
```

```
    tools {
```

```
        maven 'Maven'
```

```
    }
```

```
    environment {
```

```
        NEXUS_URL = 'http://172.31.30.117:8081/repository/sabear_simplecutomerapp/'
```

```
        TOMCAT_URL = 'http://172.31.30.131:8080'
```

```

APP_NAME = 'sabear'
}

stages {

    stage('Git Clone') {
        steps {
            git(
                branch: 'feature-1.1',
                url: 'https://github.com/rajkumari-hub/sabear_simplecutomerapp.git'
            )

            sh '''
                echo "==== Git Information ====="

                git branch

                git log -1 --oneline
            '''
        }
    }

    stage('SonarQube Integration') {
        steps {
            withSonarQubeEnv('SonarQube') {
                sh '''

                    mvn clean verify org.sonarsource.scanner.maven:sonar-maven-
plugin:sonar \

```

```
        -DBUILD_NUMBER=${BUILD_NUMBER}
    ""
}
}
}
```

```
stage('Maven Compilation') {
    steps {
        sh ""
            mvn clean package -DskipTests
        ""
    }
}
```

```
stage('Nexus Artifactory') {
    steps {
        withMaven(
            mavenSettingsConfig: 'nexus-settings'
        ) {
            sh ""
                mvn deploy -DskipTests
            ""
        }
    }
}
```



```
stage('Slack Notification') {  
    steps {  
        slackSend(  
            channel: '#jenkins-alerts-notification',  
            color: 'good',  
            message: "Nexus deployment completed: ${JOB_NAME}  
#${BUILD_NUMBER}"  
        )  
    }  
}
```

```
stage('Deploy On Tomcat') {  
    steps {  
  
        withCredentials([  
            usernamePassword(  
                credentialsId: 'tomcat-credentials',  
                usernameVariable: 'TOMCAT_USER',  
                passwordVariable: 'TOMCAT_PASSWORD'  
            )  
        ]) {  
  
            sh ""  
                echo "===== WAR FILE ====="  
                ls -lh target/*.war
```

```
echo "===== Deploying to Tomcat ====="
```

```
curl -v \
```

```
-u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
```

```
-T target/*.war \
```

```
"$TOMCAT_URL/manager/text/deploy?path=/$APP_NAME&update=true"
```

```
""
```

```
}
```

```
}
```

```
}
```

```
}
```

```
post {
```

```
    success {
```

```
        slackSend(
```

```
            channel: '#jenkins-alerts-notification',
```

```
            color: 'good',
```

```
            message: "SUCCESS: ${JOB_NAME} #${BUILD_NUMBER} - Application  
deployed successfully to Tomcat."
```

```
        )
```

```
    }
```

```
    failure {
```

```
        slackSend(
```

```
            channel: '#jenkins-alerts-notification',
```

```

        color: 'danger',

        message: "FAILED: ${JOB_NAME} #${BUILD_NUMBER} - Pipeline failed.
Check Jenkins console."

    )
}

always {

    echo "==== Pipeline Finished ====="

}

}

}

```

← → ↻ ⚠ Not secure 100.27.192.97:8080/job/sabear-simplecustomerapp-feature-1.1/configure ☆

Jenkins / sabear-simplecustomerapp... / Configure 🔍 ⚙️ 👤

Define your pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script ▾

Script ?

```

1 pipeline {
2     agent any
3
4     tools {
5         maven 'Maven'
6     }
7
8     environment {
9         NEXUS_URL = 'http://172.31.30.117:8081/repository/sabear_simplecutomerapp/'
10        TOMCAT_URL = 'http://172.31.30.131:8080'
11        APP_NAME = 'sabear'
12    }
13
14    stages {
15

```

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

← → ↻ Not secure 100.27.192.97:8080/job/sabear-simplecustomerapp-feature-1.1/

Jenkins sabear-simplecustomerapp...

Status

</> Changes

▶ Build Now

🗑 Delete Pipeline

⚙ Configure

✎ Rename

🔍 SonarQube

📅 Stages

✓ Maven

🔗 Pipeline Syntax

📦 **sabear-simplecustomerapp-feature-1.1** 🔍 Add description

📦 Last Successful Artifacts

- SimpleCustomerApp-14-SNAPSHOT.pom 1.23 KB [🔍 view](#)
- SimpleCustomerApp-14-SNAPSHOT.war 1.47 KB [🔍 view](#)

SonarQube Quality Gate

SimpleCustomerApp **Passed**

server-side processing: **Success**

Permalinks

- Last build (#14), 3 min 45 sec ago
- Last stable build (#14), 3 min 45 sec ago
- Last successful build (#14), 3 min 45 sec ago
- Last failed build (#13), 11 min ago
- Last unsuccessful build (#13), 11 min ago
- Last completed build (#14), 3 min 45 sec ago

Builds > ...

🔍 Filter /

Today

- 🟢 #14 2:46 PM [🔍](#) [📄](#) [🔍](#)
- 🔴 #13 2:38 PM [🔍](#) [📄](#) [🔍](#)
- 🟢 #12 2:13 PM [🔍](#) [📄](#) [🔍](#)
- 🟢 #11 2:09 PM [🔍](#) [📄](#) [🔍](#)
- 🟢 #10 2:07 PM [🔍](#) [📄](#) [🔍](#)
- 🔴 #9 1:54 PM [🔍](#) [📄](#) [🔍](#)

← → ↻ Not secure 100.27.192.97:8080

Jenkins

+ New Item

📅 Build History

🔗 Project Relationship

🔍 Check File Fingerprint

Build Queue ▼

No builds in the queue.

Build Executor Status ^

(0 of 2 executors busy)

All +

S	W	Name 1	Last Success	Last Failure	Last Duration	
🟢	☀	hiring-app	3 hr 58 min #9	5 hr 48 min #3	4.5 sec	▶
🟢	☁	sabear-simplecustomerapp-feature-1.1	4 min 15 sec #14	11 min #13	38 sec	▶
🟢	☁	VProfile-Pipeline	1 day 20 hr #28	1 day 21 hr #24	49 sec	▶

Icon: S M L ...

← → ↻ Not secure 100.27.192.97:8080/job/sabear-simplecustomerapp-feature-1.1/14/console

Jenkins sabear-simplecustomerapp... ▼ / #14 ▼

```
/:running:0:ROOT
/examples:running:0:examples
/sabear:running:0:sabear
/host-manager:running:0:host-manager
/hiring:running:1:hiring
/manager:running:1:manager
/docs:running:0:docs
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
===== Pipeline Finished =====
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: rajkumaridevops, channel: #jenkins-alerts-notification, color: good, botUser: true,
tokenCredentialId: slack-token, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

← → ↻ Not secure 3.239.229.80:9000/projects

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

1 Keep your instance current and get the [latest SonarQube Community Build](#), available now.

1 The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

My Favorites All

Filters

Quality Gate

✓ Passed 3

✗ Failed 0

Security

A ≥ 0 Info issues 2

B ≥ 1 low issue 0

C ≥ 1 medium issue 0

D ≥ 1 high issue 1

E ≥ 1 blocker issue 0

Reliability

SimpleCustomerApp Public ✓ Passed

Last analysis: 1 minute ago · 34 Lines of Code · XML

A 0 A 0 A 0 A — — 0.0%

Security Reliability Maintainability Hotspots Reviewed Coverage Duplications

VProfile Public ✓ Passed

Last analysis: 1 day ago · 1.6k Lines of Code · JSP, Java, ...

D 8 E 50 A 57 A — 30.1% 2.6%

Security Reliability Maintainability Hotspots Reviewed Coverage Duplications

4 of 4 shown

SonarQube™ technology is powered by SonarSource Sàrl Community Build · v26.7.0.124771 · MGR MODE LGPL v3 Community Documentation Plugins Web API

← → ↻ Not secure 107.21.188.143:8081/#browse/browse:sabear_simplecutomerapp

sonatype nexus repository Community Edition

Search components or CVEs...

Dashboard

Search

Browse

Upload

Malware Risk

Settings

Browse / sabear_simplecutomerapp

HTML View

Advanced search...

com

javatpoint

SimpleCustomerApp

10-SNAPSHOT

11-SNAPSHOT

12-SNAPSHOT

14-SNAPSHOT

14-20260816.144652-1

SimpleCustomerApp-14-20260816.144652-1.pom

SimpleCustomerApp-14-20260816.144652-1.pom.md5

SimpleCustomerApp-14-20260816.144652-1.pom.sha1

SimpleCustomerApp-14-20260816.144652-1.war

SimpleCustomerApp-14-20260816.144652-1.war.md5

SimpleCustomerApp-14-20260816.144652-1.war.sha1

maven-metadata.xml

maven-metadata.xml.md5

maven-metadata.xml.sha1

9-SNAPSHOT

maven-metadata.xml

maven-metadata.xml.md5

maven-metadata.xml.sha1

The top screenshot shows the Tomcat Web Application Manager interface. The 'Applications' table lists several deployed applications, with 'sabear' highlighted in red. The bottom screenshot shows a Jenkins CI/CD pipeline log in a Slack channel, displaying deployment status messages for 'sabear-simplecustomerapp-feature-1.1'.

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/hiring	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/sabear	None specified		true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

The bottom screenshot shows a Jenkins CI/CD pipeline log in a Slack channel. The log displays deployment status messages for 'sabear-simplecustomerapp-feature-1.1'.

```

#jenkins-alerts-notification
FAILED: sabear-simplecustomerapp-feature-1.1 #9 - Pipeline failed. Check Jenkins console.
RajSlack 8PM 7:30 PM
Nexus deployment completed: sabear-simplecustomerapp-feature-1.1 #10
SUCCESS: sabear-simplecustomerapp-feature-1.1 #10 - Application deployed successfully to Tomcat.
Nexus deployment completed: sabear-simplecustomerapp-feature-1.1 #11
SUCCESS: sabear-simplecustomerapp-feature-1.1 #11 - Application deployed successfully to Tomcat.
Nexus deployment completed: sabear-simplecustomerapp-feature-1.1 #12
SUCCESS: sabear-simplecustomerapp-feature-1.1 #12 - Application deployed successfully to Tomcat.
RajSlack 8PM 8:08 PM
FAILED: sabear-simplecustomerapp-feature-1.1 #13 - Pipeline failed. Check Jenkins console.
RajSlack 8PM 8:16 PM
Nexus deployment completed: sabear-simplecustomerapp-feature-1.1 #14
SUCCESS: sabear-simplecustomerapp-feature-1.1 #14 - Application deployed successfully to Tomcat.
RajSlack 8PM 8:39 PM
FAILED: sabear-simplecustomerapp-feature-1.1 - scripted
  
```

3) Setup a Jenkins CI/CD pipeline using Scripted pipeline using feature-1.1 branch.

Source Code:

https://github.com/betawins/sabear_simplecustomerapp/tree/feature-1.1

stages:

- **Git Clone**
- **SonarQube Integration**
- **Maven Compilation**
- **Nexus Artifactory**
- **Slack Notification**
- **Deploy On tomcat**

1. Create the Jenkins Pipeline job

Go to:

Jenkins → New Item

Enter:

sabear-simplecustomerapp-feature-1.1-scripted

Select:

Pipeline

Click **OK**.

Scroll to **Pipeline**.

Set:

Definition: Pipeline script

Then paste the following script.

← → ↻ ⚠ Not secure 100.27.192.97:8080/view/all/newJob 🔍 ☆ 👤 ⋮

 **Jenkins** / All ▾ / New Item 🔍 ⚙️ 👤

New Item

Enter an item name

sabear-simplecustomerapp-feature-1.1-scripted

Select an item type

 **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

 **Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

2. Complete Scripted Pipeline

Use this version:

The screenshot shows the Jenkins web interface. The browser address bar indicates a 'Not secure' connection to '100.27.192.97:8080/job/sabear-simplecutomerapp-feature-1.1-...'. The Jenkins logo and the job name 'sabear-simplecutomerapp...' are visible in the header, along with a 'Configure' link. Below the header, the 'Definition' section is active, showing a 'Pipeline script' dropdown. The script editor contains the following Groovy code:

```
100      echo "==== WAR FILE ===="
101      ls -lh target/*.war
102
103      echo "==== Deploying ${APP_NAME} to Tomcat ===="
104
105      curl -v \
106      -u "${TOMCAT_USER}:${TOMCAT_PASSWORD}" \
107      -T target/*.war \
108      "${TOMCAT_URL}/manager/text/deploy?path=${APP_NAME}&update=true"
109
110      echo
111      echo "==== Tomcat Applications ===="
112
113      curl -s \
114      -u "${TOMCAT_USER}:${TOMCAT_PASSWORD}" \
```

Below the script editor, the 'Use Groovy Sandbox' checkbox is checked. A 'Pipeline Syntax' link is also present. The 'Advanced' section is partially visible at the bottom.

3. Important: use your actual Git repository

Your assignment gives:

https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

But your current work has been pushed to:

https://github.com/rajkumari-hub/sabear_simplecutomerapp.git

Therefore, because your Jenkins pipeline needs to use the repository containing your corrected code, I used:

git branch: 'feature-1.1',

url: 'https://github.com/rajkumari-hub/sabear_simplecutomerapp.git'

If your assignment specifically requires the original betawins repository, change that URL back.

4. Check your Jenkins credentials

You previously had this exact error:

ERROR: Could not find credentials entry with ID 'tomcat-credentials'

So **before running this job**, make sure this credential exists.

Go to:

Jenkins → Manage Jenkins → Credentials → System → Global credentials

Create:

Kind: Username with password

Username: jenkins

Password: Jenkins@123

ID: tomcat-credentials

Description: Tomcat deployment credentials

Your Tomcat server already confirmed these credentials work:

```
curl -u 'jenkins:Jenkins@123' \
```

```
http://172.31.30.131:8080/manager/text/list
```

and returned:

```
/sabear:running:0:sabear
```

So this credential configuration is correct.

5. Check Maven/Nexus configuration

Your pom.xml contains:

```
<distributionManagement>
```

```
  <snapshotRepository>
```

```
    <id>nexus</id>
```

```
    <url>http://172.31.30.117:8081/repository/sabear_simplecutomerapp</url>
```

```
  </snapshotRepository>
```

```
</distributionManagement>
```

Therefore Jenkins must have Nexus authentication configured for:

server ID = nexus

inside Maven settings.xml.

The Pipeline Maven Integration plugin's withMaven step configures Maven for Pipeline execution and can use Maven settings configured in Jenkins.

If your Nexus upload is already working from the previous job, **don't change it**.

6. Your WAR must contain the application

This is particularly important because you previously had the 404 problem.

Your current correct WAR showed:

index.html

Welcome.jsp

Customer.jsp

WEB-INF/web.xml

css/

images/

So after Maven compilation, Jenkins should show:

==== Generated WAR ====

-rw-r--r-- ... target/sabear.war

You can also verify from Jenkins:

```
unzip -l target/*.war | head -50
```

You should see things such as:

index.html

Welcome.jsp

Customer.jsp

WEB-INF/web.xml

images/

css/

7. Tomcat deployment path

Your deployment is:

WAR:

target/sabear.war

Tomcat:

172.31.30.131:8080

Context:

sabear

Therefore the deployment command becomes:

```
curl -u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
```

```
-T target/sabear.war \
```

```
"http://172.31.30.131:8080/manager/text/deploy?path=/sabear&update=true"
```

The resulting application URL is:

http://50.19.71.247:8080/sabear/

The important distinction is:

Tomcat server private IP

↓

172.31.30.131:8080

Public browser IP

↓

50.19.71.247:8080

Application context



/sabear

8. One correction to your stage order

Your assignment explicitly says:

Git Clone

SonarQube Integration

Maven Compilation

Nexus Artifactory

Slack Notification

Deploy On tomcat

The script above follows **exactly that order**.

However, in a production CI/CD pipeline, I would normally send the success Slack notification **after deployment**, because then Slack means the complete deployment succeeded.

But for your assignment, keep the order exactly as given.

9. What you should see in Jenkins

When you click **Build Now**, the Pipeline should show:

Git Clone



SonarQube Integration



Maven Compilation



↓

Nexus Artifactory

✓

↓

Slack Notification

✓

↓

Deploy On Tomcat

✓

At the end:

Finished: SUCCESS

And Tomcat should show:

/sabear:running:0:sabear

Then open:

<http://50.19.71.247:8080/sabear/>

← → ↻

⚠ Not secure

100.27.192.97:8080/job/sabear-simplecustomerapp-feature-1.1-...

🔍 ☆

Jenkins

/ sabear-simplecustomerap... / #2

🔍 ⚙️ 👤

+ echo ===== Tomcat Applications =====

===== Tomcat Applications =====

+ curl -f -u jenkins:**** http://172.31.30.131:8080/manager/text/list

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current	
			Dload	Upload	Total	Spent	Left	Speed
0	0	0	0	0	0	0	0	0
100	235	0	235	0	0	78281	0	117500

OK - Listed applications for virtual host [localhost]

/:running:0:ROOT

/examples:running:0:examples

/sabear:running:0:sabear

/host-manager:running:0:host-manager

/hire:running:0:hire

/manager:running:0:manager

/docs:running:0:docs

[Pipeline] }

[Pipeline] // withCredentials

[Pipeline] }

[Pipeline] // stage

[Pipeline] echo

===== PIPELINE SUCCESS =====

[Pipeline] slackSend

Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: rajkumaridevops, channel: #jenkins-alerts-notification, color: good, botUser: true, tokenCredentialId: slack-token, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

Finished: SUCCESS

← → ↻

⚠ Not secure

100.27.192.97:8080/job/sabear-simplecustomerapp-feature-1.1-scripted/

🔍 ☆

Jenkins

/ sabear-simplecustomerap...

🔍 ⚙️ 👤

Status

📁 Changes

🏗 Build Now

🗑 Delete Pipeline

⚙ Configure

✎ Rename

🔗 SonarQube

📅 Stages

📦 Maven

🔗 Pipeline Syntax

Builds >

🔍 Filter

Today

🟢 #3 4:28 PM

🟢 #2 4:24 PM

🔴 #1 3:09 PM

🟢 sabear-simplecustomerapp-feature-1.1-scripted

Last Successful Artifacts

SimpleCustomerApp-1.0-SNAPSHOT.jar 1.22 KIB

SimpleCustomerApp-1.0-SNAPSHOT.war 1.46 KIB

SonarQube Quality Gate

SimpleCustomerApp Passed

server-side processing: Success

Permalinks

- Last build (#2), 3 min 44 sec ago
- Last stable build (#2), 3 min 44 sec ago
- Last successful build (#2), 3 min 44 sec ago
- Last failed build (#1), 1 hr 18 min ago
- Last unsuccessful build (#1), 1 hr 18 min ago
- Last completed build (#2), 3 min 44 sec ago

Add description

RFCT API

Jenkins 2.568.2

sonatype nexus repository

Search components or CVEs...

Dashboard

Search

Browse

Upload

Malware Risk

Settings

Browse / seabear_simplecutomerapp

HTML View

Advanced search...

com

javatpoint

SimpleCustomerApp

SimpleCustomerApp-1.0-20260816.161604-1

SimpleCustomerApp-1.0-20260816.161604-1.pom

SimpleCustomerApp-1.0-20260816.161604-1.pom.md5

SimpleCustomerApp-1.0-20260816.161604-1.pom.sha1

SimpleCustomerApp-1.0-20260816.161604-1.war

SimpleCustomerApp-1.0-20260816.161604-1.war.md5

SimpleCustomerApp-1.0-20260816.161604-1.war.sha1

SimpleCustomerApp-1.0-20260816.162440-2

SimpleCustomerApp-1.0-20260816.162440-2.pom

SimpleCustomerApp-1.0-20260816.162440-2.pom.md5

SimpleCustomerApp-1.0-20260816.162440-2.pom.sha1

SimpleCustomerApp-1.0-20260816.162440-2.war

SimpleCustomerApp-1.0-20260816.162440-2.war.md5

SimpleCustomerApp-1.0-20260816.162440-2.war.sha1

SimpleCustomerApp-1.0-20260816.162831-3

SimpleCustomerApp-1.0-20260816.162831-3.pom

3.239.229.80:9000/projects

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

Keep your instance current and get the [latest SonarQube Community Build](#), available now.

The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

My Favorites All

Filters

Quality Gate

Passed 3

Failed 0

Security

0 info issues 2

1 low issue 0

1 medium issue 0

1 high issue 1

1 blocker issue 0

Reliability

SimpleCustomerApp Public

Last analysis: 3 minutes ago · 34 Lines of Code · XML

Passed

0 0 0 — — 0.0%

Security Reliability Maintainability Hotspots Reviewed Coverage Duplications

VProfile Public

Last analysis: 1 day ago · 1.6k Lines of Code · JSP, Java, ...

Passed

8 50 57 — 30.1% 2.6%

Security Reliability Maintainability Hotspots Reviewed Coverage Duplications

4 of 4 shown

SonarQube™ technology is powered by SonarSource Sàrl

Community Build · v26.7.0.124771 · MQR MODE

LGPL v3 Community Documentation Plugins Web API

50.19.71.247:8080/manager/html/list

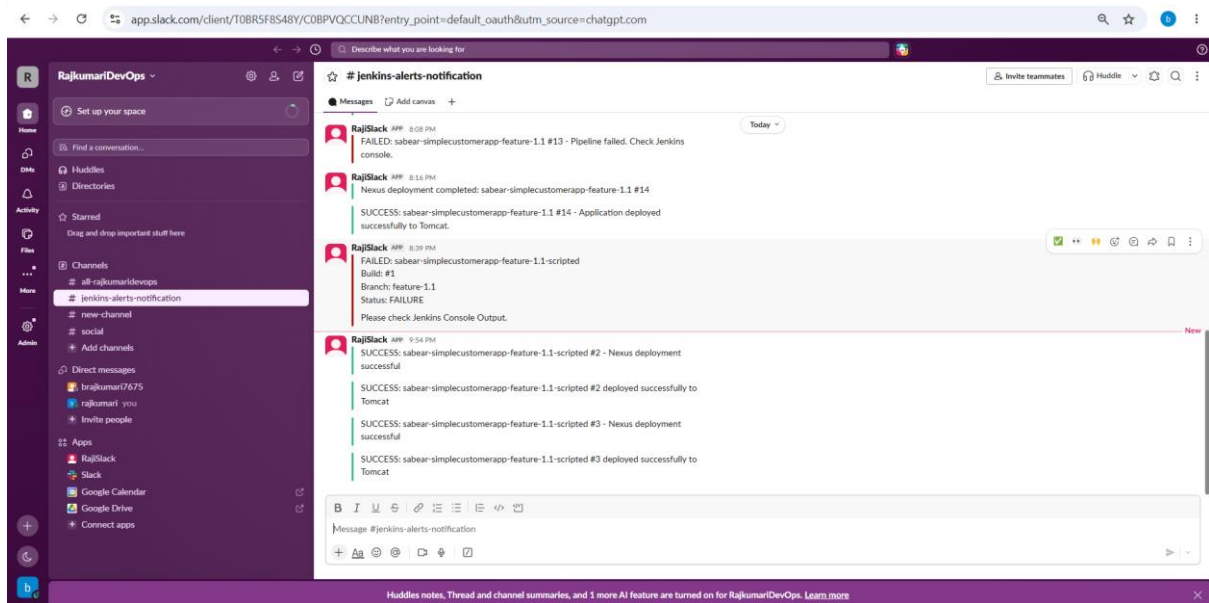
Tomcat Web Application Manager

Message: OK

Manager

List Applications HTML Manager Help Manager Help Server Status

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/thing	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/seabear	None specified		true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes



Step 8 — Run the pipeline

Click:

Build Now

The expected result is:

Git Clone ☒

SonarQube Integration ☒

Maven Compilation ☒

Nexus Artifactory ☒

Slack Notification ☒

Deploy On Tomcat ☒

At the end you should see:

===== PIPELINE SUCCESS =====

Finished: SUCCESS

4) Write sample skeleton of pipelines.

5) Create a parameterized job in Jenkins.

Source Code:

<https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

4) Write sample skeleton of pipelines

For this assignment, you can create **sample skeletons** without connecting them to a real application.

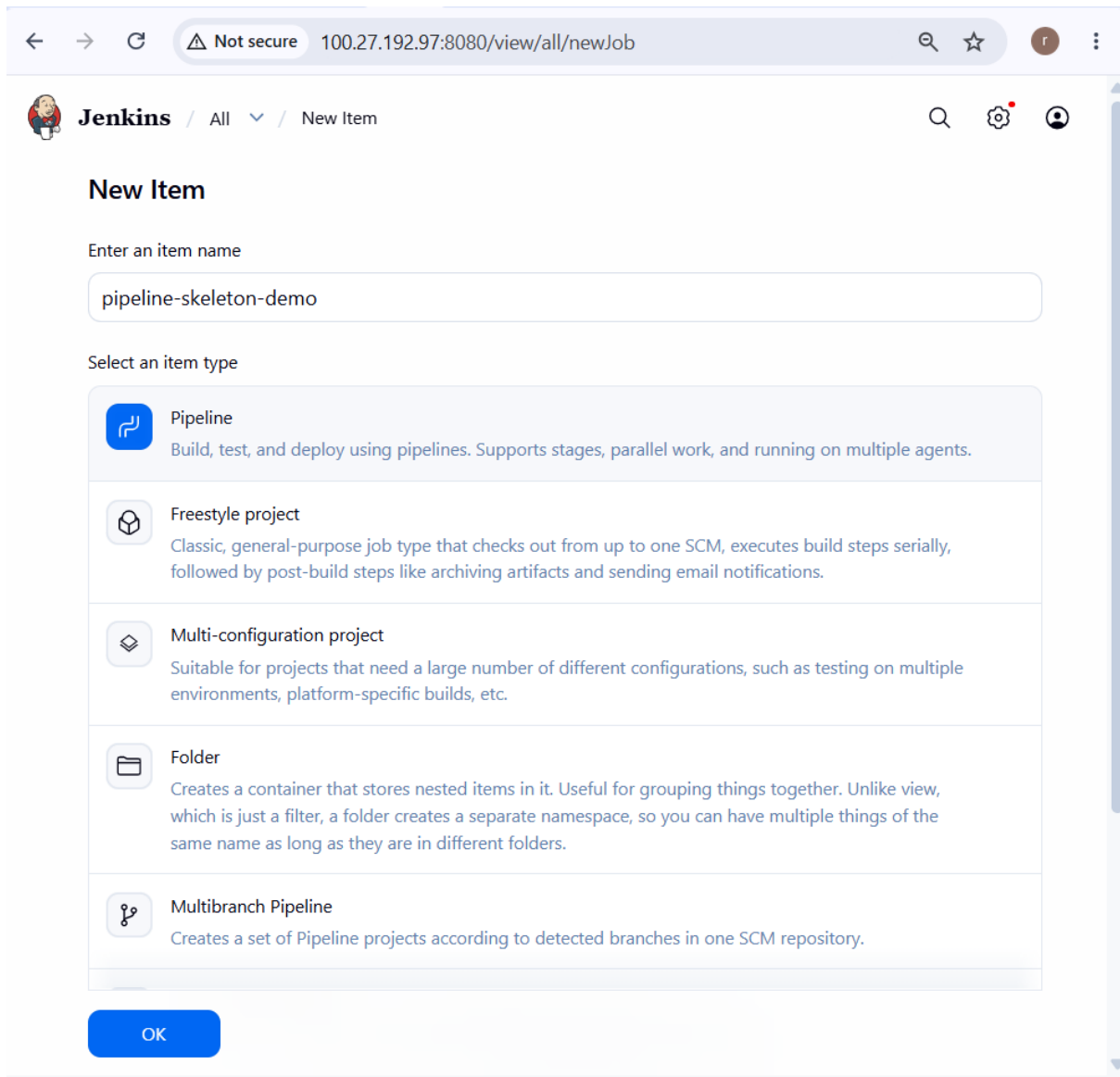
Create a Jenkins **Pipeline** job called:

pipeline-skeleton-demo

Go to:

Jenkins Dashboard → New Item → pipeline-skeleton-demo → Pipeline → OK

In the **Pipeline → Script** section, you can demonstrate the following.



A. Declarative Pipeline skeleton

```
pipeline {
```

```
    agent any
```

```
    stages {
```

```
        stage('Git Clone') {
```

```
    steps {
        echo 'Cloning source code...'
    }
}

stage('Build') {
    steps {
        echo 'Building application...'
    }
}

stage('Test') {
    steps {
        echo 'Running tests...'
    }
}

stage('Deploy') {
    steps {
        echo 'Deploying application...'
    }
}

post {
    success {
```

```

        echo 'Pipeline completed successfully'
    }

    failure {
        echo 'Pipeline failed'
    }
}

```

This demonstrates the standard Declarative structure:

pipeline

```

├── agent
├── stages
│   ├── stage
│   ├── stage
│   ├── stage
│   └── stage
└── post

```

Jenkins documents this as the basic Declarative Pipeline structure.

B. Scripted Pipeline skeleton

Since your previous assignment already used Scripted Pipeline, this is important for your interview/practical assignment.

```

node {

    stage('Git Clone') {

```

```
    echo 'Cloning source code...'
}
```

```
stage('Build') {
    echo 'Building application...'
}
```

```
stage('Test') {
    echo 'Running tests...'
}
```

```
stage('Deploy') {
    echo 'Deploying application...'
}
```

```
}
```

The basic difference is:

Declarative:

```
pipeline {
    agent any
    stages {
        ...
    }
}
```

Scripted:

```
node {
```

```
stage('Build') {  
    ...  
}  
}
```

Jenkins describes node as the fundamental Scripted Pipeline construct, while pipeline is specific to Declarative syntax.

C. Simple Maven Pipeline skeleton

You can also show a more realistic skeleton:

```
node {  
  
    stage('Git Clone') {  
        git branch: 'master',  
            url: 'https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git'  
    }  
  
    stage('Maven Build') {  
        sh 'mvn clean package'  
    }  
  
    stage('Test') {  
        sh 'mvn test'  
    }  
  
    stage('Deploy') {  
        echo 'Deploying application...'
```

```
}  
  
}
```

For your assignment, this is a good **sample skeleton** because it shows how a real Java/Maven application would flow through Jenkins.

1. Declarative Pipeline Skeleton

Create a Jenkins Pipeline job and use:

```
#!/usr/bin/env groovy
pipeline {
    agent any

    stages {
        stage('Git Clone') {
            steps {
                echo 'Cloning source code...'
            }
        }

        stage('Build') {
            steps {
                echo 'Building application...'
            }
        }

        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying application...'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully'
        }

        failure {
            echo 'Pipeline failed'
        }
    }
}
```


 Jenkins

pipeline-skeleton-demo

#1

Status

Changes

Console Output

Delete build '#1'

Pipeline Overview

Edit build information

Restart from Stage

Replay

Pipeline Steps

Workspaces

Console

Started by user [Rajkumari](#)

[Pipeline] Start of Pipeline

[Pipeline] node

Running on Jenkins in /var/lib/jenkins/workspace/pipeline-skeleton-demo

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Git Clone)

[Pipeline] echo

Cloning source code...

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Build)

[Pipeline] echo

Building application...

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Test)

[Pipeline] echo

Running tests...

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Deploy)

[Pipeline] echo

Deploying application...

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Declarative: Post Actions)

[Pipeline] echo

Pipeline completed successfully

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

Finished: SUCCESS

2. Scripted Pipeline Skeleton

This is the style you used for your **feature-1.1** assignment.

```
<</> groovy
node {

    stage('Git Clone') {
        echo 'Cloning source code...'
    }

    stage('Build') {
        echo 'Building application...'
    }

    stage('Test') {
        echo 'Running tests...'
    }

    stage('Deploy') {
        echo 'Deploying application...'
    }

    echo 'Pipeline completed'
}
```

[Status](#)[Changes](#)[Console Output](#)[Delete build '#2'](#)[Pipeline Overview](#)[Edit build information](#)[Replay](#)[Pipeline Steps](#)[Workspaces](#)[Previous Build](#)[Next Build](#)

Console

[Download](#)[Copy](#)[View](#)

```
Started by user Rajkumari
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Scripted Pipeline Skeleton
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] echo
Cloning source code...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running tests...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] echo
Pipeline completed
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

3. Pipeline with Maven

You can also show a practical Maven pipeline skeleton:

```
#!/usr/bin/groovy
pipeline {
    agent any

    tools {
        maven 'Maven'
    }

    stages {
        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/example/project.git'
            }
        }

        stage('Maven Build') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying application...'
            }
        }
    }
}
```

 **Jenkins** / Pipeline with Maven

Status

</> Changes

▶ Build Now

🗑 Delete Pipeline

⚙ Configure

✏ Rename

📋 Stages

🔗 Pipeline Syntax

✓ Pipeline with Maven

Permalinks

- Last build (#6), 2 min 48 sec ago
- Last stable build (#6), 2 min 48 sec ago
- Last successful build (#6), 2 min 48 sec ago
- Last failed build (#4), 5 min 6 sec ago
- Last unsuccessful build (#4), 5 min 6 sec ago
- Last completed build (#6), 2 min 48 sec ago

Builds >

Filter

Today

- ✓ #6 5:01 PM
- ✓ #5 5:01 PM
- ✗ #4 4:59 PM
- ✗ #3 4:58 PM
- ✗ #2 4:57 PM
- ✗ #1 4:55 PM

5) Create a Parameterized Jenkins Job

Now create a **separate Jenkins job** for this source code:

<https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

I recommend creating:

spring3-mvc-parameterized

Step 1 — Create the job

Go to:

Jenkins Dashboard → New Item

Enter:

spring3-mvc-parameterized

Select:

Pipeline

Click:

OK

Step 2 — Enable parameters

In the job configuration, find:

General

Enable:

☒ This project is parameterized

Then click:

Add Parameter → String Parameter

Create:

Parameter 1

Name: ENVIRONMENT

Default Value: dev

Description: Deployment environment

Add another:

Parameter 2

Name: BRANCH

Default Value: master

Description: Git branch to build

Add another:

Parameter 3

Name: MESSAGE

Default Value: Hello Jenkins

Description: Message to display during the build

Your parameters will look like:

ENVIRONMENT = dev

BRANCH = master

MESSAGE = Hello Jenkins

Jenkins parameters are supplied when triggering the build and are accessible through the params object.

Step 3 — Add the Pipeline

Scroll down to:

Pipeline → Definition

Select:

Pipeline script

Paste this **Scripted Pipeline**:

```
node {
```

```
stage('Parameters') {
```

```
    echo "==== BUILD PARAMETERS ===="
```

```
    echo "Environment : ${params.ENVIRONMENT}"
```

```
    echo "Branch      : ${params.BRANCH}"
```

```
    echo "Message     : ${params.MESSAGE}"
```

```
}
```

```
stage('Git Clone') {
```

```
    echo "==== GIT CLONE ===="
```

```
    git branch: "${params.BRANCH}",
```

```
        url: 'https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git'
```

```
}
```

```
stage('Build') {
```

```
    echo "==== MAVEN BUILD ===="
```

```
    sh 'mvn clean package -DskipTests'
```

```
}
```

```
stage('Test') {
```

```
    echo "===== MAVEN TEST ====="
```

```
    sh 'mvn test'
```

```
}
```

```
stage('Display Message') {
```

```
    echo "===== USER MESSAGE ====="
```

```
    echo "${params.MESSAGE}"
```

```
}
```

```
stage('Deployment') {
```

```
    echo "===== DEPLOYMENT ====="
```

```
    echo "Application will be deployed to ${params.ENVIRONMENT}"
```

```
}
```

```
echo "===== PIPELINE COMPLETED ====="
```

```
}
```

Save the job.

Step 4 — Build with Parameters

Now click:

Build with Parameters

You should get a screen similar to:

ENVIRONMENT [dev]

BRANCH [master]

MESSAGE [Hello Jenkins]

[Build]

For the first test, use:

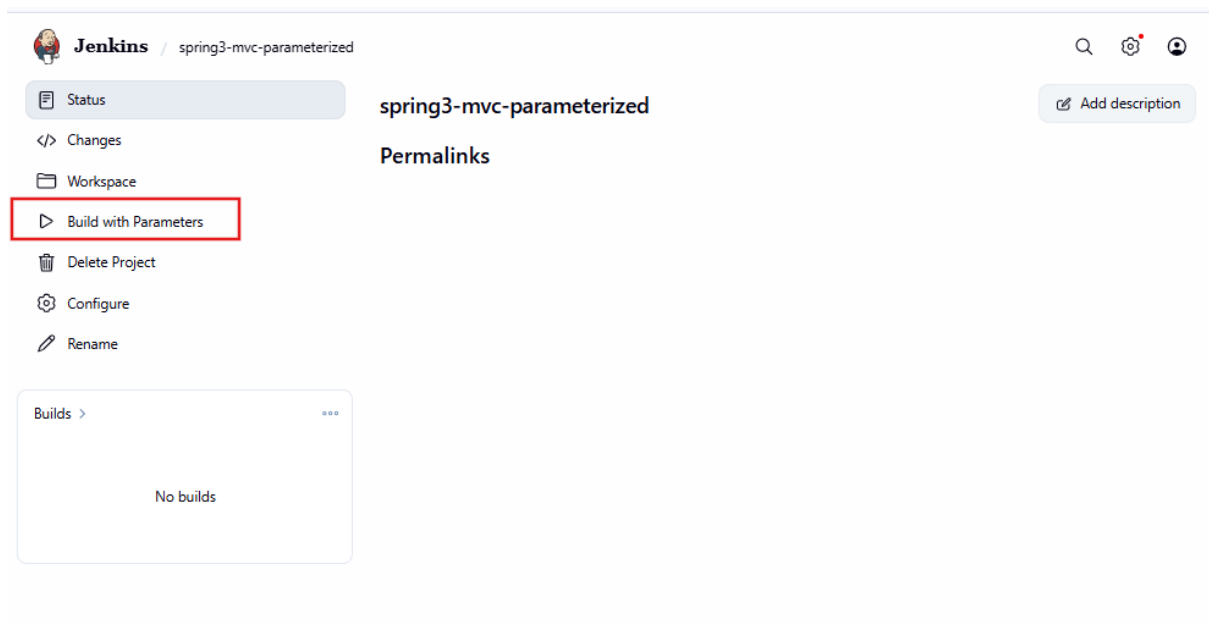
ENVIRONMENT = dev

BRANCH = master

MESSAGE = Testing parameterized Jenkins job

Click:

Build



Step 5.1: Run the parameterized job

Click:

Build with Parameters

You should see:

ENVIRONMENT

[dev]

Enter:

dev

Click:

Build

Jenkins / spring3-mvc-parameterized

Status

Changes

Workspace

Build with Parameters

Delete Project

Configure

Rename

Project spring3-mvc-parameterized

This build requires parameters:

ENVIRONMENT

Enter deployment environment

dev

Build Cancel

Builds >

No builds

Step 5 — Check Console Output

Go to:

Build #1 → Console Output

You should see:

===== BUILD PARAMETERS =====

Environment : dev

Branch : master

Message : Testing parameterized Jenkins job

===== GIT CLONE =====

===== MAVEN BUILD =====

===== MAVEN TEST =====

===== USER MESSAGE =====

Testing parameterized Jenkins job

===== DEPLOYMENT =====

Application will be deployed to dev

===== PIPELINE COMPLETED =====

Finished: SUCCESS

Important: check the repository branch first

Before you use master, verify what branches actually exist in the repository:

```
git ls-remote --heads https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git
```

If it shows:

refs/heads/master

then keep:

BRANCH = master

If it shows:

refs/heads/main

then use:

BRANCH = main

Better parameterized version for your assignment

If your instructor wants to see a **real parameterized Maven job**, I would use these parameters:

Parameter	Type	Example
BRANCH	String	master

Parameter	Type	Example
ENVIRONMENT	Choice	dev
MAVEN_GOAL	Choice	package
MESSAGE	String	Testing Jenkins

For ENVIRONMENT, you can provide:

dev

test

prod

For MAVEN_GOAL:

package

verify

install

Then your Maven stage can use:

```
stage('Maven Build') {
    sh "mvn clean ${params.MAVEN_GOAL} -DskipTests"
}
```

That demonstrates the actual purpose of parameterization: **the same Jenkins job can behave differently based on values supplied when the build starts.**

Your Assignment 4 + 5 final structure

Assignment 4

|

└── Sample Pipeline Skeleton

└── Declarative Pipeline

└── Scripted Pipeline

└── Maven Pipeline Skeleton

Assignment 5

|

└── spring3-mvc-parameterized

|

├── BRANCH

├── ENVIRONMENT

├── MESSAGE

|

├── Git Clone

├── Maven Build

├── Maven Test

├── Display Parameters

└── Deployment Skeleton

This is a clean way to demonstrate both assignments without interfering with your existing **sabear** → **SonarQube** → **Nexus** → **Slack** → **Tomcat** pipeline.

Step 8: Test another parameter

Run:

Build with Parameters

Enter:

ENVIRONMENT = test

Click **Build**.

Console should show:

Environment selected: test

Run it again with:

ENVIRONMENT = production

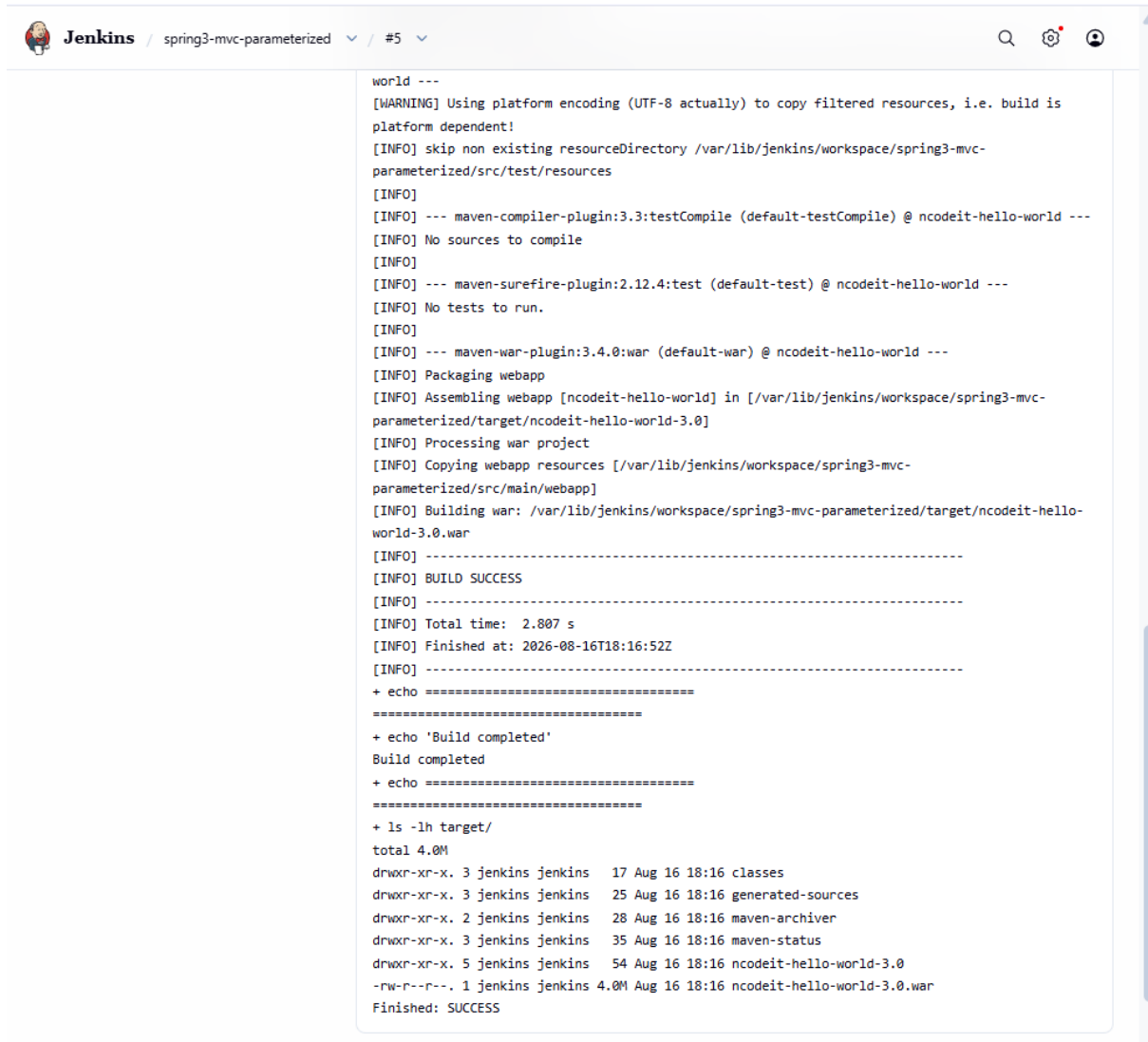
You should see:

Environment selected: production

This demonstrates that the Jenkins job is actually **parameterized**.

The screenshot shows the Jenkins web interface for a job named "spring3-mvc-parameterized". The interface includes a sidebar with navigation options: Status, Changes, Workspace, Build with Parameters, Delete Project, Configure, and Rename. The main content area displays the job's status as "Completed" with a green checkmark. Below this, there are "Permalinks" for various build types: Last build (#5), Last stable build (#4), Last successful build (#4), Last failed build (#2), Last unsuccessful build (#2), and Last completed build (#4). A "Builds" section is visible, showing a list of builds with their status (green checkmark for success, red X for failure) and timestamps. The builds are listed in descending order of time, with build #5 being the most recent and successful.

Build Number	Status	Timestamp
#5	Success	6:16 PM
#4	Success	6:15 PM
#3	Success	6:14 PM
#2	Failure	5:50 PM
#1	Failure	5:14 PM



The image shows a screenshot of the Jenkins web interface. At the top, the header displays the Jenkins logo, the job name 'spring3-mvc-parameterized', and the build number '#5'. The main content area shows the console output of the build process. The output includes various Maven lifecycle messages, such as warnings about platform encoding, information about skipping non-existing resource directories, and details about the compilation and packaging of the 'ncodeit-hello-world' project. The build concludes with a 'BUILD SUCCESS' message, a total time of 2.807 seconds, and a completion timestamp of 2026-08-16T18:16:52Z. Finally, an 'ls -lh target/' command is executed, showing the contents of the target directory, including files like 'classes', 'generated-sources', 'maven-archiver', 'maven-status', 'ncodeit-hello-world-3.0', and 'ncodeit-hello-world-3.0.war'.

```
world ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is
platform dependent!
[INFO] skip non existing resourceDirectory /var/lib/jenkins/workspace/spring3-mvc-
parameterized/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.3:testCompile (default-testCompile) @ ncodeit-hello-world ---
[INFO] No sources to compile
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ ncodeit-hello-world ---
[INFO] No tests to run.
[INFO]
[INFO] --- maven-war-plugin:3.4.0:war (default-war) @ ncodeit-hello-world ---
[INFO] Packaging webapp
[INFO] Assembling webapp [ncodeit-hello-world] in [/var/lib/jenkins/workspace/spring3-mvc-
parameterized/target/ncodeit-hello-world-3.0]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/spring3-mvc-
parameterized/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/spring3-mvc-parameterized/target/ncodeit-hello-
world-3.0.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.807 s
[INFO] Finished at: 2026-08-16T18:16:52Z
[INFO] -----
+ echo =====
=====
+ echo 'Build completed'
Build completed
+ echo =====
=====
+ ls -lh target/
total 4.0M
drwxr-xr-x. 3 jenkins jenkins 17 Aug 16 18:16 classes
drwxr-xr-x. 3 jenkins jenkins 25 Aug 16 18:16 generated-sources
drwxr-xr-x. 2 jenkins jenkins 28 Aug 16 18:16 maven-archiver
drwxr-xr-x. 3 jenkins jenkins 35 Aug 16 18:16 maven-status
drwxr-xr-x. 5 jenkins jenkins 54 Aug 16 18:16 ncodeit-hello-world-3.0
-rw-r--r--. 1 jenkins jenkins 4.0M Aug 16 18:16 ncodeit-hello-world-3.0.war
Finished: SUCCESS
```

6) Setup one slave machine for Jenkins.

Jenkins Official Documentation

Recommended

- Pipeline: <https://www.jenkins.io/doc/book/pipeline/>
- Environment Variables:
<https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#using-environment-variables>

- Parameters: <https://www.jenkins.io/doc/book/pipeline/syntax/#parameters>
- Handling credentials in Pipeline:
<https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#handling-credentials>

Part 1 — Create the Agent EC2

Step 1 — Open AWS

Go to:

AWS Console → EC2 → Instances → Launch instance

Use:

Setting	Value
Name	jenkins-agent
AMI	Amazon Linux 2023
Instance type	t2.micro or t3.micro
Key pair	Your existing key pair
Network	Same VPC as Jenkins
Subnet	Preferably same/public subnet
Auto-assign Public IP	Enabled

For your assignment, a small Amazon Linux instance is enough.

Part 2 — Security Group

Your agent must allow SSH from the Jenkins controller.

Go to the agent's:

Security → Security Groups → Inbound rules

Add:

Type: SSH

Protocol: TCP

Port: 22

Source: Jenkins Server Security Group

If you cannot reference the Jenkins security group, temporarily use:

Source: Jenkins Server private IP/32

For example:

172.31.x.x/32

Do not use 0.0.0.0/0 for SSH in a real environment.

The screenshot displays the AWS Management Console for the 'Instances' page. The left-hand navigation pane is open, showing the 'Instances' section under 'EC2'. The main area shows a list of instances. The table has columns for 'Name', 'Instance ID', 'Instance state', and 'Instance type'. Two instances are listed: 'Jenkins-Server' with ID 'i-0048f2c1d99b49893' and 'jenkins-agent' with ID 'i-0caf2c62afc1391d9'. Both are in a 'Running' state. Below the table, there are two summary cards: 'Instance alarm states' showing 'None' and 'Service health' showing 'Operating normally'.

Name	Instance ID	Instance state	Instance type
Jenkins-Server	i-0048f2c1d99b49893	Running	t3.sm
jenkins-agent	i-0caf2c62afc1391d9	Running	t3.mic

Part 3 — Connect to the New Agent

SSH into the new EC2:

```
ssh -i your-key.pem ec2-user@AGENT_PUBLIC_IP
```

Then become root:

```
sudo su -
```

Check hostname:

```
hostname
```

You should see your agent machine hostname.

Part 4 — Install Java on Agent

Jenkins agents require Java. Jenkins documentation describes agents as Java processes that connect to the controller.

Since you're already using Java 17 on your Jenkins server, use Java 17 on the agent too.

Run:

```
dnf install java-17-amazon-corretto -y
```

Check:

```
java -version
```

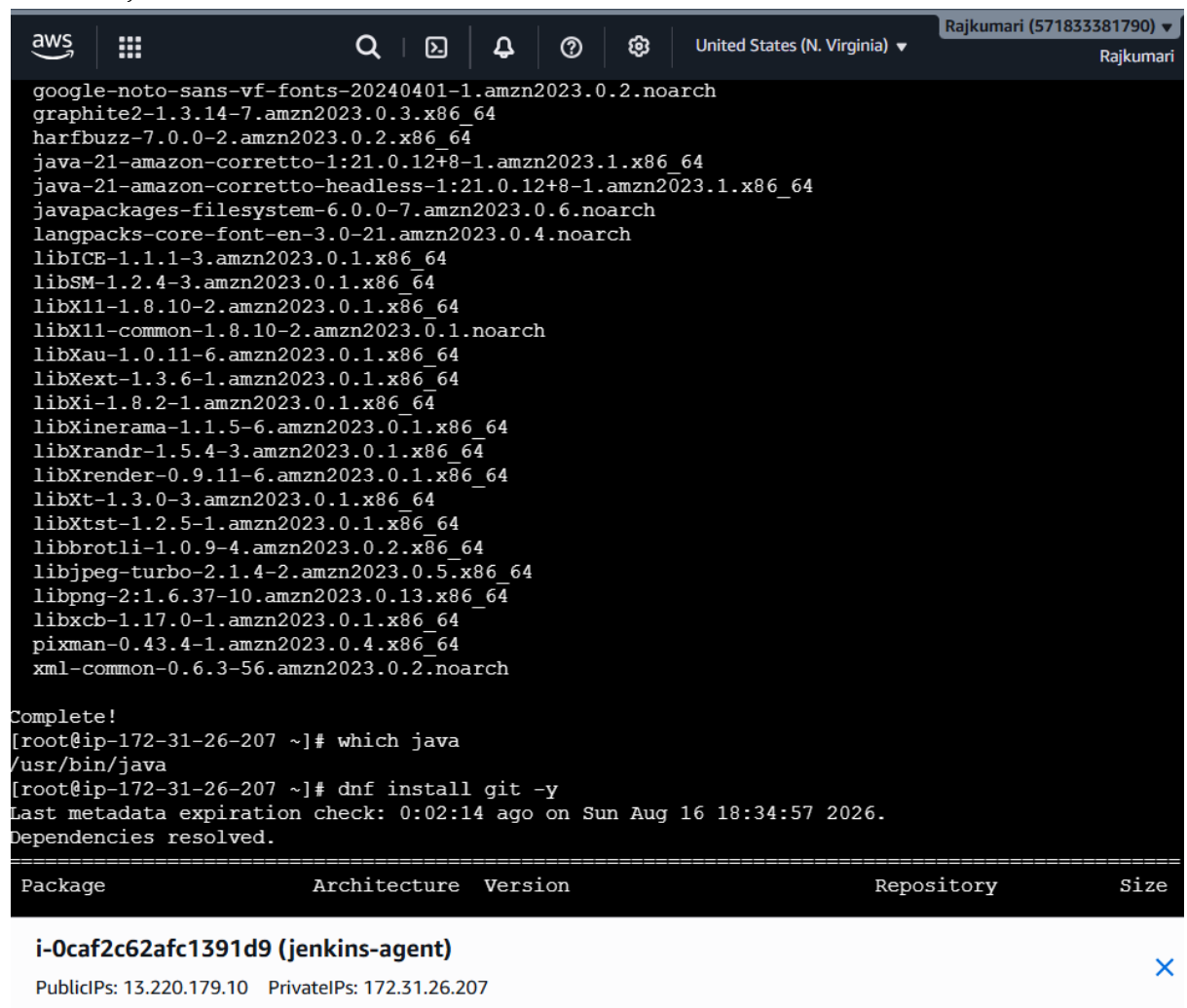
You should see Java 17.

Also check:

```
which java
```

For example:

```
/usr/bin/java
```



```
aws | [Icons] | [Search] | [Help] | [Settings] | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari
```

```
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
graphite2-1.3.14-7.amzn2023.0.3.x86_64
harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64
java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_64
javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
libICE-1.1.1-3.amzn2023.0.1.x86_64
libSM-1.2.4-3.amzn2023.0.1.x86_64
libX11-1.8.10-2.amzn2023.0.1.x86_64
libX11-common-1.8.10-2.amzn2023.0.1.noarch
libXau-1.0.11-6.amzn2023.0.1.x86_64
libXext-1.3.6-1.amzn2023.0.1.x86_64
libXi-1.8.2-1.amzn2023.0.1.x86_64
libXinerama-1.1.5-6.amzn2023.0.1.x86_64
libXrandr-1.5.4-3.amzn2023.0.1.x86_64
libXrender-0.9.11-6.amzn2023.0.1.x86_64
libXt-1.3.0-3.amzn2023.0.1.x86_64
libXtst-1.2.5-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
libpng-2:1.6.37-10.amzn2023.0.13.x86_64
libxcb-1.17.0-1.amzn2023.0.1.x86_64
pixman-0.43.4-1.amzn2023.0.4.x86_64
xml-common-0.6.3-56.amzn2023.0.2.noarch

Complete!
[root@ip-172-31-26-207 ~]# which java
/usr/bin/java
[root@ip-172-31-26-207 ~]# dnf install git -y
Last metadata expiration check: 0:02:14 ago on Sun Aug 16 18:34:57 2026.
Dependencies resolved.

=====
Package                                Architecture Version                               Repository                               Size
-----
i-0caf2c62afc1391d9 (jenkins-agent)
PublicIPs: 13.220.179.10  PrivateIPs: 172.31.26.207
```

Part 5 — Install Git

Run:

dnf install git -y

Verify:

git --version

```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [settings icon] | United States (N. Virginia) | Rajkumari (571833381790) | Rajkumari

-----
Total 44 MB/s | 7.9 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing : 1/1
  Installing : git-core-2.50.1-1.amzn2023.0.1.x86_64 1/8
  Installing : git-core-doc-2.50.1-1.amzn2023.0.1.noarch 2/8
  Installing : perl-lib-0.65-477.amzn2023.0.9.x86_64 3/8
  Installing : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64 4/8
  Installing : perl-File-Find-1.37-477.amzn2023.0.9.noarch 5/8
  Installing : perl-Error-1:0.17030-2.amzn2023.0.1.noarch 6/8
  Installing : perl-Git-2.50.1-1.amzn2023.0.1.noarch 7/8
  Installing : git-2.50.1-1.amzn2023.0.1.x86_64 8/8
Running scriptlet: git-2.50.1-1.amzn2023.0.1.x86_64 8/8
Verifying : git-2.50.1-1.amzn2023.0.1.x86_64 1/8
Verifying : git-core-2.50.1-1.amzn2023.0.1.x86_64 2/8
Verifying : git-core-doc-2.50.1-1.amzn2023.0.1.noarch 3/8
Verifying : perl-Error-1:0.17030-2.amzn2023.0.1.noarch 4/8
Verifying : perl-File-Find-1.37-477.amzn2023.0.9.noarch 5/8
Verifying : perl-Git-2.50.1-1.amzn2023.0.1.noarch 6/8
Verifying : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64 7/8
Verifying : perl-lib-0.65-477.amzn2023.0.9.x86_64 8/8

Installed:
git-2.50.1-1.amzn2023.0.1.x86_64 git-core-2.50.1-1.amzn2023.0.1.x86_64
git-core-doc-2.50.1-1.amzn2023.0.1.noarch perl-Error-1:0.17030-2.amzn2023.0.1.noarch
perl-File-Find-1.37-477.amzn2023.0.9.noarch perl-Git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64 perl-lib-0.65-477.amzn2023.0.9.x86_64

Complete!
[root@ip-172-31-26-207 ~]#
```

i-0caf2c62afc1391d9 (jenkins-agent)

PublicIPs: 13.220.179.10 PrivateIPs: 172.31.26.207



Part 6 — Install Maven

Because your agent will eventually perform builds, install Maven:

dnf install maven -y

Verify:

mvn -version

You should see something similar to:

Apache Maven 3.x

Java version: 17

```
aws | [grid icon] | [search icon] | [terminal icon] | [bell icon] | [help icon] | [settings icon] | United States (N. Virginia) ▼ | Rajkumari (571833381790) ▼ | Rajkumari
```

```
apache-commons-codec-1.15-6.amzn2023.0.3.noarch
apache-commons-io-1:2.8.0-7.amzn2023.0.5.noarch
apache-commons-lang3-3.18.0-1.amzn2023.0.1.noarch
atinject-1.0.5-3.amzn2023.0.3.noarch
cdi-api-2.0.2-6.amzn2023.0.3.noarch
google-guice-4.2.3-8.amzn2023.0.6.noarch
guava-31.0.1-3.amzn2023.0.6.noarch
httpcomponents-client-4.5.13-4.amzn2023.0.4.noarch
httpcomponents-core-4.4.13-6.amzn2023.0.4.noarch
jakarta-annotations-1.3.5-13.amzn2023.0.3.noarch
jansi-2.4.0-3.amzn2023.0.3.x86_64
java-17-amazon-corretto-devel-1:17.0.20+8-1.amzn2023.1.x86_64
java-17-amazon-corretto-headless-1:17.0.20+8-1.amzn2023.1.x86_64
jcl-over-slf4j-1.7.32-3.amzn2023.0.4.noarch
jsoup-1.16.1-4.amzn2023.0.2.noarch
jsr-305-3.0.2-5.amzn2023.0.4.noarch
maven-1:3.8.4-3.amzn2023.0.5.noarch
maven-amazon-corretto17-1:3.8.4-3.amzn2023.0.5.noarch
maven-lib-1:3.8.4-3.amzn2023.0.5.noarch
maven-resolver-1:1.7.3-3.amzn2023.0.4.noarch
maven-shared-utils-3.3.4-4.amzn2023.0.4.noarch
maven-wagon-3.4.2-6.amzn2023.0.4.noarch
plexus-cipher-1.8-3.amzn2023.0.3.noarch
plexus-classworlds-2.6.0-10.amzn2023.0.4.noarch
plexus-containers-component-annotations-2.1.0-9.amzn2023.0.4.noarch
plexus-interpolation-1.26-10.amzn2023.0.4.noarch
plexus-sec-dispatcher-2.0-3.amzn2023.0.3.noarch
plexus-utils-3.3.0-9.amzn2023.0.5.noarch
publicsuffix-list-20260116-1.amzn2023.0.1.noarch
sisu-1:0.3.4-9.amzn2023.0.4.noarch
slf4j-1.7.32-3.amzn2023.0.4.noarch

Complete!
[root@ip-172-31-26-207 ~]#
```

i-0caf2c62afc1391d9 (jenkins-agent) ✕

PublicIPs: 13.220.179.10 PrivateIPs: 172.31.26.207

Part 7 — Create Jenkins Agent User

This is a good practice because you shouldn't run Jenkins builds as root.

On the **agent server**:

```
useradd -m -s /bin/bash jenkins
```

Set a password:

```
passwd jenkins
```

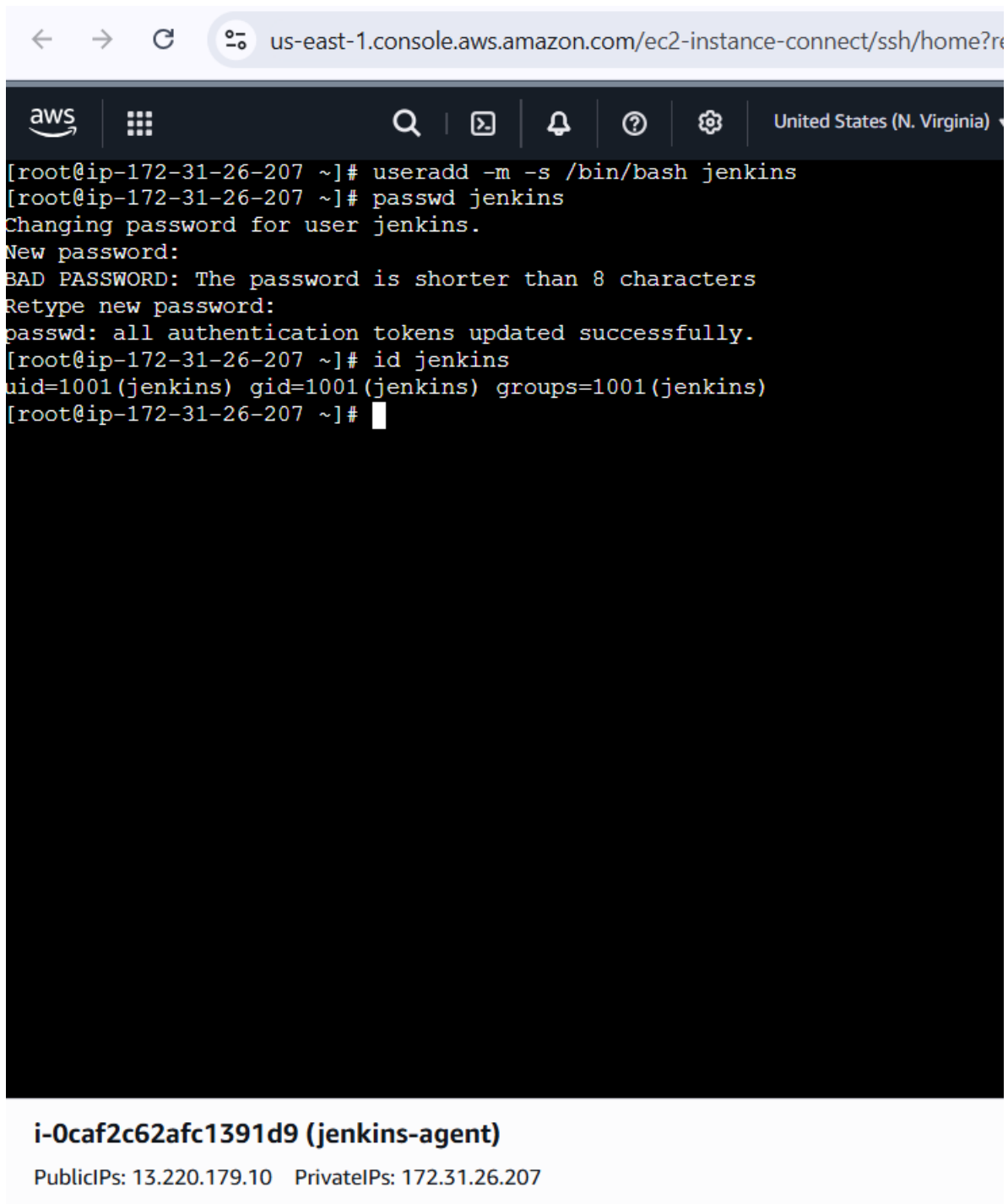
You can use a temporary password for this assignment.

Then:

```
id jenkins
```

You should see:

```
uid=... (jenkins)
```



The screenshot shows the AWS Management Console interface for an EC2 instance. The terminal window displays the following commands and output:

```
[root@ip-172-31-26-207 ~]# useradd -m -s /bin/bash jenkins
[root@ip-172-31-26-207 ~]# passwd jenkins
Changing password for user jenkins.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[root@ip-172-31-26-207 ~]# id jenkins
uid=1001(jenkins) gid=1001(jenkins) groups=1001(jenkins)
[root@ip-172-31-26-207 ~]#
```

Below the terminal window, the instance details are shown:

i-0caf2c62afc1391d9 (jenkins-agent)
PublicIPs: 13.220.179.10 PrivateIPs: 172.31.26.207

Part 8 — Prepare Agent Directory

Run:

```
mkdir -p /home/jenkins/agent
```

Change ownership:

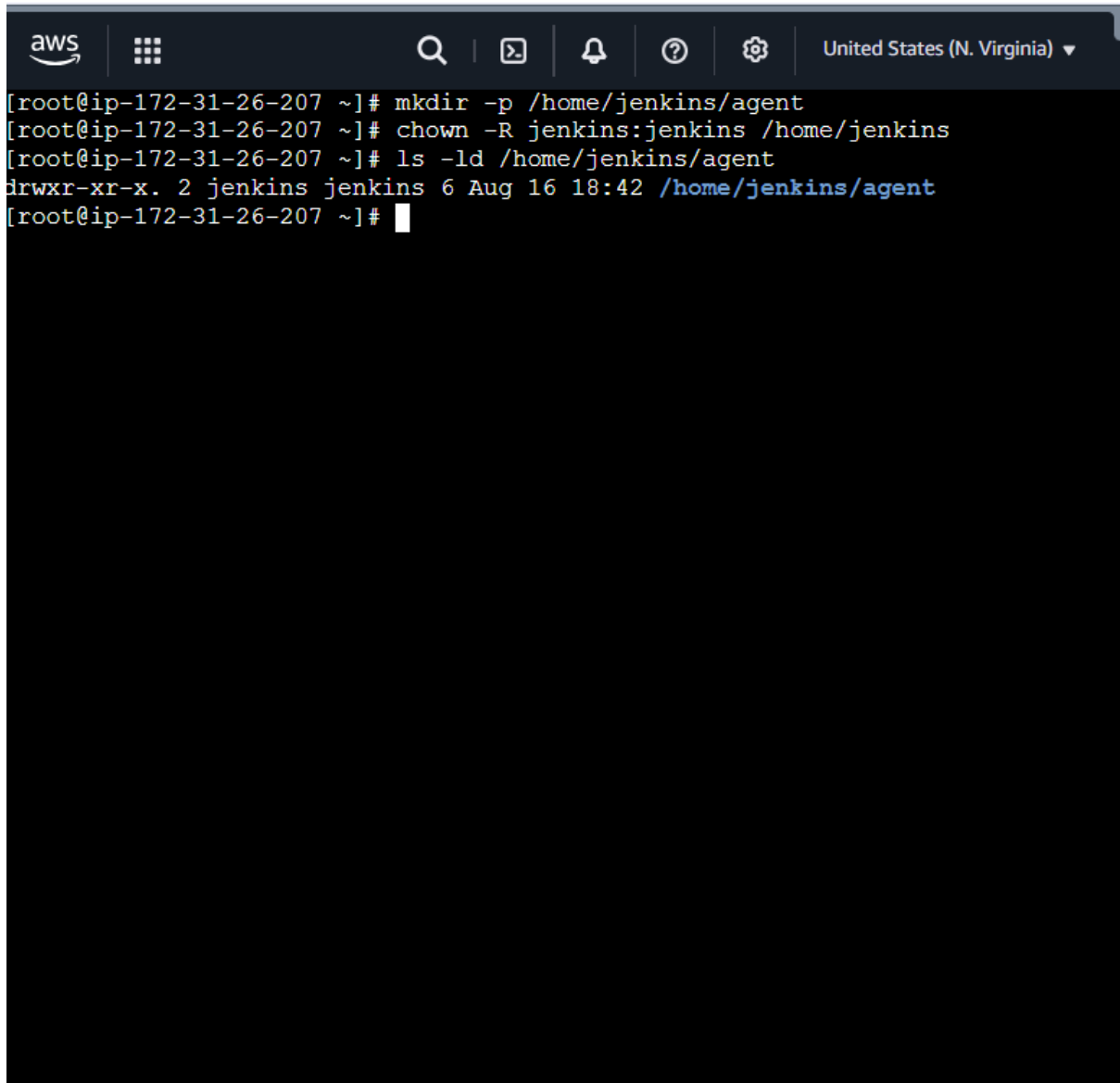
```
chown -R jenkins:jenkins /home/jenkins
```

Check:

```
ls -ld /home/jenkins/agent
```

It should show:

```
jenkins jenkins
```

A terminal window with a dark background and light-colored text. The terminal shows a series of commands and their outputs. The first command is 'mkdir -p /home/jenkins/agent'. The second is 'chown -R jenkins:jenkins /home/jenkins'. The third is 'ls -ld /home/jenkins/agent', which outputs 'drwxr-xr-x. 2 jenkins jenkins 6 Aug 16 18:42 /home/jenkins/agent'. The terminal window has a top bar with the AWS logo, a grid icon, a search icon, a terminal icon, a bell icon, a question mark icon, a gear icon, and a dropdown menu showing 'United States (N. Virginia)'.

i-0caf2c62afc1391d9 (jenkins-agent)

PublicIPs: 13.220.179.10 PrivateIPs: 172.31.26.207

Part 9 — Verify SSH Connectivity from Jenkins Controller

Now go to your **Jenkins controller server**.

You need to test:

```
ssh jenkins@AGENT_PRIVATE_IP
```

If this asks for a password, enter the password you created.

Jenkins Dashboard



Manage Jenkins



Nodes

Depending on your Jenkins version, this may appear as:

Manage Jenkins

→ Nodes

Jenkins' official agent setup follows the same basic process: create a new node, configure its remote root, labels, usage and launch method.

Part 11 — Create New Node

Click:

New Node

Enter:

jenkins-agent

Select:

Permanent Agent

Click:

Create

← → ↻ ⚠ Not secure 100.27.192.97:8080/manage/computer/new 🔍 ☆ 🌐 ⋮

Jenkins / Manage Jenkins ▾ / Nodes ▾ / New node 🔍 ⚙️ 👤

New node

Node name

Type

☒ **Permanent Agent**
Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

Create

Part 12 — Configure Node

You'll get the agent configuration page.

Name

jenkins-agent

Description

You can enter:

Jenkins build agent for DevOps assignments

Number of executors

Use:

1

One executor is a safe starting configuration for an agent.

Remote root directory

Enter:

/home/jenkins/agent

This is where Jenkins will keep its agent workspace/files.

Labels

Enter:

devops-agent

This is **very important**.

Your Pipeline can later specifically request:

```
agent {  
    label 'devops-agent'  
}
```

Usage

Select:

Only build jobs with label expressions matching this node

This makes it easy to demonstrate that the job is running specifically on your new agent.

 Jenkins

Manage Jenkins ▾ / Nodes ▾

🔍 ⚙️ 🌐

Name ?

jenkins-agent

Description ?

Jenkins build agent for DevOps task

Plain text Preview

Number of executors ?

1

Remote root directory ?

/home/jenkins/agent

Labels ?

devops-agent

Usage ?

Only build jobs with label expressions matching this node

Launch method ?

Launch agent by connecting it to the controller

Availability ?

Keep this agent online as much as possible

Node Properties

Save

Part 13 — Launch Method

For your assignment, use:

Launch agents via SSH

Then configure:

Host

Enter the **private IP address** of your agent.

For example:

172.31.20.100

Use your actual private IP.

Credentials

Click:

Add → Jenkins

You need credentials that allow Jenkins controller to SSH into the agent.

Part 14 — Create SSH Credentials

If you're using username/password for the assignment:

Select:

Kind:

Username with password

Username:

jenkins

Password:

<password you created on agent>

ID:

jenkins-agent-ssh

Description:

SSH credentials for Jenkins agent

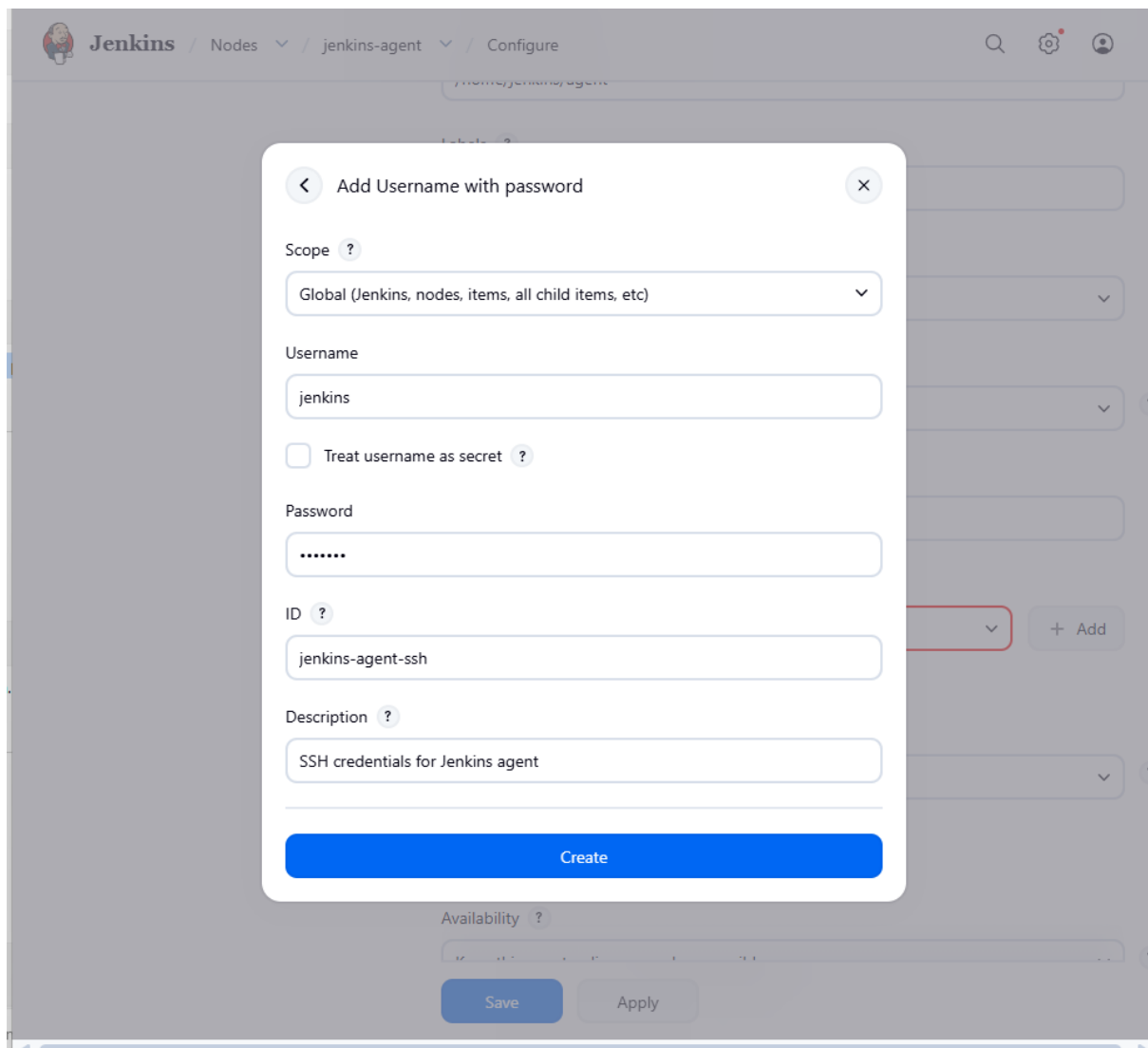
Click:

Add

Then select:

jenkins-agent-ssh

Jenkins supports storing credentials such as username/password and SSH private keys in its credential system rather than hard-coding them into jobs.



Part 15 — Host Key Verification

For a lab assignment, if you see:

Host Key Verification Strategy

you can select:

Non verifying Verification Strategy

However, for production environments, don't disable host-key verification. Proper host-key verification is safer.

Part 16 — Save

Click:

Save

Your node may initially show:

jenkins-agent

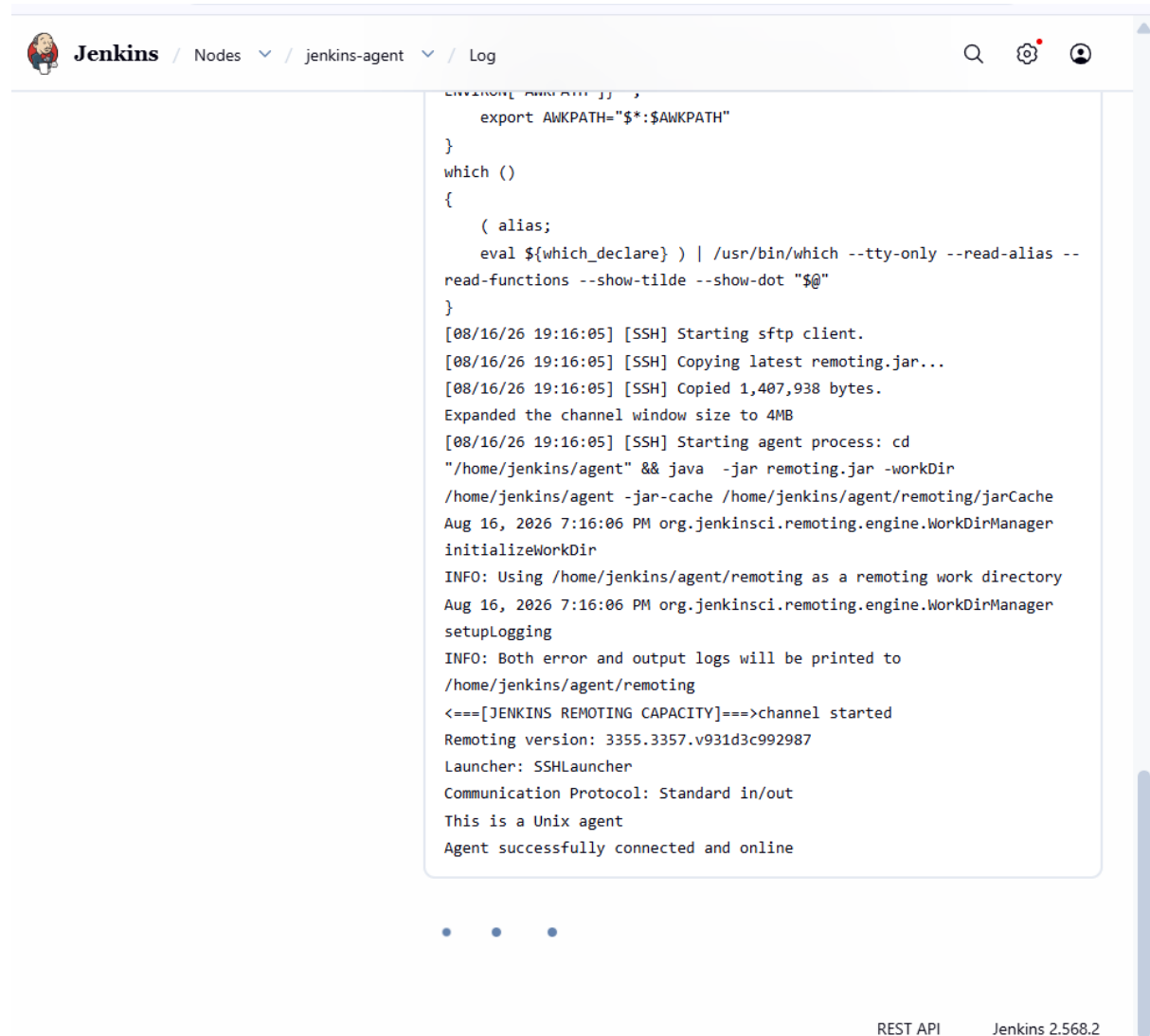
Offline

That's okay for a moment.

Jenkins should attempt to connect.

Official Jenkins documentation says a successfully launched agent should eventually show:

Agent successfully connected and online
in its log.



The screenshot shows the Jenkins web interface. The breadcrumb navigation at the top reads "Jenkins / Nodes / jenkins-agent / Log". The main content area displays a log for the "jenkins-agent" node. The log starts with shell commands for setting environment variables and then shows a series of SSH logs indicating the agent's startup process. Key log entries include: "[SSH] Starting sftp client.", "[SSH] Copying latest remoting.jar...", "[SSH] Copied 1,407,938 bytes.", "[SSH] Starting agent process: cd '/home/jenkins/agent' && java -jar remoting.jar -workDir /home/jenkins/agent -jar-cache /home/jenkins/agent/remoting/jarCache". This is followed by informational messages from the "org.jenkinsci.remoting.engine.WorkDirManager" and a final status message: "Agent successfully connected and online". The footer of the page shows "REST API" and "Jenkins 2.568.2".

```
ENVIRON[ AWKPATH ] ;
export AWKPATH="$*:AWKPATH"
}
which ()
{
    ( alias;
    eval ${which_declare} ) | /usr/bin/which --tty-only --read-alias --
read-functions --show-tilde --show-dot "$@"
}
[08/16/26 19:16:05] [SSH] Starting sftp client.
[08/16/26 19:16:05] [SSH] Copying latest remoting.jar...
[08/16/26 19:16:05] [SSH] Copied 1,407,938 bytes.
Expanded the channel window size to 4MB
[08/16/26 19:16:05] [SSH] Starting agent process: cd
"/home/jenkins/agent" && java -jar remoting.jar -workDir
/home/jenkins/agent -jar-cache /home/jenkins/agent/remoting/jarCache
Aug 16, 2026 7:16:06 PM org.jenkinsci.remoting.engine.WorkDirManager
initializeWorkDir
INFO: Using /home/jenkins/agent/remoting as a remoting work directory
Aug 16, 2026 7:16:06 PM org.jenkinsci.remoting.engine.WorkDirManager
setupLogging
INFO: Both error and output logs will be printed to
/home/jenkins/agent/remoting
<==[JENKINS REMOTING CAPACITY]==>channel started
Remoting version: 3355.3357.v931d3c992987
Launcher: SSHLauncher
Communication Protocol: Standard in/out
This is a Unix agent
Agent successfully connected and online
```

REST API Jenkins 2.568.2

Part 18 — Verify the Agent is Online

Go to:

Manage Jenkins → Nodes

You should see:

jenkins-agent

Online

You have now completed the basic **slave/agent setup**.

Part 19 — Create a Test Freestyle Job

Now we need to prove that the agent actually works.

Create:

Jenkins Dashboard → New Item

Name:

agent-test

Select:

Freestyle project

Click:

OK

 Jenkins / All ▾ / New Item 🔍 ⚙️

New Item

Enter an item name

agent-test

Select an item type

 Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

 Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

 Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.

OK

Part 20 — Restrict Job to Agent

Inside the job configuration, find:

General

Enable:

Restrict where this project can be run

Label Expression:

devops-agent

This tells Jenkins:

Run this job only on the node having the devops-agent label.

Jenkins' official agent tutorial uses the same concept: assign a label to the agent and restrict the job to that label.

The screenshot shows the Jenkins web interface for configuring a job named 'agent-test'. The breadcrumb navigation at the top reads 'Jenkins / agent-test / Configure'. On the left, a sidebar lists configuration sections: 'General' (selected), 'Source Code Management', 'Triggers', 'Environment', 'Build Steps', and 'Post-build Actions'. The main area is titled 'Configure' and contains a 'Plain text' field at the top. Below it, several options are listed with checkboxes and help icons: 'Discard old builds', 'GitHub project', 'This project is parameterized', 'Throttle builds', 'Execute concurrent builds if necessary', and 'Restrict where this project can be run'. The 'Restrict where this project can be run' option is checked. Below this, a 'Label Expression' field contains the text 'jenkins-agent'. A note below the field states: 'Label jenkins-agent matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.' At the bottom of the configuration section is an 'Advanced' dropdown menu. Below the configuration section is the 'Source Code Management' section, which includes a description: 'Connect and manage your code repository to automatically pull the latest code for your builds.' and two radio button options: 'None' (selected) and 'Git'. At the very bottom are two buttons: 'Save' and 'Apply'.

Part 21 — Add Execute Shell

Under:

Build Steps → Execute shell

Enter:

```
echo "=====
```

```
echo "JENKINS AGENT TEST"
```

```
echo "=====
```

```
echo "Node Name:"  
echo "$NODE_NAME"
```

```
echo "Workspace:"  
echo "$WORKSPACE"
```

```
echo "Java Version:"  
java -version
```

```
echo "Maven Version:"  
mvn -version
```

```
echo "Hostname:"  
hostname
```

```
echo "====="
```

Click:

Save

Jenkins / agent-test / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps**
- Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

[See the list of available environment variables](#)

```
echo "===== "  
echo "JENKINS AGENT TEST"  
echo "===== "  
  
echo "Node Name:"  
echo "$NODE_NAME"  
  
echo "Workspace:"  
echo "$WORKSPACE"  
  
echo "Java Version:"  
java -version  
  
echo "Maven Version:"  
mvn -version  
  
echo "Hostname:"  
hostname  
  
echo "===== "
```

Save **Apply**

Part 22 — Build the Test Job

Click:

Build Now

Open:

Console Output



Status



agent-test

📝 Add description

</> Changes

📁 Workspace

▶ Build Now

🗑 Delete Project

⚙ Configure

✎ Rename

Permalinks

- [Last build \(#2\), 15 sec ago](#)
- [Last stable build \(#2\), 15 sec ago](#)
- [Last successful build \(#2\), 15 sec ago](#)
- [Last completed build \(#2\), 15 sec ago](#)

Builds >

...



Filter



Today

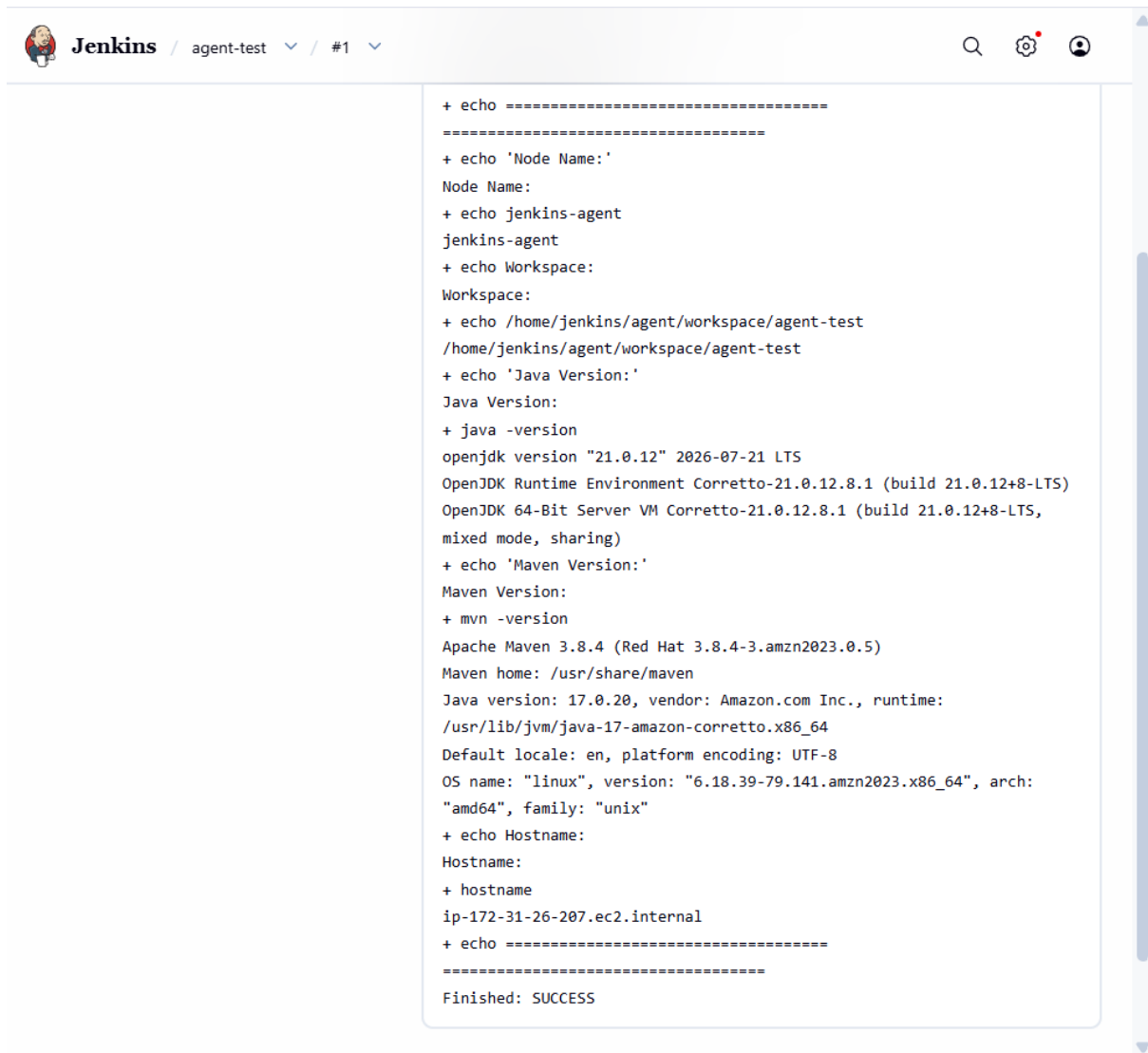


#2 7:30 PM



#1 7:28 PM





The screenshot shows the Jenkins web interface. At the top, the header includes the Jenkins logo, the text 'Jenkins', and navigation links for 'agent-test' and '#1'. On the right, there are icons for search, settings, and a user profile. The main area displays the console output of a shell script executed on a Jenkins agent. The output shows various system and environment details, including Node Name, Workspace, Java Version, Maven Version, and Hostname. The script concludes with 'Finished: SUCCESS'.

```
+ echo =====
+ echo 'Node Name:'
Node Name:
+ echo jenkins-agent
jenkins-agent
+ echo Workspace:
Workspace:
+ echo /home/jenkins/agent/workspace/agent-test
/home/jenkins/agent/workspace/agent-test
+ echo 'Java Version:'
Java Version:
+ java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS,
mixed mode, sharing)
+ echo 'Maven Version:'
Maven Version:
+ mvn -version
Apache Maven 3.8.4 (Red Hat 3.8.4-3.amzn2023.0.5)
Maven home: /usr/share/maven
Java version: 17.0.20, vendor: Amazon.com Inc., runtime:
/usr/lib/jvm/java-17-amazon-corretto.x86_64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.18.39-79.141.amzn2023.x86_64", arch:
"amd64", family: "unix"
+ echo Hostname:
Hostname:
+ hostname
ip-172-31-26-207.ec2.internal
+ echo =====
Finished: SUCCESS
```

Part 23 — Test With Your Maven Application

Now we can use your assignment project.

Create another job or modify the test job.

Use your repository:

<https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1.git>

Branch:

*/master

Restrict it to:

devops-agent

Then Execute Shell:

```
echo "Running build on:"
```

```
echo "$NODE_NAME"
```

```
java -version
```

```
mvn -version
```

```
mvn clean package
```

```
echo "WAR files:"
```

```
ls -lh target/*.war
```

This should produce:

```
[INFO] BUILD SUCCESS
```

and:

```
target/ncodeit-hello-world-3.0.war
```



📄 Status

</> Changes

📁 Workspace

▶ Build Now

🗑 Delete Project

⚙ Configure

✎ Rename

✅ agent-test

📝 Add description

Permalinks

- [Last build \(#3\), 1 min 14 sec ago](#)
- [Last stable build \(#3\), 1 min 14 sec ago](#)
- [Last successful build \(#3\), 1 min 14 sec ago](#)
- [Last completed build \(#3\), 1 min 14 sec ago](#)

Builds >

⋮

🔍 Filter

/

Today

✅	#4	7:33 PM		✖	▼
✅	#3	7:32 PM			▼
✅	#2	7:30 PM			▼
✅	#1	7:28 PM			▼

 Jenkins / agent-test ▾ / #3 ▾

Downloaded from central:
<https://repo.maven.apache.org/maven2/com/github/luben/zstd-jni/1.5.5-2/zstd-jni-1.5.5-2.jar> (5.9 MB at 16 MB/s)
Downloaded from central:
<https://repo.maven.apache.org/maven2/org/sonatype/sisu/sisu-inject-bean/1.4.2/sisu-inject-bean-1.4.2.jar> (153 kB at 419 kB/s)
Downloaded from central:
<https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-classworlds/2.2.3/plexus-classworlds-2.2.3.jar> (46 kB at 126 kB/s)
Downloaded from central:
<https://repo.maven.apache.org/maven2/org/sonatype/sisu/sisu-guice/2.1.7/sisu-guice-2.1.7-noaop.jar> (472 kB at 1.2 MB/s)
[INFO] Packaging webapp
[INFO] Assembling webapp [ncodeit-hello-world] in
[/home/jenkins/agent/workspace/agent-test/target/ncodeit-hello-world-3.0]
[INFO] Processing war project
[INFO] Copying webapp resources [/home/jenkins/agent/workspace/agent-test/src/main/webapp]
[INFO] Building war: /home/jenkins/agent/workspace/agent-test/target/ncodeit-hello-world-3.0.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 19.618 s
[INFO] Finished at: 2026-08-16T19:32:27Z
[INFO] -----
+ echo 'WAR files:'
WAR files:
+ ls -lh target/ncodeit-hello-world-3.0.war
-rw-rw-r--. 1 jenkins jenkins 4.0M Aug 16 19:32 target/ncodeit-hello-world-3.0.war
Finished: SUCCESS

Part 24 — Pipeline Test

Because your assignment also gives the Jenkins Pipeline documentation, you can demonstrate the agent with a Pipeline.

Create:

New Item → Pipeline

Name:

agent-pipeline

Under **Pipeline → Definition**, select:

Pipeline script

Use:


```
pipeline {
    agent {
        label 'devops-agent'
    }

    environment {
        APP_NAME = 'spring3-mvc'
        BUILD_ENV = 'dev'
    }

    parameters {
        choice(
            name: 'ENVIRONMENT',
            choices: ['dev', 'test', 'prod'],
            description: 'Select deployment environment'
        )

        string(
            name: 'VERSION',
            defaultValue: '3.0',
            description: 'Application version'
        )
    }

    stages {
```

```
stage('Agent Information') {  
    steps {  
        echo "Running on node: ${env.NODE_NAME}"  
        echo "Workspace: ${env.WORKSPACE}"  
        echo "Environment: ${params.ENVIRONMENT}"  
        echo "Version: ${params.VERSION}"  
    }  
}
```

```
stage('Git Clone') {  
    steps {  
        git branch: 'master',  
            url: 'https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-  
world-1.git'  
    }  
}
```

```
stage('Maven Build') {  
    steps {  
        sh 'java -version'  
        sh 'mvn -version'  
        sh 'mvn clean package'  
    }  
}
```

```
stage('Verify WAR') {
```

```
    steps {  
        sh 'ls -lh target/*.war'  
    }  
}  
  
post {  
    success {  
        echo 'Build completed successfully on Jenkins agent.'  
    }  
  
    failure {  
        echo 'Build failed.'  
    }  
}  
}
```

Jenkins Pipeline supports agent, environment, and parameters directly in Declarative Pipeline syntax. Parameters are available through params, while environment variables are available through env.

Jenkins / agent-pipeline

Search, Settings, User icons

Add description

agent-pipeline

Permalinks

- Last build (#1), 34 sec ago
- Last stable build (#1), 34 sec ago
- Last successful build (#1), 34 sec ago
- Last completed build (#1), 34 sec ago

Builds >

Filter

Today

#1 7:34 PM

Part 25 — Build Pipeline

Click:

Build with Parameters

Select:

ENVIRONMENT = dev

VERSION = 3.0

Click:

Build

Your console should show:

Running on node: jenkins-agent

Then:

Environment: dev

Version: 3.0

Then:

[INFO] BUILD SUCCESS

Finally:

Build completed successfully on Jenkins agent.

About the credentials assignment

Your supplied documentation also mentions:

Handling credentials in Pipeline.

For this assignment, don't put passwords or PATs directly into the Jenkinsfile. Jenkins recommends storing credentials in Jenkins and referring to them by credential ID.

For example:

```
environment {  
    GITHUB_CREDS = credentials('github-credentials')  
}
```

Then Jenkins exposes the appropriate credential variables to the Pipeline.

Do not do this:

```
GITHUB_PASSWORD = 'mypassword123'
```



- Status
- </> Changes
- ▶ Build with Parameters
- 🗑 Delete Pipeline
- ⚙ Configure
- ✎ Rename
- 📖 Stages
- ❓ Pipeline Syntax

Builds >

⋮

🔍 Filter



Today



#1 7:34 PM



Pipeline agent-pipeline

This build requires parameters:

ENVIRONMENT

Select deployment environment

dev



VERSION

Application version

3.0

▶ Build

Cancel



```
[INFO] -----  
-----  
[INFO] BUILD SUCCESS  
[INFO] -----  
-----  
[INFO] Total time: 2.979 s  
[INFO] Finished at: 2026-08-16T19:37:17Z  
[INFO] -----  
-----  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] stage  
[Pipeline] { (Verify WAR)  
[Pipeline] sh  
+ ls -lh target/ncodeit-hello-world-3.0.war  
-rw-rw-r--. 1 jenkins jenkins 4.0M Aug 16 19:37 target/ncodeit-hello-  
world-3.0.war  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] stage  
[Pipeline] { (Declarative: Post Actions)  
[Pipeline] echo  
Build completed successfully on Jenkins agent.  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] }  
[Pipeline] // withEnv  
[Pipeline] }  
[Pipeline] // node  
[Pipeline] End of Pipeline  
Finished: SUCCESS
```